

Rust and Candle for machine learning: the consolidated guide

One file that replaces the 35 notes under `candle_practice/docs/` written in September and October 2025. Every statement was re-checked on 19 September 2026, and again on 27 September 2026, against `rustc 1.98.1`, the candle version this project locks (0.9.1, git `f5838914`) and the current candle release (0.11.0). Every code block was compiled and run on both dates. Section 18 says exactly how.

Contents

0. How to read this guide	5
Part A: Rust you meet in every candle program	7
1. Results, <code>?</code> , and boxed errors	7
2. The turbofish and type inference	12
3. Tensors: shapes, types, devices, layout	14
Part B: Parameters and training	21
4. <code>Var</code> , <code>VarMap</code> and <code>VarBuilder</code>	21
5. Dot products, dense layers and the MLP	31
6. Softmax done safely, and cross-entropy	37
Part C: Convolutions	43
7. Convolution as a matrix, and its transpose	43
8. 1×1 convolutions	51
9. Grouped and depthwise convolutions	58
10. Dilated convolutions, receptive fields and dense prediction	61
Part D: Normalisation and channel attention	67
11. Batch normalisation	67
12. Fusing a convolution with its <code>BatchNorm</code>	72
13. Squeeze-and-Excitation	74
Part E: Attention	79
14. Attention, self-attention and multi-head attention	79
15. Positional encoding	89
16. YOLOv10 notes: where the pieces meet	91
Part F: Appendices	93
17. Error messages you will actually see	93
18. How this guide was verified	94
19. Where each old note went	96
20. Term index	97

0. How to read this guide

The running example: the café

Everything in this guide is explained with one small business: a café with a till and a camera above the counter. The till writes one line per sale. The camera takes a photo of the counter every few seconds. Over the guide we teach a computer to:

- predict whether the next hour will be busy from a few numbers (a dense network, section 5);
- look at a counter photo and say what is in it, pixel by pixel (convolutions, sections 7 to 10);
- decide which of its own internal "channels" matter for a given photo (Squeeze-and-Excitation, section 13);
- read a handwritten order slip such as "two large lattes, oat milk, no sugar" and work out which words belong together (attention, section 14);
- find every cup in the photo, fast (the YOLOv10 notes, section 16).

The café is only a way to keep the examples concrete. Nothing in the maths depends on it.

Conventions

- Every ````rust` block is a complete program. It was compiled and run as written.
- A ````text` block placed directly under a program is that program's exact standard output.
- A block whose first line is `// ✗ expected: ...` is meant to fail to compile, and the line says what the compiler reports. These blocks show you the trap, not the fix.
- A block whose first line is `// ~ output varies` prints random numbers, so its output is not reproduced.
- One block (section 2.3) carries a second line, `// ✓ on: candle 0.11.0`: it fails on 0.9.1 exactly as its first line says, and compiles and runs on 0.11.0; the output shown under it is the 0.11.0 output.
- "0.9.1" means candle-core and candle-nn 0.9.1 at git commit f5838914, which is what this project's Cargo.lock pins. "0.11.0" means the crates.io release of 26 June 2026. When they differ, the text says so. Everything else behaved identically on both.
- candle programs use `candle_core::Result<()>` as the return type of main unless the section is about a different error type.

The Cargo.toml the examples assume

```
[package]
name = "cafe"
version = "0.1.0"
edition = "2024"

[dependencies]
anyhow = "1"
candle-core = "0.11"
candle-nn = "0.11"
```

This project's own `Cargo.toml` points at the candle Git repository instead of crates.io and enables the `metal` feature for the Mac GPU. Its `Cargo.lock` pins commit `f5838914` (version 0.9.1). Because the manifest names no rev, a cargo update would move the project to the newest commit on candle's main branch; the differences that matter for this guide are listed in section 18.

Part A: Rust you meet in every candle program

1. Results, ?, and boxed errors

Absorbs 00_rust-result-error-guide.md.

1.1 Three ways a candle program says "this can fail"

Every candle operation returns a Result. You will meet three error types in practice:

Return type	Who uses it	What it can hold
<code>candle_core::Result<T> = Result<T, candle_core::Error></code>	candle itself	candle's own error enum: shape and type mismatches, missing tensors, a free-text Msg, a wrapped foreign error
<code>anyhow::Result<T></code>	most candle examples and applications	any error that is <code>Send + Sync + 'static</code> , plus an optional context message per layer
<code>Result<T, Box<dyn std::error::Error>></code>	plain standard-library code	any error type, on the heap, behind a trait object

They interoperate. The `?` operator converts a candle error into a `Box<dyn Error>` or into an `anyhow::Error` automatically: `Box<dyn Error>` implements `From` for every `'static` type that implements `std::error::Error`, and `anyhow::Error` does the same for every such type that is also `Send + Sync + 'static` (candle's error is).

1.2 What ? actually does

`expr?` on a `Result<T, E>` means: if it is `Ok(v)`, use `v`; if it is `Err(e)`, return `Err(From::from(e))` from the current function right now. The `From::from` is the important part: it lets a function that returns `Box<dyn Error>` call functions that return `std::io::Error`, `std::num::ParseIntError`, or `candle_core::Error`, and each error is boxed on the way out.

Here is the till. It reads a line of takings, and can fail in three different ways:

```
use std::error::Error;
use std::fmt;

#[derive(Debug)]
struct TillError { code: u32 }

impl fmt::Display for TillError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "till error code {}", self.code)
    }
}

impl Error for TillError {}

/// Reads one line of takings in pence. Three failure paths, three error types.
fn parse_takings(line: &str) -> Result<i64, Box<dyn Error>> {
    let pence: i64 = line.trim().parse()?; // ParseIntError, boxed by `?`
    if pence < 0 {
        return Err("takings cannot be negative".into()); // &str -> Box<dyn Error>
    }
    if pence > 1_000_000 {
        return Err(Box::new(TillError { code: 7 })); // our own type, boxed by hand
    }
    Ok(pence)
}

fn classify(e: &Box<dyn Error>) -> &'static str {
    if e.downcast_ref::<std::num::ParseIntError>().is_some() {
        "parse"
    } else if e.downcast_ref::<TillError>().is_some() {
        "till"
    } else {
        "other"
    }
}

fn main() -> Result<(), Box<dyn Error>> {
    for line in ["1250", " 42 ", "-5", "abc", "9999999"] {
        match parse_takings(line) {
            Ok(p) => println!("{line:?} -> Ok({p})",),
            Err(e) => println!("{line:?} -> Err({e}) [{e:?}] {e:?}", classify(&e)),
        }
    }
    let total = parse_takings("1250")? + parse_takings("42")?;
    println!("total {total}");
    Ok(())
}
```

```
"1250" -> Ok(1250)
" 42 " -> Ok(42)
"-5" -> Err(takings cannot be negative) [other] "takings cannot be negative"
"abc" -> Err(invalid digit found in string) [parse] ParseIntError { kind: InvalidDigit }
"9999999" -> Err(till error code 7) [till] TillError { code: 7 }
total 1292
```

Three things to notice:

- "...".into() works because the standard library implements From<&str> and From<String> for Box<dyn Error>. The string becomes an error whose Display is the text itself.

- `downcast_ref:::<T>()` asks "is the thing inside this box a T?" and gives you a reference if so. It is how you recover the concrete type when you must react differently to different failures. If you find yourself doing that a lot, define an error enum instead (section 1.6).
- When `main` returns `Err`, Rust prints `Error:` followed by the error's `Debug` form to standard error and exits with status 1. For a failed parse the last line is `Error: ParseIntError { kind: InvalidDigit }`. That is why `Box<dyn Error>` is comfortable for a `main` that only needs to report, not recover.

1.3 How big are these errors?

A `Box<dyn Error>` is a "fat pointer": one word for the address and one for the vtable (the table of method addresses that makes calls through `dyn` possible), so 16 bytes on a 64-bit machine.

`anyhow::Error` deliberately stores the vtable inside the allocation, so it is one word (8 bytes); the `anyhow` documentation describes it as "represented as a narrow pointer — exactly one word in size instead of two". `candle`'s own `Error` is a plain enum with many variants and inline data, and on this

toolchain it is 80 bytes, which means every `candle_core::Result<Tensor>` is 80 bytes too. None of this matters for correctness; it matters if you keep millions of `Results` around, and it explains why anyhow boxes.

```
use candle_core::{Device, Tensor};
use std::error::Error;

fn shape_bug() -> candle_core::Result<Tensor> {
    let a = Tensor::zeros((2, 3), candle_core::DType::F32, &Device::Cpu)?;
    let b = Tensor::zeros((2, 3), candle_core::DType::F32, &Device::Cpu)?;
    a.matmul(&b) // (2,3) x (2,3) cannot multiply
}

fn checked(pence: i64) -> candle_core::Result<i64> {
    if pence < 0 {
        candle_core::bail!("takings cannot be negative, got {pence}")
    }
    Ok(pence)
}

fn via_anyhow() -> anyhow::Result<()> {
    use anyhow::Context;
    let _ = shape_bug().context("while scoring the order slip");
    Ok(())
}

fn via_box() -> Result<(), Box<dyn Error>> {
    let _ = checked(-1)?;
    Ok(())
}

fn main() {
    println!(
        "sizes: candle Error {} | candle Result<Tensor> {} | anyhow::Error {} | Box<dyn\n
        Error> {}",
        std::mem::size_of::<candle_core::Error>(),
        std::mem::size_of::<candle_core::Result<Tensor>>(),
        std::mem::size_of::<anyhow::Error>(),
        std::mem::size_of::<Box<dyn Error>>(),
    );
    println!("candle Display : {}", shape_bug().unwrap_err());
    println!("bail! : {}", checked(-5).unwrap_err());
    println!("anyhow context : {:#}", via_anyhow().unwrap_err());
    println!("boxed : {}", via_box().unwrap_err());
    let e = shape_bug().unwrap_err();
    let as_std: &dyn Error = &e; // compiles only because candle's Error
    implements std::error::Error
    println!("source() of a candle error: {:?}", as_std.source().map(|s| s.to_string()));
}
```

```
sizes: candle Error 80 | candle Result<Tensor> 80 | anyhow::Error 8 | Box<dyn Error> 16
candle Display : shape mismatch in matmul, lhs: [2, 3], rhs: [2, 3]
bail! : takings cannot be negative, got -5
anyhow context : while scoring the order slip: shape mismatch in matmul, lhs: [2, 3], rhs: [2, 3]
boxed : takings cannot be negative, got -1
source() of a candle error: None
```

`candle_core::bail!("...")` is the quick way to return a free-text error from a function that returns `candle_core::Result`. It formats like `println!`. `candle` also has a `Context` trait (use `candle_core::Context`;) with the same `.context("...")` idea as `anyhow`, if you want to stay inside `candle`'s error type.

1.4 When the error must cross a thread

`Box<dyn Error>` on its own promises nothing about threads. If you move it into `std::thread::spawn`, the compiler refuses:

```
// ✗ expected: `dyn std::error::Error` cannot be sent between threads safely
use std::error::Error;

fn main() {
    let e: Box<dyn Error> = "x".into();
    std::thread::spawn(move || println!("{e}")).join().unwrap();
}
```

Add the bounds and it works. This is the form anyhow requires of every error it wraps, and it is why the `async` and `threaded` world writes `Box<dyn Error + Send + Sync>`:

```
use std::error::Error;

fn main() -> Result<(), Box<dyn Error + Send + Sync>> {
    let e: Box<dyn Error + Send + Sync> = "moved to a worker thread".into();
    let handle = std::thread::spawn(move || format!("worker saw: {e}"));
    println!("{}", handle.join().unwrap());
    let n: i32 = "7".parse()?;
    println!("{n}");
    Ok(())
}
```

```
worker saw: moved to a worker thread
7
```

1.5 Which one to use

- **A binary, a script, a notebook-style experiment:** `anyhow::Result<T>` for main and helpers. ? works on everything, `.context("what I was doing")` makes failures readable, and a backtrace is captured when `RUST_BACKTRACE=1` or `RUST_LIB_BACKTRACE=1` is set.
- **Code that only calls candle:** `candle_core::Result<T>` keeps the error type concrete and lets you use `bail!`. Most of candle's own examples do exactly this.
- **A library other people call:** define an error enum, derive `Display` with the `thiserror` crate, add `#[from]` conversions for the errors you wrap, and mark the enum `#[non_exhaustive]` so you can add variants later. Do not return `anyhow::Error` from a public library function; it forces every caller to depend on `anyhow` and hides the variants they might want to match on.
- **Do not match on error text.** `e.to_string().contains("negative")` breaks the day the message is reworded. Match on the variant or the downcast type.

1.6 A minimal error enum with thiserror

This is what the library form looks like. It is not compiled in this guide because `thiserror` is not one of the project's dependencies; it is here so you recognise the shape.

```
use thiserror::Error;

#[derive(Debug, Error)]
#[non_exhaustive]
pub enum TillError {
    #[error("could not read the till file")]
    Io(#[from] std::io::Error),
    #[error("bad number in the till file: {0}")]
    Parse(#[from] std::num::ParseIntError),
    #[error("takings cannot be negative, got {0}")]
    Negative(i64),
}
```

Each `#[from]` line makes `?` convert that error type into the matching variant, so the calling code reads exactly like the `Box<dyn Error>` version but keeps the type information.

2. The turbofish and type inference

Absorbs 00_turbo_fish.md.

2.1 Where the type comes from

Rust infers most types, but a generic function whose type parameter appears only in its return type gives the compiler nothing to go on. `str::parse` is the classic case: `"42".parse()` could produce an `i32`, a `u8`, an `f64`, or anything else that implements `FromStr`.

```
// ✗ expected: type annotations needed
fn main() {
    let x = "42".parse().unwrap();
    println!("{}", x + 1);
}
```

The compiler reports error `[E0284]: type annotations needed`. There are two fixes, and they are equivalent: name the type on the call with the "turbofish" `::<T>`, or annotate the variable and let inference flow backwards.

```
fn main() {
    let a = "42".parse::<i32>().unwrap();           // turbofish
    let b: i32 = "42".parse().unwrap();             // annotation on the binding
    let c = "42".parse::<u8>().unwrap();
    let d = "300".parse::<u8>();                     // 300 does not fit in a u8
    let e = "x".parse::<i32>();
    let v = (1..=4).map(|i| i * 10).collect::<Vec<i32>>();
    let w: Vec<u64> = (1..=4).map(|i| i * 10).collect();
    println!("{}", a + b + c + d + e + v + w);
}
```

```
42 42 42 Err(ParseIntError { kind: PosOverflow }) Err(ParseIntError { kind: InvalidDigit })
[10, 20, 30, 40] [10, 20, 30, 40]
```

The name comes from the shape `::<>`, which people say looks like a fish. `collect` is the other everyday case: it can build a `Vec`, a `HashMap`, a `String`, or a `Result` of any of those, so without a target type it fails with `E0283`:

```
// X expected: type annotations needed
fn main() {
    let v = (1..=4).map(|i| i * 10).collect();
    println!("{:?}", v);
}
```

2.2 candle's `to_vec1::<f32>()` and friends

candle keeps a tensor's element type as a run-time value (`DType`), not as a type parameter of `Tensor`. So when you copy a tensor's numbers out into ordinary Rust values, you must say what Rust type you want, and candle checks at run time that the tensor really holds that type:

Method	Tensor rank	Returns
<code>to_scalar::<T>()</code> or <code>to_vec0::<T>()</code>	0	<code>T</code>
<code>to_vec1::<T>()</code>	1	<code>Vec<T></code>
<code>to_vec2::<T>()</code>	2	<code>Vec<Vec<T>></code>
<code>to_vec3::<T>()</code>	3	<code>Vec<Vec<Vec<T>>></code>

`T` must implement candle's `WithDType` trait. On 0.9.1 that is `u8`, `u32`, `i64`, `f16`, `bf16`, `f32`, `f64` (and the 8-bit float type); 0.11.0 adds `i16` and `i32`. Both the rank and the dtype are checked when the call runs, not when it compiles:

```
use candle_core::{DType, Device, Tensor};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let t1 = Tensor::new(&[1.0f32, 2.0, 3.0, 4.0], &dev)?;
    let t2 = Tensor::new(&[[1.0f32, 2.0, 3.0], [4.0, 5.0, 6.0]], &dev)?;
    let t0 = Tensor::new(7.5f32, &dev)?;

    let v1 = t1.to_vec1::<f32>()?;           // turbofish
    let v2: Vec<Vec<f32>> = t2.to_vec2()?;    // annotation; same thing
    let s0 = t0.to_scalar::<f32>()?;
    println!("ranks {} {} {}: {v1:?} {v2:?} {s0}", t1.rank(), t2.rank(), t0.rank());

    println!("wrong dtype: {}", t1.to_vec1::<i64>().unwrap_err());
    println!("wrong rank : {}", t2.to_vec1::<f32>().unwrap_err());
    println!("scalar of a 1-D tensor: {}", t1.to_scalar::<f32>().unwrap_err());

    let ti = Tensor::new(&[[1i64, 2], [3, 4]], &dev)?;
    println!("{:?} {:?}", ti.dtype(), ti.to_vec2::<i64>()?);
    println!("cast first: {:?}", t2.to_dtype(DType::F64)?.to_vec2::<f64>()?);
    Ok(())
}
```

```
ranks 1 2 0: [1.0, 2.0, 3.0, 4.0] [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]] 7.5
wrong dtype: unexpected dtype, expected: I64, got: F32
wrong rank : unexpected rank, expected: 1, got: 2 ([2, 3])
scalar of a 1-D tensor: unexpected rank, expected: 0, got: 1 ([4])
I64 [[1, 2], [3, 4]]
cast first: [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]]
```

Leave the type out and you get the same inference failure as with `parse`:

```
// ✗ expected: type annotations needed
use candle_core::{Device, Tensor};

fn main() -> candle_core::Result<()> {
    let t = Tensor::new(&[[1.0f32, 2.0], [3.0, 4.0]], &Device::Cpu)?;
    let v = t.to_vec2()?;
    println!("{}", v.len());
    Ok(())
}
```

2.3 The i32 trap, and how it changed between versions

The old note said that calling `to_vec1::<i32>()` on an f32 tensor is a "Runtime Error". On the candle this project locks it is not even that: i32 does not implement `WithDType` on 0.9.1, so the program does not compile. The same applies to `Tensor::new(&[[1, 2], [3, 4]], ..)`, because an untyped integer literal defaults to i32. On 0.11.0 candle gained an I32 dtype, so both compile and produce an I32 tensor.

```
// ✗ expected: the trait `WithDType` is not implemented for `i32`
// ✓ on: candle 0.11.0
use candle_core::{Device, Tensor};

fn main() -> candle_core::Result<()> {
    let t = Tensor::new(&[[1, 2], [3, 4]], &Device::Cpu)?; // i32 literals
    println!("{}", t.dtype(), t.to_vec2::<i32>()?);
    Ok(())
}
```

```
I32 [[1, 2], [3, 4]]
```

Write `1i64` or `1u32` or `1.0f32` on the first literal and the rest of the array follows; that works on every version.

2.4 The Go analogy, in one paragraph

If you come from Go: `Vec<f32>` is `[]float32` that owns its memory; `Vec<Vec<f32>>` is `[][]float32` except that a tensor always gives you rectangular rows. The `turbofish` is a compile-time choice of type, like a type parameter, while `to_vec1::<f32>()` also performs a run-time check, like a type assertion `x.(float32)` with the `ok` bool folded into the `Result`. And `? is if err != nil { return err }` with the conversion done for you.

3. Tensors: shapes, types, devices, layout

Absorbs 01_structureOfMatrices.md, the tensor part of 2_attention_misc.md, and the memory-layout part of 00_turbo_fish.md.

3.1 Making tensors

A `Tensor` is a handle to a block of numbers on some device, plus a shape, a dtype and a layout. `Tensor::new` takes nested arrays; `Tensor::from_slice` takes a flat slice and a shape; `zeros`, `ones`, `full`, `arange` and `randn` build tensors from a shape. Shapes are written as tuples, or a single `usize`

for one dimension. Copying a Tensor with `.clone()` copies the handle, not the numbers.

```
use candle_core::{DType, Device, Tensor};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let a = Tensor::new(&[[1.0f32, 2.0, 3.0], [4.0, 5.0, 6.0]], &dev)?;
    let b = Tensor::from_slice(&[1u32, 2, 3, 4, 5, 6], (3, 2), &dev)?;
    let c = Tensor::zeros((2, 3), DType::F32, &dev)?;
    let d = Tensor::arange(0f32, 6.0, &dev)?.reshape((2, 3))?;
    let e = Tensor::full(0.5f32, (1, 3), &dev)?;
    println!("a {:?} {:?}", a.dims(), a.dtype());
    println!("b {:?} {:?}", b.dims(), b.dtype());
    println!("c {:?} elements {}", c.shape(), c.elem_count());
    println!("d {:?}", d.to_vec2:<f32>());
    println!("e {:?}", e.to_vec2:<f32>());
    let a2 = a.clone(); // a second handle to the same numbers
    println!("clone shares storage: same dims {:?}, is a variable: {}", a2.dims(),
            a2.is_variable());
    Ok(())
}
```

```
a [2, 3] F32
b [3, 2] U32
c [2, 3] elements 6
d [[0.0, 1.0, 2.0], [3.0, 4.0, 5.0]]
e [[0.5, 0.5, 0.5]]
clone shares storage: same dims [2, 3], is a variable: false
```

The `is_variable` flag matters in section 4: an ordinary tensor is immutable, and only a `Var` (a tensor created as a variable) can be overwritten in place.

3.2 The rows-are-items convention, and what a Linear layer computes

`candle` follows the same convention as `PyTorch`: the first dimension counts items. A batch of 128 café photos flattened to 784 numbers each is a tensor of shape `(128, 784)`. A hidden layer with 20 neurons that reads those 784 numbers stores its weights as `(20, 784)`: one row per neuron, one column per input. To apply it, you multiply the inputs by the transpose of the weights:

```
inputs (128, 784) @ weights.t() (784, 20) = outputs (128, 20)
```

This is what `candle_nn::Linear` does. Its documentation calls the layer $y = x@w.t() + b$; the weight is created with shape `(out_dim, in_dim)`; and the bias, of shape `(out_dim,)`, is added to every row with `broadcast_add`. The code below builds a `Linear` by hand and checks the arithmetic:

```
use candle_core::{Device, Module, Tensor};
use candle_nn::Linear;

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let w = Tensor::new(&[[1.0f32, 2.0], [3.0, 4.0], [5.0, 6.0]], &dev)?; // 3 neurons, 2
        inputs
    let b = Tensor::new(&[0.5f32, 0.0, -1.0], &dev)?;
    let layer = Linear::new(w.clone(), Some(b.clone()));

    let x = Tensor::new(&[[10.0f32, 100.0]], &dev)?; // one item, 2
        inputs
    println!("layer : {:?}", layer.forward(&x)?.to_vec2:::<f32>()?);
    println!("by hand: {:?}", x.matmul(&w.t())?.broadcast_add(&b)?.to_vec2:::<f32>()?);

    let seq = Tensor::new(&[[[10.0f32, 100.0], [1.0, 1.0]]], &dev)?; // (batch 1, 2 items,
        2 inputs)
    println!("3-D input: {:?}", layer.forward(&seq)?.to_vec3:::<f32>()?);

    let wrong = Tensor::new(&[[1.0f32, 2.0, 3.0]], &dev)?;
    println!("wrong width: {}", layer.forward(&wrong).unwrap_err());
    Ok(())
}
```

```
layer : [[210.5, 430.0, 649.0]]
by hand: [[210.5, 430.0, 649.0]]
3-D input: [[[210.5, 430.0, 649.0], [3.5, 7.0, 10.0]]]
wrong width: shape mismatch in matmul, lhs: [1, 3], rhs: [2, 3]
```

Neuron 1 computed $10 \cdot 1 + 100 \cdot 2 + 0.5 = 210.5$. A 3-D input (batch, items, features) also works: `Linear` applies the same weights to every item of every batch element.

3.3 No silent broadcasting

This is the rule that breaks the most copied code. In candle, $a * b$, $a + b$, $a.mul(\&b)$, $a.sub(\&b)$ and so on require the two shapes to be identical. If you want NumPy-style broadcasting (a (2, 64, 1, 1) gate scaled across a (2, 64, 32, 32) image), you must ask for it with `broadcast_mul`, `broadcast_add`, `broadcast_sub`, `broadcast_div`. The only exception is an f64 scalar on either side of the operator: $t * 2.0$, $2.0 * \&t$, $t + 1.0$, $1.0 - \&t$, $t / 3.0$ all apply to every element.

```
use candle_core::{Device, Tensor};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let image = Tensor::new(&[[[1.0f32, 2.0], [3.0, 4.0]], [[10.0, 20.0], [30.0, 40.0]]],
        &dev)?; // (1, 2, 2, 2)
    let gate = Tensor::new(&[[[0.5f32]], [[2.0]]], &dev)?;
        // (1, 2, 1, 1)
    println!("plain * : {}", (&image * &gate).unwrap_err());
    println!("broadcast: {:?}", image.broadcast_mul(&gate)?.flatten_all()?.to_vec1:::<f32>
        ());
    println!("scalar : {:?}", (&image * 2.0)?.flatten_all()?.to_vec1:::<f32>());
    println!("scalar on the left: {:?}", (1.0 - &image)?.flatten_all()?.to_vec1:::<f32>());
    println!("same shape works: {:?}", (&image + &image)?.flatten_all()?.to_vec1:::<f32>());
    Ok(())
}
```

```
plain * : shape mismatch in mul, lhs: [1, 2, 2, 2], rhs: [1, 2, 1, 1]
broadcast: [0.5, 1.0, 1.5, 2.0, 20.0, 40.0, 60.0, 80.0]
scalar : [2.0, 4.0, 6.0, 8.0, 20.0, 40.0, 60.0, 80.0]
scalar on the left: [0.0, -1.0, -2.0, -3.0, -9.0, -19.0, -29.0, -39.0]
same shape works: [2.0, 4.0, 6.0, 8.0, 20.0, 40.0, 60.0, 80.0]
```

The broadcasting rule itself is the usual one: shapes are aligned from the right, and a dimension of size 1 stretches to match. Section 12 shows a case where that right-alignment silently multiplies the wrong axis when you forget to reshape.

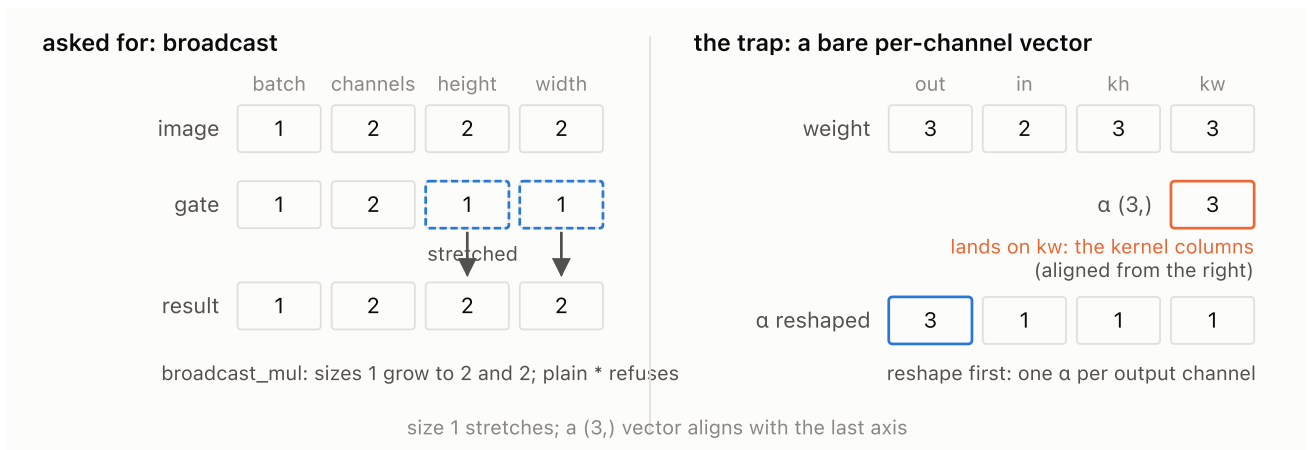


Figure 1. Broadcasting lines shapes up from the right, so the (1, 2, 1, 1) gate stretches over every pixel of each channel of the counter photo, while a bare (3,) vector would land on a kernel's columns unless it is reshaped to (3, 1, 1, 1) first.

3.4 Views, contiguous(), and turning an image into a sequence

`transpose`, `t()`, `narrow` and `squeeze` do not move numbers. They return a new handle with different strides over the same storage. Such a tensor is "non-contiguous". Most operations accept it; a few (for example `matmul` on some backends and any custom kernel that assumes a plain layout) want `.contiguous()`, which copies the numbers into row-major order.

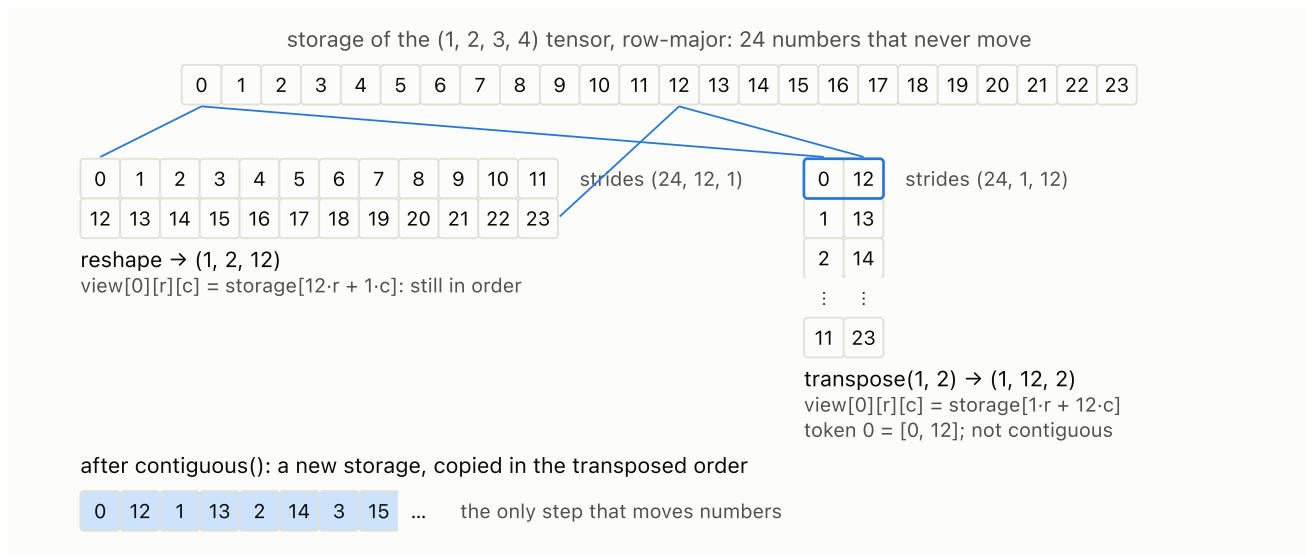


Figure 2. The 24 numbers of the (1, 2, 3, 4) photo never move: reshape regroups them as (1, 2, 12) with strides (24, 12, 1), transpose(1, 2) reads the same storage as (1, 12, 2) with strides (24, 1, 12), and only contiguous() copies them into a new order.

The transform below turns a batch of counter photos (*b*, *c*, *h*, *w*) into a sequence of *h*·*w* tokens with *c* features each, which is exactly the shape an attention layer wants (section 14).

flatten_from(2) merges every dimension from index 2 onwards:

```
use candle_core::{Device, Tensor};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let xs = Tensor::arange(0f32, 24.0, &dev)?.reshape((1, 2, 3, 4))?; // (b, c, h, w)
    let (b, c, h, w) = xs.dims4()?;
    let long = xs.reshape((b, c, h * w)).transpose(1, 2)?; // (b, h*w, c)
    let short = xs.flatten_from(2)?.transpose(1, 2)?; // the same,
    // idiomatically
    let same = (&long - &short)?.abs()?.max_all()?.to_scalar:::<f32>()? == 0.0;
    println!("{:?} -> {:?}; identical: {same}", xs.dims(), long.dims());
    println!("contiguous before: {}, after .contiguous(): {}", long.is_contiguous(),
        long.contiguous()?.is_contiguous());
    println!("token 0 = pixel (0,0) of every channel: {:?}", long.contiguous()?.to_vec3:::<f32>()?[0][0]);
    println!("dims4 on a 3-D tensor: {}", long.dims4().unwrap_err());
    Ok(())
}
```

```
[1, 2, 3, 4] -> [1, 12, 2]; identical: true
contiguous before: false, after .contiguous(): true
token 0 = pixel (0,0) of every channel: [0.0, 12.0]
dims4 on a 3-D tensor: unexpected rank, expected: 4, got: 3 ([1, 12, 2])
```

Token 0 holds the value at pixel (0, 0) of channel 0 (which is 0) and of channel 1 (which is 12), so each token really is "one pixel, all channels".

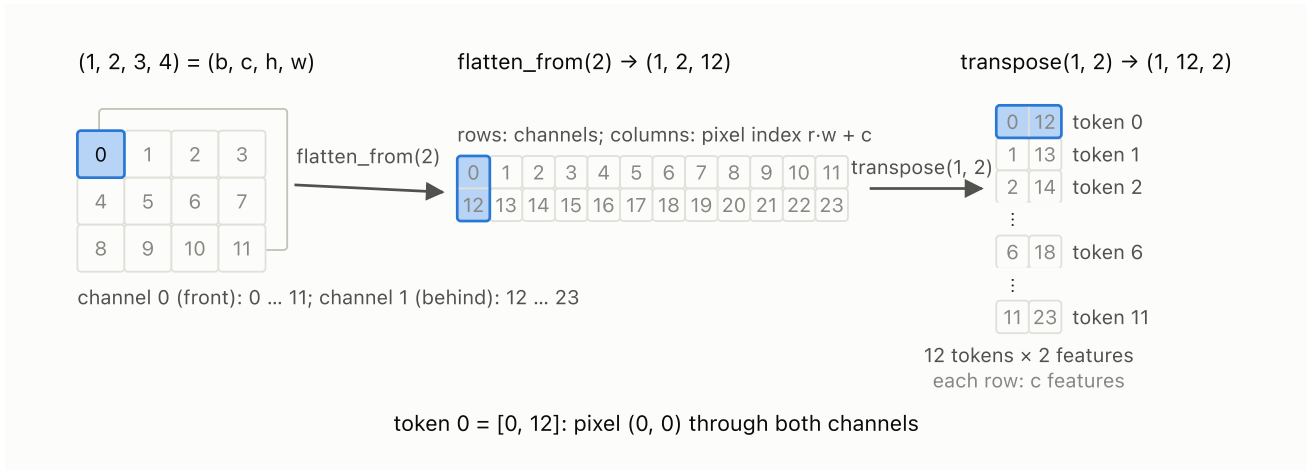


Figure 3. `flatten_from(2).transpose(1, 2)` turns the (1, 2, 3, 4) counter photo into 12 tokens of 2 features each; token 0 is pixel (0, 0) seen through both channels, the values 0 and 12.

3.5 Devices

`Device::Cpu` always exists. `Device::cuda_if_available(0)` gives you the first CUDA GPU when candle was built with the `cuda` feature and one is present, and falls back to the CPU otherwise. `Device::new_metal(0)` is the Mac GPU and needs the `metal` feature; without that feature the call still compiles but returns an error at run time. Tensors and layers must live on the same device; `to_device` moves a tensor. This project's `main.rs` uses `Device::new_metal(0)?` and enables the feature in `Cargo.toml`, so it runs on the Mac GPU.

```
// ~ output varies (depends on the machine and on the features candle was built with)
use candle_core::{Device, Tensor, utils};

fn main() -> candle_core::Result<()> {
    println!("cuda available: {}, metal available: {}", utils::cuda_is_available(),
            utils::metal_is_available());
    let dev = Device::cuda_if_available(0)?;
    println!("cuda_if_available -> {dev:?}");
    match Device::new_metal(0) {
        Ok(d) => println!("metal: {d:?}"),
        Err(e) => println!("metal: {e}"),
    }
    let t = Tensor::new(&[1.0f32, 2.0], &dev)?;
    println!("on {:?}; moved to cpu: {:?}", t.device(), t.to_device(&Device::Cpu)?.dims());
    Ok(())
}
```

On the checking machine, with neither feature enabled, this printed `cuda available: false, metal available: false, cuda_if_available -> Cpu`, and `metal: the candle crate has not been built with metal support`.

3.6 Instrumenting with tracing

Absorbs the tracing::Span explanation of 3_attention_misc_2.md.

This project depends on the `tracing` crate, and the attention code in the old notes stored one span in the attention struct and entered it on every forward pass. A span is a named stretch of time with structured fields. `span!(Level::TRACE, "mhsa")` creates one; `span.enter()` returns a guard, and the span counts as active until that guard is dropped, which is Rust's usual way of tying "until the end of this scope" to a value. On their own the macros record nothing: a *subscriber* (for example the

tracing-subscriber crate, which this project does not include) decides what to do with spans and events; without one they cost a couple of branches at run time and record nothing, and `is_disabled()` reports true. That is why library code can leave spans in place at negligible cost.

```
use tracing::{Level, span};

fn score_slip(words: usize) -> usize {
    let span = span!(Level::TRACE, "attention", words = words);
    let _guard = span.enter();           // active until _guard is dropped at the end of the
        function
    words * words                        // the work being measured: one score per pair of
        words
}

fn main() {
    let s = span!(Level::TRACE, "mhsa");
    println!("without a subscriber the span is disabled: {}", s.is_disabled());
    println!("scores computed: {}", score_slip(4));
}
```

```
without a subscriber the span is disabled: true
scores computed: 16
```

Part B: Parameters and training

4. Var, VarMap and VarBuilder

Absorbs 00_VarBuilder.md, 00_VarBuilder1.md, the parameter-management essay repeated in 04_mlp_multi_layer_perceptron.md, ..2.md and ..3.md, and the VarBuilder summary in 2_attention_misc.md.

4.1 Three types, three jobs

A model is a pile of numbers that training is allowed to change. candle separates three concerns:

Type	Job	Café picture
<code>candle_core::Var</code>	one tensor whose storage may be overwritten in place (set); everything else in candle is immutable	one page of the recipe book, written in pencil
<code>candle_nn::VarMap</code>	a thread-safe map from a full name such as <code>till.layer1.weight</code> to a <code>Var</code> ; cloning a <code>VarMap</code> clones the handle, not the contents; can save to and load from a safetensors file (Hugging Face's simple format for named tensors)	the recipe book
<code>candle_nn::VarBuilder</code>	a cursor that carries a name prefix, a dtype and a device, and hands out tensors by name; <code>pp("x")</code> returns a new cursor one level deeper; backed by a <code>VarMap</code> when training, or by a weights file when loading	the pen that writes page titles

The pattern in every candle model is the same: a constructor receives a `VarBuilder`, asks it for tensors by name, and stores the tensors in the layer structs. If the builder is backed by a `VarMap`, every requested tensor is created (with the initialisation you asked for) and registered in the map under its full name. If the builder is backed by a file, the tensor is read from the file instead. The model code does not know which.

4.2 Building a model and looking inside the map

```
use candle_core::{DType, Device, Module, Tensor};
use candle_nn::{Linear, VarBuilder, VarMap, linear};

/// The café's busy-hour predictor: 3 numbers in, 2 hidden, 1 out.
struct Predictor {
    hidden: Linear,
    out: Linear,
}

impl Predictor {
    fn new(vb: VarBuilder) -> candle_core::Result<Self> {
        Ok(Self {
            hidden: linear(3, 2, vb.pp("hidden"))?, // creates hidden.weight [2,3] and
                                                    // hidden.bias [2]
            out: linear(2, 1, vb.pp("out"))?,       // creates out.weight [1,2] and
                                                    // out.bias [1]
        })
    }
}

impl Module for Predictor {
    fn forward(&self, x: &Tensor) -> candle_core::Result<Tensor> {
        self.out.forward(&self.hidden.forward(x)?.relu())
    }
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let model = Predictor::new(vb.pp("till"))?;

    let mut names: Vec<(String, Vec<usize>)> = varmap
        .data().lock().unwrap()
        .iter().map(|(name, var)| (name.clone(), var.dims().to_vec()))
        .collect();
    names.sort();
    for (name, dims) in &names {
        println!("{name:<20} {dims:?}");
    }
    let total: usize = varmap.all_vars().iter().map(|v| v.elem_count()).sum();
    println!("{variables, {total} numbers to learn", varmap.all_vars().len());
    println!("prefix of vb.pp(\"till\").pp(\"hidden\"): {:?\"",
        vb.pp("till").pp("hidden").prefix());

    let x = Tensor::new(&[[0.5f32, 0.25, 1.0]], &dev)?;
    println!("output shape {:?}", model.forward(&x)?.dims());
    Ok(())
}
```

```
till.hidden.bias      [2]
till.hidden.weight    [2, 3]
till.out.bias         [1]
till.out.weight       [1, 2]
4 variables, 11 numbers to learn
prefix of vb.pp("till").pp("hidden"): "till.hidden"
output shape [1, 1]
```

`vb.pp("till")` produces the prefix `till`; inside the constructor `vb.pp("hidden")` produces `till.hidden`; and `linear` adds `.weight` and `.bias`. The names are ordinary strings joined with dots, decided at run time. That is also how the names inside a `safetensors` file are matched when you load weights written by another program.

4.3 Initialisation: what exists

`vb.get(shape, name)` creates a tensor with `Init::default()`, and that default is `Const(0.)`: all zeros. Zeros are fine for a bias and wrong for a weight (every unit in the layer starts identical and receives identical gradients, so the units never diverge). The layer constructors avoid this by passing an explicit `Init`; when you create weights yourself, use `vb.get_with_hints(shape, name, init)`. These are the only `Init` values in `candle-nn`, on both versions:

Init value	Meaning
<code>Init::Const(c)</code>	every element equals <code>c</code> ; <code>init::ZERO</code> and <code>init::ONE</code> are <code>Const(0.)</code> and <code>Const(1.)</code>
<code>Init::Randn { mean, stdev }</code>	normal distribution
<code>Init::Uniform { lo, up }</code>	uniform distribution
<code>Init::Kaiming { dist, fan, non_linearity }</code>	He (Kaiming) initialisation: random values scaled by $1/\sqrt{\text{fan_in}}$ times a gain for the activation, so that activations keep roughly the same size from layer to layer; <code>init::DEFAULT_KAIMING_NORMAL</code> and <code>init::DEFAULT_KAIMING_UNIFORM</code> are the two ready-made settings (fan-in, ReLU gain)

There are no `init::kaiming_normal()` or `init::xavier_uniform()` functions; the old notes invented them. `candle_nn::linear` and `candle_nn::conv2d` use `DEFAULT_KAIMING_NORMAL` for the weight, and for the bias a `Uniform` in $\pm 1/\sqrt{\text{in_dim}}$ (`linear`) or $\pm 1/\sqrt{\text{in_channels}}$ (`conv2d`; the kernel area does not enter the bound).

```
use candle_core::{DType, Device, Tensor};
use candle_nn::{Init, VarBuilder, VarMap, init};

fn main() -> candle_core::Result<()> {
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &Device::Cpu);
    let all_zero = |t: &Tensor| t.flatten_all().unwrap().to_vec1:::<f32>
        ().unwrap().iter().all(|v| *v == 0.0);
    let w_default = vb.get((2, 3), "w_default"?);
    let w_kaiming = vb.get_with_hints((2, 3), "w_kaiming", init::DEFAULT_KAIMING_NORMAL)?;
    let b_zero = vb.get_with_hints(3, "b_zero", init::ZERO)?;
    println!("Init::default() = {:?}", Init::default());
    println!("vb.get          -> all zero: {}", all_zero(&w_default));
    println!("get_with_hints(Kaiming) -> all zero: {}", all_zero(&w_kaiming));
    println!("get_with_hints(ZERO)    -> all zero: {}", all_zero(&b_zero));
    Ok(())
}
```

```
Init::default() = Const(0.0)
vb.get          -> all zero: true
get_with_hints(Kaiming) -> all zero: false
get_with_hints(ZERO)    -> all zero: true
```

```
// X expected: cannot find function `kaiming_normal` in module `init`
use candle_nn::init;

fn main() {
    let i = init::kaiming_normal();
    println!("{i:?}");
}
```

4.4 The same-name rule

Ask a VarMap-backed builder for a name twice and you get the same variable twice, provided the shapes agree. This is how weight sharing works, and it is also how an accidental name clash goes unnoticed. If the shapes disagree, you get an error at the second request.

```
use candle_core::{DType, Device, Tensor, Var};
use candle_nn::{VarBuilder, VarMap, init};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);

    let a = vb.get_with_hints((2, 2), "shared", init::ZERO?);
    let b = vb.get_with_hints((2, 2), "shared", init::ZERO?); // same name, same shape:
    // same variable
    println!("entries in the map: {}", varmap.data().lock().unwrap().len());

    let var = varmap.data().lock().unwrap().get("shared").unwrap().clone();
    var.set(&Tensor::new(&[[9.0f32, 9.0], [9.0, 9.0]], &dev)?); // overwrite in place
    println!("a now {:?}", a.to_vec2::<f32>());
    println!("b now {:?}", b.to_vec2::<f32>());

    println!("different shape: {}", vb.get((3, 3), "shared").unwrap_err());

    // Var::from_tensor on a tensor that already is a variable returns the same variable
    let again = Var::from_tensor(&a)?;
    again.set(&Tensor::zeros((2, 2), DType::F32, &dev)?);
    println!("a after zeroing through a Var made from it: {:?}", a.to_vec2::<f32>());

    // Var::from_tensor on an ordinary tensor makes a copy
    let plain = Tensor::new(&[1.0f32, 2.0], &dev)?;
    let copy = Var::from_tensor(&plain)?;
    copy.set(&Tensor::new(&[7.0f32, 7.0], &dev)?);
    println!("plain tensor after zeroing its copy: {:?}; variable? {} / {}", plain.to_vec1::
    <f32>(), plain.is_variable(), copy.is_variable());
    Ok(())
}
```

```
entries in the map: 1
a now [[9.0, 9.0], [9.0, 9.0]]
b now [[9.0, 9.0], [9.0, 9.0]]
different shape: shape mismatch on shared: [3, 3] <> [2, 2]
a after zeroing through a Var made from it: [[0.0, 0.0], [0.0, 0.0]]
plain tensor after zeroing its copy: [1.0, 2.0]; variable? false / true
```

Two facts from this run carry the whole training story:

- A tensor handed out by a VarMap-backed builder is a *view of the variable's storage*. Overwriting the variable changes what the layer sees.

- `Var::from_tensor` does not copy a tensor that is already a variable; it wraps the same storage. It copies only ordinary tensors. This project's `src/models/mlp.rs` stores `Var::from_tensor(&vb.get_with_hints(..)?)` in its `ConditionalLayer`, and because the builder is `VarMap`-backed that `Var` is the map's own variable, so the optimizer's updates reach the layer.

4.5 The "ownership problem" that does not exist

The old notes built a story in which a `VarBuilder` is "moved into" the model, is then "no longer available for the optimizer", and must therefore be cloned. None of that is right, for three reasons:

1. The optimizer never wants the builder. It wants the variables: `SGD::new(varmap.all_vars(), lr)`.
2. `vb.pp("x")` takes `&self` and returns a fresh builder, so passing `vb.pp("hidden")` into a constructor never moves `vb`.
3. `VarBuilder` does implement `Clone`, and the clone is cheap (it shares the map through an `Arc`), so `vb.clone()` is fine when you do need a second copy of the *same* prefix. It is just rarely needed.

What people actually hit is the by-value signature: `linear(in, out, vb: VarBuilder)` takes ownership, so `linear(3, 2, vb)` followed by another use of `vb` is a use-after-move error. The idiom is to pass `vb.pp("name")`, which is a new value each time. And the one compile error everyone makes at least once is passing the map itself instead of a reference:

```
// X expected: expected `&VarMap`, found `VarMap`
use candle_core::{DType, Device};
use candle_nn::{VarBuilder, VarMap};

fn main() {
    let vb = VarBuilder::from_varmap(VarMap::new(), DType::F32, &Device::Cpu);
    println!("{:?}", vb.dtype());
}
```

4.6 Why one optimizer step changes the model

Put the two facts of section 4.4 together with a loss and an optimizer. The layer below holds a `Var` made from the builder's tensor; the optimizer holds the map's variables; they are the same storage, so after `backward_step` the layer computes something new. The numbers are chosen so you can check

them by hand: with the weight starting at $[1, 1]$, input $[1, 2]$ and target 1, the output is 3, the squared error is 4, its gradient with respect to the weight is $2 \cdot (3-1) \cdot [1, 2] = [4, 8]$, and one step of SGD with learning rate 0.1 gives $[0.6, 0.2]$, whose output is exactly 1.

```
use candle_core::{DType, Device, Tensor, Var};
use candle_nn::{Init, Optimizer, SGD, VarBuilder, VarMap};

struct TinyLayer {
    weight: Var,
}

impl TinyLayer {
    fn new(vb: &VarBuilder) -> candle_core::Result<Self> {
        let w = vb.get_with_hints((1, 2), "weight", Init::Const(1.0))?;
        Ok(Self { weight: Var::from_tensor(&w)? })
    }

    fn forward(&self, x: &Tensor) -> candle_core::Result<Tensor> {
        x.matmul(&self.weight.as_tensor().t()?)
    }
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let layer = TinyLayer::new(&vb)?;
    let mut sgd = SGD::new(varmap.all_vars(), 0.1)?;

    let x = Tensor::new(&[[1.0f32, 2.0]], &dev)?;
    let target = Tensor::new(&[[1.0f32]], &dev)?;
    println!("output before: {:.2}", layer.forward(&x)?.to_vec2::<f32>()?[0][0]);

    let loss = candle_nn::loss::mse(&layer.forward(&x)?, &target)?;
    println!("loss: {:.2}", loss.to_scalar::<f32>()?);
    sgd.backward_step(&loss)?;

    let w = varmap.data().lock().unwrap().get("weight").unwrap().to_vec2::<f32>()?;
    println!("weight in the map after one step: [{:.2}, {:.2}]", w[0][0], w[0][1]);
    println!("output after: {:.2}", layer.forward(&x)?.to_vec2::<f32>()?[0][0]);
    println!("same storage: {}", layer.weight.as_tensor().id() == varmap.all_vars()[0].as_tensor().id());
    Ok(())
}
```

```
output before: 3.00
loss: 4.00
weight in the map after one step: [0.60, 0.20]
output after: 1.00
same storage: true
```

`backward_step` is `loss.backward()` followed by `step(&grads)`. `backward()` walks the operations that produced the loss and returns a `GradStore` keyed by variable; `step` applies the update rule to every variable it was given that has a gradient in the store. Variables not reachable from the loss are simply skipped.

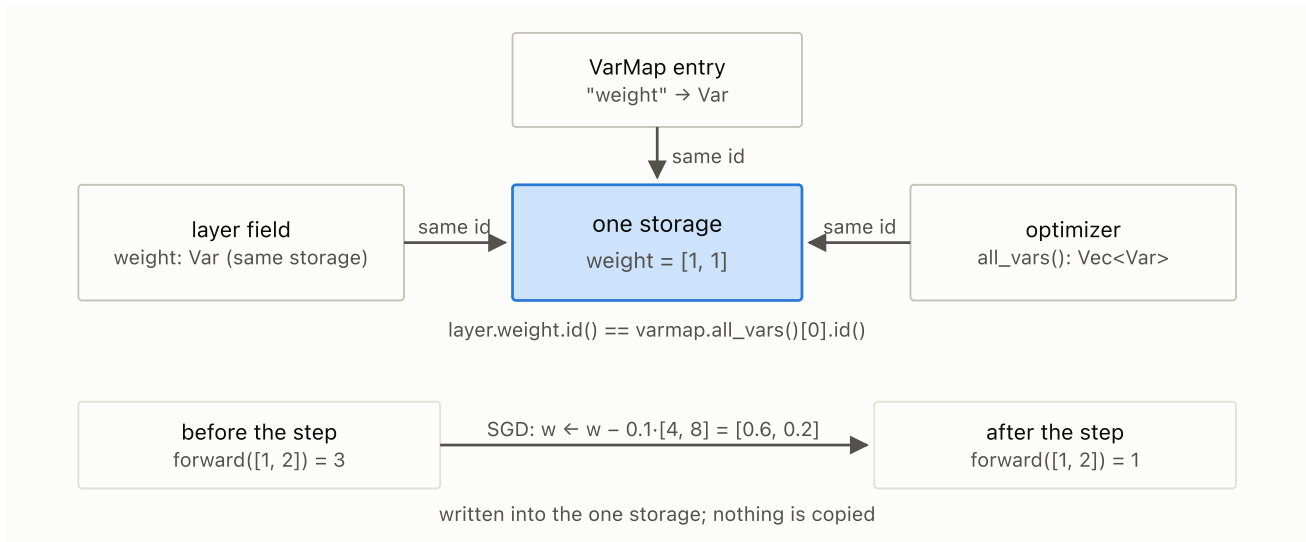


Figure 4. The `till` layer's tensor, the `VarMap`'s variable and the optimizer's variable are three handles to one storage, so the SGD step that turns the weight `[1, 1]` into `[0.6, 0.2]` also changes what the layer computes, from 3 to 1, with nothing copied.

4.7 Saving and loading a `VarMap`

`varmap.save("file.safetensors")` writes every variable under its full name. `load` is the mirror image and needs a mutable map: it walks the `map`'s names and reads each one from the file. Names present in the file but not in the map are ignored; a name missing from the file is an error. So the workflow is: build the model (which registers the names), then load.

```
use candle_core::{DType, Device, Module, Tensor};
use candle_nn::{Linear, VarBuilder, VarMap, linear};

fn build(varmap: &VarMap, dev: &Device) -> candle_core::Result<Linear> {
    let vb = VarBuilder::from_varmap(varmap, DType::F32, dev);
    linear(3, 2, vb.pp("till").pp("hidden"))
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let path = std::env::temp_dir().join("cafe_guide_section_4_7.safetensors");

    let mut trained = VarMap::new();
    let _layer = build(&trained, &dev)?;
    trained.set_one("till.hidden.weight", Tensor::full(0.25f32, (2, 3), &dev)?); //
    // pretend we trained it
    trained.save(&path)?;

    let mut fresh = VarMap::new();
    let layer = build(&fresh, &dev)?; // random weights, same
    // names
    let all_quarter = |t: &Tensor| t.flatten_all().unwrap().to_vec1::<f32>
        ().unwrap().iter().all(|v| *v == 0.25);
    println!("before load, weight is all 0.25: {}", all_quarter(layer.weight()));
    fresh.load(&path)?;
    println!("after load, weight is all 0.25: {}", all_quarter(layer.weight()));
    println!("the layer still works: {:?}" , layer.forward(&Tensor::ones((1, 3), DType::F32,
        &dev)?).dims());

    std::fs::remove_file(&path).ok();
    Ok(())
}
```

```
before load, weight is all 0.25: false
after load, weight is all 0.25: true
the layer still works: [1, 2]
```

```
// X expected: cannot borrow `varmap` as mutable, as it is not declared as mutable
use candle_nn::VarMap;

fn main() -> candle_core::Result<()> {
    let varmap = VarMap::new();
    varmap.load("model.safetensors")?;
    Ok(())
}
```

4.8 Loading weights from files: the constructors that exist

For inference you usually do not want a VarMap at all. You want a VarBuilder that reads straight from a weights file, and the layer constructors work unchanged. These are the constructors and methods you will use, from candle-nn's source, on both versions (the full list adds `from_pth_with_state`, `from_backend`, `rename` with a `Renamer`, `root`, `set_prefix` and `to_dtype`):

```
VarBuilder::from_varmap(&varmap, dtype, &device)           // training
unsafe { VarBuilder::from_mmaped_safetensors(&[paths], dtype, &device)? }
VarBuilder::from_buffered_safetensors(bytes: Vec<u8>, dtype, &device)?
VarBuilder::from_slice_safetensors(&bytes, dtype, &device)?
VarBuilder::from_pth("model.pth", dtype, &device)?       // PyTorch pickles
VarBuilder::from_npz("model.npz", dtype, &device)?       // NumPy archives
VarBuilder::from_tensors(HashMap<String, Tensor>, dtype, &device)
VarBuilder::zeros(dtype, &device)                       // every request returns zeros
vb.rename_f(|name| ...)                                 // remap names on the fly
vb.pp("x") / vb.push_prefix("x") / vb.prefix() / vb.contains_tensor("x")
vb.get(shape, "x") / vb.get_with_hints(shape, "x", init) / vb.get_with_hints_dtype(...)
```

There is no `VarBuilder::from_safetensors` on any version this project can use; it existed in early candle and was removed at the 0.3.0 release in October 2023. The memory-mapped constructor is unsafe because a file mapped into memory can be changed by another process while you read it. 0.11.0 adds `get_unchecked`, `get_unchecked_dtype`, `set_device` and `set_dtype`.

`from_tensors` is the easiest way to see a file-free loading path. The tensors it returns are ordinary tensors, not variables, so nothing about them can be trained and no gradient graph is recorded:

```
use candle_core::{DType, Device, Module, Tensor};
use candle_nn::{VarBuilder, linear};
use std::collections::HashMap;

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let mut weights = HashMap::new();
    weights.insert("till.hidden.weight".to_string(), Tensor::new(&[[1.0f32, 0.0, 0.0], [0.0, 1.0, 0.0]], &dev)?);
    weights.insert("till.hidden.bias".to_string(), Tensor::new(&[0.0f32, 0.5], &dev)?);

    let vb = VarBuilder::from_tensors(weights, DType::F32, &dev);
    println!("has hidden.weight: {}, has hidden.bias: {}, has out.weight: {}",
        vb.contains_tensor("till.hidden.weight"), vb.contains_tensor("till.hidden.bias"),
        vb.contains_tensor("till.out.weight"));
    let layer = linear(3, 2, vb.pp("till").pp("hidden"))?;
    println!("is a variable: {}", layer.weight().is_variable());
    println!("{:?}", layer.forward(&Tensor::new(&[[3.0f32, 4.0, 5.0]], &dev)?).to_vec2::()?);
    println!("missing name: {}", linear(2, 1, vb.pp("till").pp("out")).unwrap_err());
    Ok(())
}
```

```
has hidden.weight: true, has hidden.bias: true, has out.weight: false
is a variable: false
[[3.0, 4.5]]
missing name: cannot find tensor till.out.weight
```

4.9 Freezing part of a model

There is no `requires_grad` flag. To fine-tune only the head of a model, give the optimizer only the head's variables. Filtering the map by name prefix is the simplest way; keeping two `VarMaps` (one loaded, one fresh) is the other.

```
use candle_core::{DType, Device, Tensor, Var};
use candle_nn::{Init, Optimizer, SGD, VarBuilder, VarMap};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let backbone = vb.get_with_hints(1, "backbone.w", Init::Const(1.0));
    let head = vb.get_with_hints(1, "head.w", Init::Const(1.0));

    let head_only: Vec<Var> = varmap.data().lock().unwrap()
        .iter()
        .filter(|(name, _)| name.starts_with("head"))
        .map(|(_, var)| var.clone())
        .collect();
    let mut sgd = SGD::new(head_only, 0.1);

    let x = Tensor::new(&[2.0f32], &dev)?;
    let out = x.mul(&backbone)?.mul(&head)?;
    let loss = (out - 1.0)?.sqr()?.sum_all()?;
    sgd.backward_step(&loss)?;
    // 2 * 1 * 1 = 2
    // (2 - 1)^2 = 1
    // gradient for head.w is 2*
    // (2-1)*2 = 4

    println!("backbone.w {:.2} (frozen)", backbone.to_vec1:<f32>()?[0]);
    println!("head.w      {:.2} (1 - 0.1 * 4)", head.to_vec1:<f32>()?[0]);
    Ok(())
}
```

```
backbone.w 1.00 (frozen)
head.w      0.60 (1 - 0.1 * 4)
```

4.10 Compared with PyTorch

	PyTorch	candle
a trainable tensor	<code>nn.Parameter</code> (a tensor with <code>requires_grad=True</code>)	<code>Var</code> (a tensor whose storage can be overwritten)
how a parameter gets its name	attribute name on an <code>nn.Module</code> , or <code>register_parameter("name", p)</code>	the <code>VarBuilder</code> prefix plus the name you ask for
listing parameters	<code>model.parameters()</code> , <code>named_parameters()</code>	<code>varmap.all_vars()</code> , <code>varmap.data()</code>
saving	<code>state_dict()</code> + <code>torch.save</code>	<code>varmap.save</code> (safetensors)
gradient tracking	autograd records operations as they run	the same: every op on a tensor that descends from a <code>Var</code> is recorded; <code>loss.backward()</code> returns the gradients
freezing	<code>p.requires_grad = False</code>	give the optimizer a subset of variables
when names are checked	at run time	at run time

Two claims in the old notes were wrong and are worth unlearning: candle does not use "static computation graphs" (it records operations dynamically, like PyTorch's eager mode), and there is no "compile-time path validation" (a name is a String built at run time; a typo is a run-time error when loading a file, or a silently separate parameter when training).

5. Dot products, dense layers and the MLP

Absorbs 01_dotProductAndGradientDescent.md, 01_structureOfMatrices.md, and the MLP parts of 04_mlp_multi_layer_perceptron.md, ..2.md and ..3.md.

5.1 A dot product is a "how much do you look like me" score

Take the last three numbers the till produced for an hour: sales in the previous hour, the hour of the day scaled to 0..1, and a 1 if it is a weekend. That is a vector x . A neuron is another vector w of the same length, plus a bias b . Its output is $w \cdot x + b$: multiply matching entries, add them up.

Geometrically $w \cdot x = \|w\| \|x\| \cos \theta$, where θ is the angle between the two vectors. So a neuron scores how well the input lines up with its own preferred direction. A big positive score means "this input looks like what I detect"; near zero means "unrelated"; negative means "the opposite of what I detect". During training, gradient descent nudges each w so that the inputs it should fire for line up with it better and the others line up worse. The forward pass measures alignment; the backward pass rotates the detectors.

```
fn dot(a: &[f32], b: &[f32]) -> f32 { a.iter().zip(b).map(|(x, y)| x * y).sum() }
fn norm(a: &[f32]) -> f32 { dot(a, a).sqrt() }

fn main() {
  let saturday_lunch = [0.9f32, 0.5, 1.0]; // high previous sales, midday, weekend
  let tuesday_dawn = [0.1f32, 0.05, 0.0];
  let busy_detector = [0.8f32, 0.3, 0.6];
  let quiet_detector = [-0.8f32, -0.3, -0.6];
  for (name, x) in [("saturday lunch", saturday_lunch), ("tuesday dawn", tuesday_dawn)] {
    for (dname, w) in [("busy", busy_detector), ("quiet", quiet_detector)] {
      let score = dot(&x, &w);
      let cos = score / (norm(&x) * norm(&w));
      println!("{name:<15} vs {dname:<5} detector: score {score:>6.3} cos θ {cos:>6.3}");
    }
  }
}
```

```
saturday lunch vs busy detector: score 1.470 cos θ 0.981
saturday lunch vs quiet detector: score -1.470 cos θ -0.981
tuesday dawn vs busy detector: score 0.095 cos θ 0.814
tuesday dawn vs quiet detector: score -0.095 cos θ -0.814
```

A whole layer is many detectors stacked as rows of a matrix W of shape (neurons, inputs), and a whole batch is many inputs stacked as rows of X of shape (items, inputs). $X @ W^T$ computes every dot product at once; that is the (items, inputs) @ (inputs, neurons) product of section 3.2.

5.2 From one layer to a network

Stacking two linear layers with nothing in between is pointless: $W_2(W_1x + b_1) + b_2$ is just another linear map. The non-linear "activation" between layers is what lets the network bend. ReLU, $\max(0, z)$, is the default choice: cheap, and its gradient is 1 or 0. A network of linear layers with activations between them is a multi-layer perceptron (MLP). Its output layer has no activation: for classification the raw outputs (logits) go into a softmax and a cross-entropy loss (section 6); for regression they are the prediction.

Layer by layer, with $h^{(0)} = x$:

$z^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}$	(linear)
$h^{(l)} = f(z^{(l)})$	(activation, except on the last layer)

5.3 The café MLP

This is the MLP type from this project's `src/models/mlp.rs`, cleaned up and made to implement `Module`. Hidden layers get ReLU; the output layer does not.

```
use candle_core::{DType, Device, Module, Tensor};
use candle_nn::{Linear, VarBuilder, VarMap, linear};

pub struct Mlp {
    layers: Vec<Linear>,
}

impl Mlp {
    pub fn new(vb: &VarBuilder, input: usize, hidden: &[usize], output: usize) ->
        candle_core::Result<Self> {
        let mut layers = Vec::new();
        let mut prev = input;
        for (i, &h) in hidden.iter().enumerate() {
            layers.push(linear(prev, h, vb.pp(format!("layer_{i}")))?);
            prev = h;
        }
        layers.push(linear(prev, output, vb.pp("output"))?);
        Ok(Self { layers })
    }
}

impl Module for Mlp {
    fn forward(&self, x: &Tensor) -> candle_core::Result<Tensor> {
        let mut cur = x.clone();
        for layer in &self.layers[..self.layers.len() - 1] {
            cur = layer.forward(&cur)?.relu()?;
        }
        self.layers.last().unwrap().forward(&cur)
    }
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let mlp = Mlp::new(&vb, 4, &[8, 8], 3)?; // 4 till numbers -> 8 -> 8 -> 3
        classes
    let batch = Tensor::zeros((5, 4), DType::F32, &dev)?;
    println!("{:?}", batch.dims(), mlp.forward(&batch)?.dims());
    let n: usize = varmap.all_vars().iter().map(|v| v.elem_count()).sum();
    println!("parameters: {n} = (4*8+8) + (8*8+8) + (8*3+3) = {}", 4 * 8 + 8 + 8 * 8 + 8 + 8
        * 3 + 3);
    Ok(())
}
```

```
[5, 4] -> [5, 3]
parameters: 139 = (4*8+8) + (8*8+8) + (8*3+3) = 139
```

5.4 Training: loss, backward, step

Training is a loop of four lines: run the model, measure how wrong it was (the loss), compute the gradient of the loss with respect to every variable (backward), and move every variable a little against its gradient (step). The optimizer owns the last part and its hyper-parameters.

Optimizer	Construct	Update rule
SGD	<code>SGD::new(vars, learning_rate)</code>	$w \leftarrow w - lr \cdot g$
AdamW	<code>AdamW::new(vars, ParamsAdamW { lr, beta1, beta2, eps, weight_decay })</code> or <code>AdamW::new_lr(vars, lr)</code>	Adam's running averages of g and g^2 with bias correction, plus decoupled weight decay; defaults lr 0.001, β_1 0.9, β_2 0.999, ϵ $1e-8$, $weight_decay$ 0.01

`AdamW::new` takes a `ParamsAdamW`, not a number. The old notes passed a plain `f64` learning rate to `AdamW::new`, which does not compile:

```
// X expected: expected `ParamsAdamW`, found floating-point number
use candle_nn::{AdamW, Optimizer, VarMap};

fn main() -> candle_core::Result<()> {
    let varmap = VarMap::new();
    let _opt = AdamW::new(varmap.all_vars(), 0.001)?;
    Ok(())
}
```

Here is the whole loop on a made-up café dataset: 64 hours described by four numbers, labelled busy (1) during the breakfast and lunch rushes and quiet (0) otherwise. The initial weights are random, so the exact numbers vary from run to run; on the two checking runs the loss went from 0.73 to 0.0003 and from 2.03 to 0.28 in 300 steps.

```
// ~ output varies (random initial weights)
use candle_core::{D, DType, Device, Module, Tensor};
use candle_nn::{AdamW, Linear, Optimizer, VarBuilder, VarMap, linear, loss};

struct Mlp { layers: Vec<Linear> }

impl Mlp {
    fn new(vb: &VarBuilder, sizes: &[usize]) -> candle_core::Result<Self> {
        let mut layers = Vec::new();
        for (i, pair) in sizes.windows(2).enumerate() {
            layers.push(linear(pair[0], pair[1], vb.pp(format!("layer_{i}")))?);
        }
        Ok(Self { layers })
    }
}

impl Module for Mlp {
    fn forward(&self, x: &Tensor) -> candle_core::Result<Tensor> {
        let mut cur = x.clone();
        for (i, layer) in self.layers.iter().enumerate() {
            cur = layer.forward(&cur)?;
            if i + 1 < self.layers.len() { cur = cur.relu()?; }
        }
        Ok(cur)
    }
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    // features: hour/16, weekday/4, distance from 8 o'clock, constant 1
    let xs: Vec<f32> = (0..64).flat_map(|i| {
        let h = (i % 16) as f32; let d = (i / 16) as f32;
        vec![h / 16.0, d / 4.0, ((h - 8.0) / 4.0).abs(), 1.0]
    }).collect();
    let ys: Vec<u32> = (0..64).map(|i| { let h = i % 16; u32::from((7..=9).contains(&h) ||
        (12..=14).contains(&h)) }).collect();
    let x = Tensor::from_slice(&xs, (64, 4), &dev)?;
    let y = Tensor::from_slice(&ys, 64, &dev)?; // class indices, U32

    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let model = Mlp::new(&vb, &[4, 16, 16, 2])?;
    let mut opt = AdamW::new_lr(varmap.all_vars(), 0.05)?;

    let mut first = 0.0;
    let mut last = 0.0;
    for step in 0..300 {
        let logits = model.forward(&x)?; // (64, 2)
        let l = loss::cross_entropy(&logits, &y)?; // scalar
        let grads = l.backward()?; // gradients for every variable
        opt.step(&grads)?; // AdamW update
        let v = l.to_scalar::<f32>()?;
        if step == 0 { first = v; }
        last = v;
    }
    let predicted = model.forward(&x)?.argmax(D::Minus1)?; // U32
    // class per row
    let accuracy = predicted.eq(&y)?.to_dtype(DType::F32)?.mean_all()?.to_scalar::<f32>()?;
    println!("loss {first:.4} -> {last:.4}, decreased: {}, accuracy {accuracy:.3}", last <
        first);
    Ok(())
}
```

Points that the old notes got wrong or left out:

- `loss::cross_entropy(logits, targets)` takes logits of shape `(items, classes)` and targets of shape `(items,)` holding class indices in an integer dtype (U8, U32 or I64). No one-hot vectors, no manual softmax.
- `opt.backward_step(&loss)` is the two lines backward and step in one call.
- There is no `Tensor::log_softmax` method; it is `candle_nn::ops::log_softmax(&t, dim)` (section 6.5).
- `Var::randn(0.0, 1.0, ..)` creates an F64 variable because the literals are f64; multiplying it with f32 inputs fails with dtype mismatch in `matmul`. Write `0f32`.

5.5 What backward does, in words

The loss is a scalar at the end of a chain of operations. `backward` walks that chain in reverse and applies the chain rule at every step: the gradient of the loss with respect to a layer's output is multiplied by that layer's local derivative to get the gradient with respect to its input and its parameters. For a linear layer $z = W h + b$ with incoming gradient δ (the derivative of the loss with respect to z):

```

 $\partial L / \partial W = \delta h^T$       (outer product: each weight's gradient is "how much this output wanted
                             to move" times "what this input was")
 $\partial L / \partial b = \delta$ 
 $\partial L / \partial h = W^T \delta$   (passed on to the previous layer)

```

For ReLU the local derivative is 1 where the input was positive and 0 elsewhere, so δ is simply zeroed where the neuron was off. You never write this in candle; it is recorded automatically because every tensor remembers the operation and inputs that produced it, as long as some ancestor is a `Var`. Two practical consequences:

- Intermediate tensors of the forward pass stay alive until the loss (and its `GradStore`) are dropped, because the backward pass needs them. Drop them each iteration; the loop above does that naturally.
- `tensor.detach()` gives a new handle to the same numbers that forgets its history; no copy is made. Use it when you want a value out of the graph, for example to log an accuracy without keeping the whole forward pass alive.

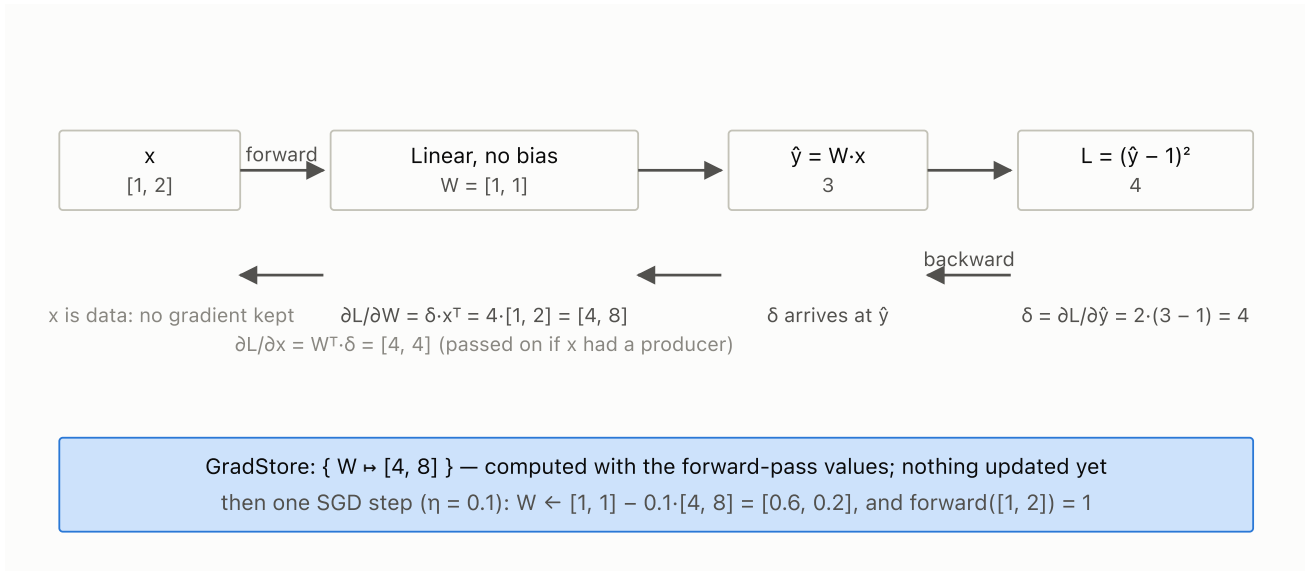


Figure 5. backward runs the till's chain in reverse: the loss gradient $\delta = 2 \cdot (3 - 1) = 4$ at the output becomes $\delta \cdot x^T = [4, 8]$ for the weight, and only then does one SGD step with rate 0.1 move $[1, 1]$ to $[0.6, 0.2]$.

5.6 Inference

Inference is a forward pass without a loss: no backward, no optimizer, ideally no VarMap at all (load a file-backed VarBuilder, section 4.8, and the tensors are plain). Batch several inputs into one tensor rather than looping. Layers that behave differently in training, such as BatchNorm and Dropout, implement ModuleT and take a `train: bool`; call `forward_t(&x, false)` (section 11). Reduced-precision and quantised inference exist in candle (DType::F16, BF16, and the `candle_core::quantized module`) but are outside this guide.

6. Softmax done safely, and cross-entropy

Absorbs `01_safe_softmax.md`.

6.1 What softmax is for

A classifier's last layer produces one number per class, the logits. Softmax turns them into probabilities: every output is between 0 and 1 and they sum to 1.

$$\text{softmax}(z)_i = \exp(z_i) / \sum_j \exp(z_j)$$

`exp` makes every value positive and exaggerates differences; the division makes them sum to 1. It is "soft" because the largest logit gets most of the mass rather than all of it, unlike `argmax`. The same function, applied across the keys of an attention score row, decides how much each word of the order slip attends to each other word (section 14).

6.2 Why the textbook formula breaks

`exp` overflows fast. In f64, `exp(709.7)` is finite and `exp(709.8)` is infinity; in f32 the limit is about 88.7. Logits of a few hundred are ordinary in a badly scaled network, and infinity divided by infinity is NaN. Underflow is the mirror problem: `exp(-745.2)` is exactly 0 in f64, and if every term underflows the denominator is 0.

6.3 The fix: subtract the maximum

Softmax does not change if you add the same constant to every logit, because the constant factors out of numerator and denominator:

$$\exp(z_i - m) / \sum_j \exp(z_j - m) = \exp(z_i) \cdot \exp(-m) / (\exp(-m) \cdot \sum_j \exp(z_j)) = \exp(z_i) / \sum_j \exp(z_j)$$

Choose $m = \max(z)$. Then the largest exponent is exactly 0, nothing can overflow, and the denominator is at least 1, so it can never be 0. Terms far below the maximum may still underflow to 0, and that is the right answer: they had negligible probability anyway.

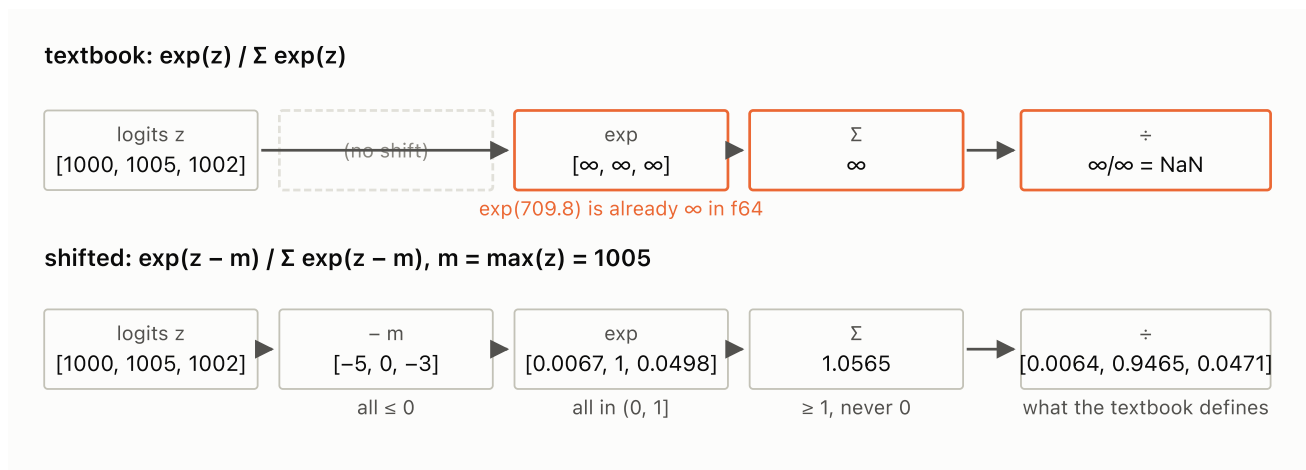


Figure 6. Softmax of the class scores [1000, 1005, 1002] is the same with or without subtracting 1005 first, but only the shifted version survives in floating point: $\exp(1005)$ is infinity in f64 and ∞/∞ is NaN, while the shifted logits $-5, 0, -3$ give the probabilities 0.0064, 0.9465, 0.0471.

6.4 A plain-Rust implementation, with the checks that were wrong in the old note

The old note included tests whose expected values for `softmax([1, 5, 2])` were wrong (it said 0.9359 for the middle entry; the value is 0.9362) and whose code had been corrupted by footnote markers. This version prints the values so you can compare them against candle in the next section. It returns NaN if the input contains NaN or $+\infty$ (because $\infty - \infty$ is NaN) rather than silently returning a uniform distribution as the old code did; a NaN that surfaces is a bug you can find.

```
/// Softmax over a slice, stable against overflow. Empty input gives an empty output.
pub fn safe_softmax(z: &[f64]) -> Vec<f64> {
    if z.is_empty() {
        return Vec::new();
    }
    let max = z.iter().copied().fold(f64::NEG_INFINITY, f64::max);
    let exps: Vec<f64> = z.iter().map(|&v| (v - max).exp()).collect();
    let sum: f64 = exps.iter().sum();
    exps.into_iter().map(|e| e / sum).collect()
}

/// The textbook formula, for comparison only.
pub fn naive_softmax(z: &[f64]) -> Vec<f64> {
    let exps: Vec<f64> = z.iter().map(|&v| v.exp()).collect();
    let sum: f64 = exps.iter().sum();
    exps.into_iter().map(|e| e / sum).collect()
}

fn show(v: &[f64]) -> String {
    v.iter().map(|x| format!("{x:.10}")).collect::<Vec<_>>().join(" ")
}

fn main() {
    let cases: [&[f64]; 6] = [
        &[1.0, 5.0, 2.0],
        &[1000.0, 1005.0, 1002.0],
        &[-1000.0, -1005.0, -1002.0],
        &[0.0, 1.0],
        &[5.0, 5.0, 5.0, 5.0],
        &[f64::INFINITY, 1.0],
    ];
    for z in cases {
        let s = safe_softmax(z);
        println!("{z:?}\n safe : {} (sum {:.6})\n naive: {}", show(&s), s.iter().sum::
            <f64>(), show(&naive_softmax(z)));
    }
    println!("empty: {:?}", safe_softmax(&[]));
    println!("f64 exp(709.7) finite: {}, exp(709.8) finite: {}", 709.7f64.exp().is_finite(),
        709.8f64.exp().is_finite());
    println!("f32 exp(88.7) finite: {}, exp(88.8) finite: {}", 88.7f32.exp().is_finite(),
        88.8f32.exp().is_finite());
    println!("f64 exp(-745.2) = {}", (-745.2f64).exp());
}
```

```

[1.0, 5.0, 2.0]
  safe : 0.0171478255 0.9362395519 0.0466126226 (sum 1.000000)
  naive: 0.0171478255 0.9362395519 0.0466126226
[1000.0, 1005.0, 1002.0]
  safe : 0.0063774609 0.9464991226 0.0471234165 (sum 1.000000)
  naive: NaN NaN NaN
[-1000.0, -1005.0, -1002.0]
  safe : 0.8756005951 0.0058997504 0.1184996545 (sum 1.000000)
  naive: NaN NaN NaN
[0.0, 1.0]
  safe : 0.2689414214 0.7310585786 (sum 1.000000)
  naive: 0.2689414214 0.7310585786
[5.0, 5.0, 5.0, 5.0]
  safe : 0.2500000000 0.2500000000 0.2500000000 0.2500000000 (sum 1.000000)
  naive: 0.2500000000 0.2500000000 0.2500000000 0.2500000000
[inf, 1.0]
  safe : NaN NaN (sum NaN)
  naive: NaN 0.0000000000
empty: []
f64 exp(709.7) finite: true, exp(709.8) finite: false
f32 exp(88.7) finite: true, exp(88.8) finite: false
f64 exp(-745.2) = 0

```

Read the second case: the naive formula returns NaN for logits around 1000 while the safe one returns a proper distribution. The third case shows that very negative logits are just as dangerous for the naive formula (everything underflows to 0, then 0/0). The identical-logits case gives exactly 0.25 each.

6.5 What candle does

`candle_nn::ops::softmax(&t, dim)` subtracts the maximum along `dim` before exponentiating, so it is the safe version. `softmax_last_dim` is a fused, faster kernel for the last dimension. `log_softmax` computes $\log(\text{softmax})$ directly as $(z - m) - \log \sum \exp(z - m)$, which is both stable and what cross-entropy needs. All three are free functions in `candle_nn::ops`, not methods on `Tensor`.

```

// X expected: no method named `log_softmax` found for struct `candle_core::Tensor`
use candle_core::{Device, Tensor};

fn main() -> candle_core::Result<()> {
    let t = Tensor::new(&[[1.0f32, 2.0]], &Device::Cpu)?;
    let l = t.log_softmax(1)?;
    println!("{}", l);
    Ok(())
}

```

Cross-entropy for one item is $-\log p_{\text{target}}$, the negative log of the probability the model gave the correct class. Written with logits it is $-(z_{\text{target}} - \log \text{sumexp}(z))$, which is why `log_softmax` is the building block. `candle_nn::loss::cross_entropy(logits, targets)` computes `log_softmax` over the class dimension, picks the entry of the correct class for each row, and averages the negatives:

```
use candle_core::{D, Device, Tensor};
use candle_nn::{loss, ops};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let big = Tensor::new(&[[1000.0f32, 1005.0, 1002.0]], &dev)?;
    println!("softmax of huge logits: {:?}", ops::softmax(&big, D::Minus1)?.to_vec2::<f32>()?);
    println!("softmax_last_dim      : {:?}", ops::softmax_last_dim(&big)?.to_vec2::<f32>()?);
    let naive = big.exp()?.broadcast_div(&big.exp()?.sum_keepdim(1)??);
    println!("naive exp / sum      : {:?}", naive.to_vec2::<f32>()?);
    println!("softmax([1, 5, 2])    : {:?}", ops::softmax(&Tensor::new(&[[1.0f32, 5.0, 2.0]], &dev)?, D::Minus1)?.to_vec2::<f32>()?);

    let masked = Tensor::new(&[[1.0f32, f32::NEG_INFINITY, 2.0], [f32::NEG_INFINITY, f32::NEG_INFINITY, f32::NEG_INFINITY]], &dev)?;
    println!("with -inf entries      : {:?}", ops::softmax(&masked, D::Minus1)?.to_vec2::<f32>()?);

    let logits = Tensor::new(&[[2.0f32, 0.5, -1.0], [0.1, 0.2, 3.0]], &dev)?; // 2 items, 3 classes
    let targets = Tensor::new(&[0u32, 2], &dev)?; // correct class per item
    let ce = loss::cross_entropy(&logits, &targets)?;
    let by_hand = ops::log_softmax(&logits, D::Minus1)?.gather(&targets.unsqueeze(1)?, 1)? // (2, 1): the log-probability of each correct class
        .neg()?
        .mean_all()?;
    println!("cross_entropy {:.6} by hand {:.6}", ce.to_scalar::<f32>(), by_hand.to_scalar::<f32>());
    println!("gather needs equal ranks: {}", ops::log_softmax(&logits, D::Minus1)?.gather(&targets, 1).unwrap_err());
    println!("cross_entropy needs rank 2: {}", loss::cross_entropy(&Tensor::zeros((2, 3, 4), candle_core::DType::F32, &dev)?, &targets).unwrap_err());
    println!("mse: {}", loss::mse(&Tensor::new(&[[1.0f32, 2.0]], &dev)?, &Tensor::new(&[[0.0f32, 0.0]], &dev)?)?.to_scalar::<f32>());
    Ok(())
}
```

```
softmax of huge logits: [[0.006377461, 0.94649917, 0.047123417]]
softmax_last_dim      : [[0.006377461, 0.94649917, 0.047123417]]
naive exp / sum      : [[NaN, NaN, NaN]]
softmax([1, 5, 2])    : [[0.017147826, 0.93623954, 0.04661262]]
with -inf entries      : [[0.26894143, 0.0, 0.7310586], [NaN, NaN, NaN]]
cross_entropy 0.175456 by hand 0.175456
gather needs equal ranks: shape mismatch in gather, lhs: [2, 3], rhs: [2]
cross_entropy needs rank 2: cross_entropy expects an input tensor of rank 2
mse: 2.5
```

The `-inf` row is the attention-mask case of section 14: a masked entry contributes 0, but a row where *everything* is masked has $-\infty$ as its maximum, and $-\infty - (-\infty)$ is NaN. Never mask a whole row. For a yes/no target (is this hour busy?) with a single logit per item,

`loss::binary_cross_entropy_with_logit(logits, targets)` applies the sigmoid and the two-class form of the same formula; its targets are 0 or 1 values in a float tensor of the logits' own shape, not class indices.

Part C: Convolutions

7. Convolution as a matrix, and its transpose

Absorbs 00_ConvTranspose.md, 00_ConvTranspose2.md and 00_ConvTranspose3.md (the last two were identical files).

7.1 What a convolution does

A convolution slides a small grid of weights (the kernel) over a larger grid (the image or feature map) and, at every position, multiplies the overlapping numbers and adds them up. One kernel produces one output map; a layer has one kernel per output channel, and each kernel spans every input channel.

One naming point before the maths. The signal-processing definition of convolution flips the kernel before sliding it. Deep-learning frameworks, candle included, do not flip: they compute what a signal-processing book calls cross-correlation, and call it convolution. For a learned kernel this changes nothing (the network learns the flipped weights if it needs them), but it matters when you check numbers by hand. The program below proves candle does not flip: a kernel $[1, 10, 100]$ over $[1, 2, 3, 4]$ gives $1 \cdot 1 + 10 \cdot 2 + 100 \cdot 3 = 321$ first, not $100 \cdot 1 + 10 \cdot 2 + 1 \cdot 3 = 123$.

```
use candle_core::{Device, Tensor};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let x = Tensor::new(&[[[1.0f32, 2.0, 3.0, 4.0]]], &dev)?; // (batch, channels, height, width)
    let k = Tensor::new(&[[[1.0f32, 10.0, 100.0]]], &dev)?; // (out, in, kh, kw)
    let y = x.conv2d(&k, 0, 1, 1, 1)?; // padding, stride, dilation, groups
    println!("{:?} -> {:?}: {:?}", x.dims(), y.dims(), y.flatten_all()?.to_vec1:::<f32>()?);
    Ok(())
}
```

```
[1, 1, 1, 4] -> [1, 1, 1, 2]: [321.0, 432.0]
```

7.2 One dimension: the Toeplitz matrix

Convolution is linear, so it is a matrix multiplication $y = C \cdot x$ for some matrix C built from the kernel. For a three-sample signal $x = [x_1, x_2, x_3]$ and a two-tap kernel $h = [h_1, h_2]$, the "full" convolution has four outputs:

```
y1 = h1x1
y2 = h2x1 + h1x2
y3 = h2x2 + h1x3
y4 = h2x3
```

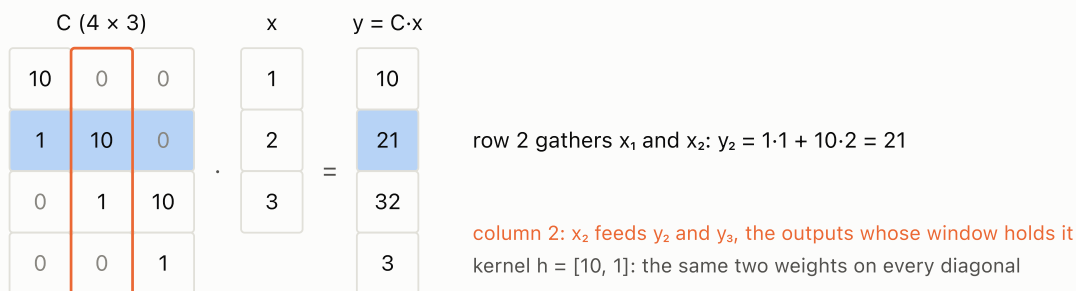
which is

$$\begin{aligned} [y_1] &= [h_1 \ 0 \ 0] [x_1] \\ [y_2] &= [h_2 \ h_1 \ 0] [x_2] \\ [y_3] &= [0 \ h_2 \ h_1] [x_3] \\ [y_4] &= [0 \ 0 \ h_2] [x_4] \end{aligned}$$

Every diagonal of C is constant. A matrix with that property is a Toeplitz matrix, and the constant diagonals are exactly the statement "the same weights are used at every position". These four equations are the textbook, flipped convolution (h_2 multiplies the earlier sample). For candle's cross-correlation, reverse h ; that only re-orders the constant diagonals, so everything said here about the Toeplitz structure holds either way. Section 7.3 builds C in candle's unflipped form.

```
fn main() {
    let x = [1.0f32, 2.0, 3.0];
    let h = [10.0f32, 1.0];
    let (n, k) = (x.len(), h.len());
    let rows = n + k - 1;
    let mut c = vec![vec![0.0f32; n]; rows]; // rows x n Toeplitz matrix
    for (r, row) in c.iter_mut().enumerate() {
        for (j, cell) in row.iter_mut().enumerate() {
            if r >= j && r - j < k {
                *cell = h[r - j]; // h[r - j] on the (r - j)-th diagonal
            }
        }
    }
    let y: Vec<f32> = c.iter().map(|row| row.iter().zip(&x).map(|(a, b)| a * b).sum()).collect();
    for row in &c { println!("{row:?}"); }
    println!("y = C x = {y:?}");
}
```

```
[10.0, 0.0, 0.0]
[1.0, 10.0, 0.0]
[0.0, 1.0, 10.0]
[0.0, 0.0, 1.0]
y = C x = [10.0, 21.0, 32.0, 3.0]
```



C^T (section 7.3) reads the same entries column by column: a column scatters one input to its outputs

Figure 7. The 4×3 matrix C of the two-tap kernel has the same weights on every diagonal: each row gathers the samples one output reads, and each column lists the outputs one sample feeds, which is exactly what C^T (section 7.3) and backpropagation (section 7.4) use.

7.3 Two dimensions: a 4×4 image, a 3×3 kernel, a 4×16 matrix

Take the image and kernel below, with no padding and stride 1. The output is 2×2, so C has 4 rows (one per output pixel) and 16 columns (one per input pixel, reading the image row by row).

image	kernel
1 2 3 4	0 -1 0
5 6 7 8	-1 5 -1
9 10 11 12	0 -1 0
13 14 15 16	

Row 1 of C places the nine kernel weights at the positions the kernel covers when it sits in the top-left corner (pixels 1,2,3,5,6,7,9,10,11); row 2 shifts one column right; row 3 one row down; row 4 both. The program builds C, multiplies, and checks candle's conv2d against it. It then transposes C and multiplies the 2×2 result back up to 4×4, and checks candle's conv_transpose2d against *that*.

```
use candle_core::{Device, Tensor};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let image: Vec<f32> = (1..=16).map(|v| v as f32).collect();
    let kernel = [0.0f32, -1.0, 0.0, -1.0, 5.0, -1.0, 0.0, -1.0, 0.0];

    // C is 4 x 16: row r = output pixel r, column j = input pixel j (row-major)
    let mut c = vec![vec![0.0f32; 16]; 4];
    for (r, (oy, ox)) in [(0usize, 0usize), (0, 1), (1, 0), (1, 1)].into_iter().enumerate()
    {
        for ky in 0..3 {
            for kx in 0..3 {
                c[r][(oy + ky) * 4 + (ox + kx)] = kernel[ky * 3 + kx];
            }
        }
    }

    let y: Vec<f32> = c.iter().map(|row| row.iter().zip(&image).map(|(a, b)| a * b).sum()).collect();
    let back: Vec<f32> = (0..16).map(|j| (0..4).map(|r| c[r][j] * y[r]).sum()).collect();

    let x_t = Tensor::from_slice(&image, (1, 1, 4, 4), &dev)?;
    let k_t = Tensor::from_slice(&kernel, (1, 1, 3, 3), &dev)?;
    let y_t = x_t.conv2d(&k_t, 0, 1, 1, 1)?;
    let back_t = y_t.conv_transpose2d(&k_t, 0, 0, 1, 1)?; // padding,
                                                         output_padding, stride, dilation

    println!("C·x by hand      : {y:?}");
    println!("candle conv2d      : {:?} shape {:?}", y_t.flatten_all()?.to_vec1::<f32>(),
        y_t.dims());
    println!("Cᵀ·y by hand      : {back:?}");
    println!("conv_transpose2d  : {:?} shape {:?}", back_t.flatten_all()?.to_vec1::<f32>(),
        back_t.dims());
    for row in back_t.squeeze(0)?.squeeze(0)?.to_vec2::<f32>()? {
        println!("    {row:?}");
    }
    Ok(())
}
```

```

C·x by hand      : [6.0, 7.0, 10.0, 11.0]
candle conv2d    : [6.0, 7.0, 10.0, 11.0] shape [1, 1, 2, 2]
CT·y by hand    : [0.0, -6.0, -7.0, 0.0, -6.0, 13.0, 18.0, -7.0, -10.0, 33.0, 38.0, -11.0,
                    0.0, -10.0, -11.0, 0.0]
conv_transpose2d : [0.0, -6.0, -7.0, 0.0, -6.0, 13.0, 18.0, -7.0, -10.0, 33.0, 38.0, -11.0,
                    0.0, -10.0, -11.0, 0.0] shape [1, 1, 4, 4]
                    [0.0, -6.0, -7.0, 0.0]
                    [-6.0, 13.0, 18.0, -7.0]
                    [-10.0, 33.0, 38.0, -11.0]
                    [0.0, -10.0, -11.0, 0.0]

```

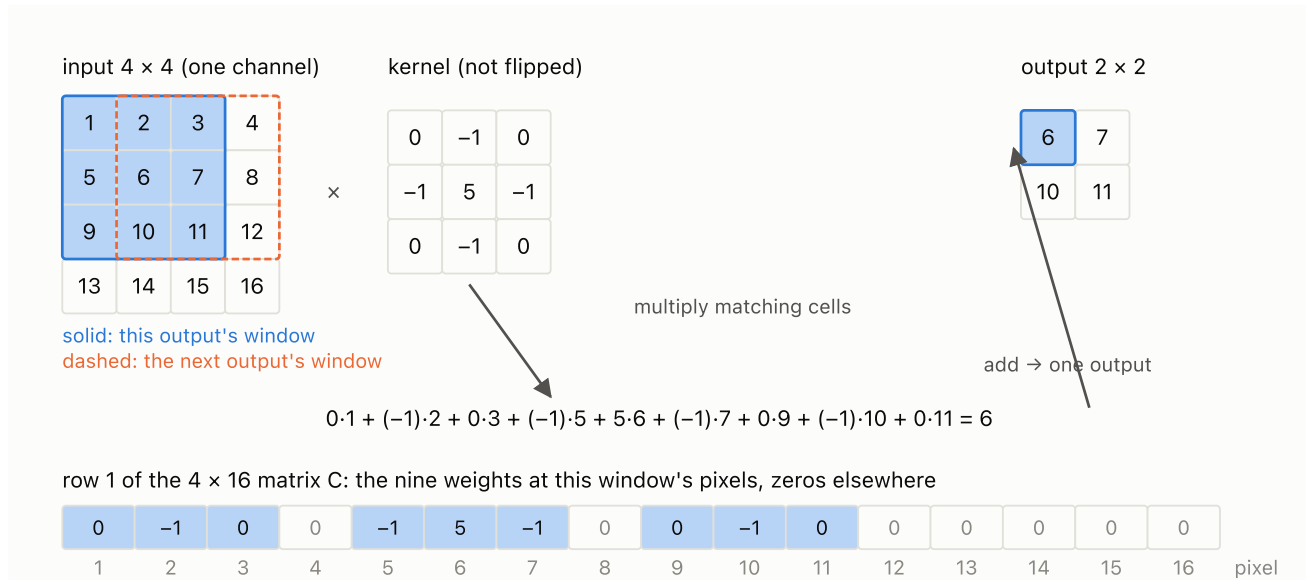


Figure 8. One output pixel of the counter photo is the 3×3 kernel laid on one 3×3 window, multiplied cell by cell and summed (6 for the top-left window, with no flip), and the same nine weights written at that window's pixels are row 1 of the 4×16 matrix C.

7.4 The transposed convolution is the gradient of the convolution

Why does C^T matter? Because it is what backpropagation multiplies by. If $y = C \cdot x$, then the gradient of any loss with respect to x is C^T times the gradient with respect to y . PyTorch's documentation for `ConvTranspose2d` says exactly this: "This module can be seen as the gradient of `Conv2d` with respect to its input." It goes on: it "is also known as a fractionally-strided convolution or a deconvolution (although it is not an actual deconvolution operation as it does not compute a true inverse of convolution)".

candle's automatic differentiation (autograd) lets you check the claim directly. Make the input a `Var`, compute `sum(conv(x) · y)` whose gradient with respect to `x` is by construction $C^T y$, and compare with `conv_transpose2d(y)`:

```
use candle_core::{Device, Tensor, Var};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let image: Vec<f32> = (1..=16).map(|v| v as f32).collect();
    let x = Var::from_tensor(&Tensor::from_slice(&image, (1, 1, 4, 4), &dev)?);
    let k = Tensor::new(&[[[0.0f32, -1.0, 0.0], [-1.0, 5.0, -1.0], [0.0, -1.0, 0.0]]], &dev)?;
    let y = x.as_tensor().conv2d(&k, 0, 1, 1, 1)?.detach();           // (1,1,2,2), treated
                                as a constant

    let score = (x.as_tensor().conv2d(&k, 0, 1, 1, 1)? * &y)?.sum_all()?;
    let grads = score.backward()?;
    let grad_x = grads.get(&x).expect("x is a variable in the graph");
    let transposed = y.conv_transpose2d(&k, 0, 0, 1, 1)?;
    let diff = (grad_x - &transposed)?.abs()?.max_all()?.to_scalar::<f32>()?;
    println!("grad wrt input equals conv_transpose2d(y): {}", diff == 0.0);
    Ok(())
}
```

```
grad wrt input equals conv_transpose2d(y): true
```

That is the whole reason the operation exists as a layer: a network that must produce an output *larger* than its input (a segmentation mask at full resolution, an image from a small code) can use C^T as a learnable upsampling step, and the same weights can be learned because gradient descent through a transposed convolution is a plain convolution.

7.5 Fractional strides, zero insertion, and the output-size formula

A convolution with stride 2 halves the resolution. Its transpose doubles it, which is why it is also called "fractionally strided": moving the kernel two steps over the *output* is like moving it half a step over the *input*. Dumoulin and Visin's guide to convolution arithmetic gives the concrete picture: a transposed convolution with stride s is an ordinary convolution applied to the input after inserting $s - 1$ zeros between neighbouring input values (real implementations skip the multiplications by those zeros). The output size is

$$H_{\text{out}} = (H_{\text{in}} - 1) \cdot \text{stride} - 2 \cdot \text{padding} + \text{dilation} \cdot (\text{kernel} - 1) + \text{output_padding} + 1$$

which is PyTorch's formula and candle's behaviour. `output_padding` adds rows and columns on one side to resolve the ambiguity that a strided convolution introduces: with a 3×3 kernel, padding 1 and stride 2, an input of 7 and an input of 8 both give 4, so going back from 4 needs to be told which one you meant.

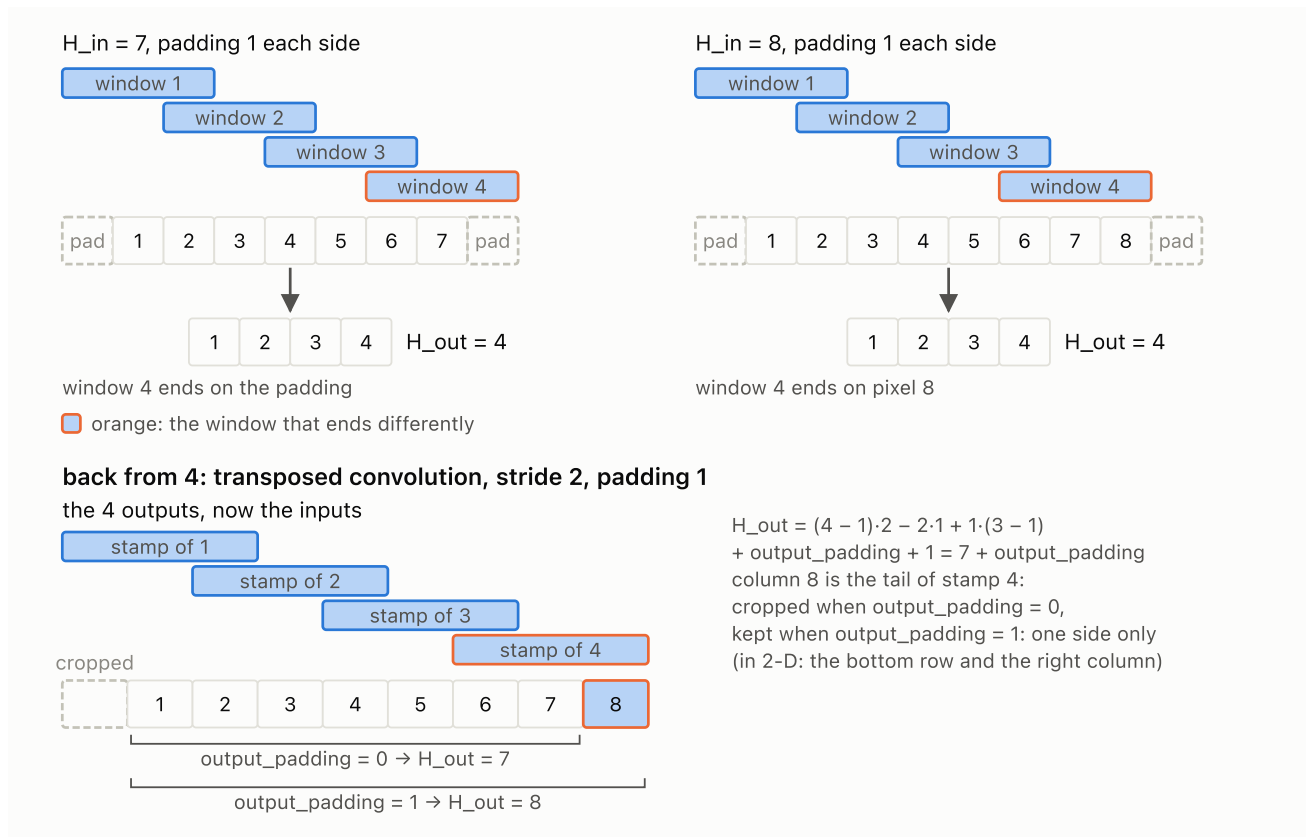


Figure 9. With a 3×3 kernel, stride 2 and padding 1, a 7-pixel and an 8-pixel row of the counter photo both give 4 outputs; going back, the transposed convolution stamps the 4 outputs into 7 pixels, and output_padding = 1 keeps the tail of the last stamp as an eighth pixel at the end of the axis, restoring the size, not the pixel values.

candle's kernel layout for `conv_transpose2d` is `(in_channels, out_channels, kh, kw)`, the reverse of `conv2d`'s `(out_channels, in_channels, kh, kw)`.
`candle_nn::ConvTranspose2dConfig` has four fields, `padding`, `output_padding`, `stride`, `dilation`; there is no `groups` field.

```
use candle_core::{DType, Device, Tensor};
use candle_nn::{ConvTranspose2dConfig, VarBuilder, VarMap, conv_transpose2d};

fn out_size(h: usize, s: usize, p: usize, d: usize, k: usize, op: usize) -> usize {
    (h - 1) * s + d * (k - 1) + op + 1 - 2 * p
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    println!("H_in stride pad k out_pad | formula candle");
    for (h, s, p, k, op) in [(2usize, 1usize, 0usize, 3usize, 0usize), (2, 2, 0, 3, 0), (2,
    2, 1, 4, 0), (2, 2, 0, 3, 1), (7, 2, 1, 3, 1)] {
        let x = Tensor::ones((1, 1, h, h), DType::F32, &dev)?;
        let kernel = Tensor::ones((1, 1, k, k), DType::F32, &dev)?;
        let y = x.conv_transpose2d(&kernel, p, op, s, 1)?;
        println!("{h:>4} {s:>6} {p:>3} {k} {op:>7} | {:>7} {:?}", out_size(h, s, p, 1, k,
        op), y.dims());
    }
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let up = conv_transpose2d(2, 3, 3, ConvTranspose2dConfig { stride: 2, padding: 1,
    output_padding: 1, ..Default::default() }, vb.pp("up"))?;
    println!("weight layout (in, out, kh, kw): {:?}", up.weight().dims());
    let x = Tensor::ones((1, 2, 8, 8), DType::F32, &dev)?;
    println!("{(1,2,8,8) -> {:>7}}", candle_core::Module::forward(&up, &x)?.dims());
    Ok(())
}
```

```
H_in stride pad k out_pad | formula candle
 2      1  0 3      0 |      4 [1, 1, 4, 4]
 2      2  0 3      0 |      5 [1, 1, 5, 5]
 2      2  1 4      0 |      4 [1, 1, 4, 4]
 2      2  0 3      1 |      6 [1, 1, 6, 6]
 7      2  1 3      1 |     14 [1, 1, 14, 14]
weight layout (in, out, kh, kw): [2, 3, 3, 3]
(1,2,8,8) -> [1, 3, 16, 16]
```

7.6 Checkerboard artifacts

When the kernel size is not a multiple of the stride, some output pixels receive contributions from more kernel positions than others. With all-ones input and kernel, the output is that contribution count, and you can see the unevenness in one row. Odena, Dumoulin and Olah showed in 2016 that this is where the checkerboard patterns in generated images come from: "deconvolution has uneven

overlap when the kernel size (the output window size) is not divisible by the stride (the spacing between points on the top)." Their fix is to upsample by nearest-neighbour or bilinear resizing and then apply an ordinary convolution, or at least to choose a kernel size divisible by the stride.

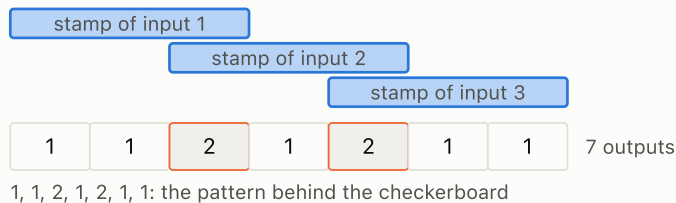
```
use candle_core::{DType, Device, Tensor};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    for (k, s) in [(3usize, 2usize), (4, 2), (2, 2)] {
        let x = Tensor::ones((1, 1, 3, 3), DType::F32, &dev)?;
        let kernel = Tensor::ones((1, 1, k, k), DType::F32, &dev)?;
        let y = x.conv_transpose2d(&kernel, 0, 0, s, 1)?;
        let rows = y.squeeze(0)?.squeeze(0)?.to_vec2::<f32>()?;
        println!("kernel {k} stride {s}: output {:?}, middle row {:?}", y.dims(),
            rows[rows.len() / 2]);
    }
    Ok(())
}
```

```
kernel 3 stride 2: output [1, 1, 7, 7], middle row [1.0, 1.0, 2.0, 1.0, 2.0, 1.0, 1.0]
kernel 4 stride 2: output [1, 1, 8, 8], middle row [2.0, 2.0, 4.0, 4.0, 4.0, 4.0, 2.0, 2.0]
kernel 2 stride 2: output [1, 1, 6, 6], middle row [1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

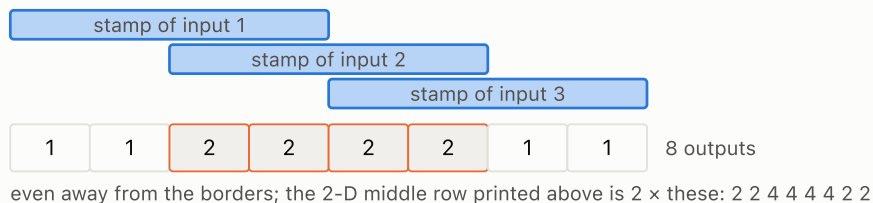
3 with stride 2 alternates between one and two contributions; 4 with stride 2 and 2 with stride 2 are even away from the borders.

kernel 3, stride 2, 3 inputs



cells outlined in orange
receive two stamps

kernel 4, stride 2, 3 inputs



kernel 2, stride 2, 3 inputs

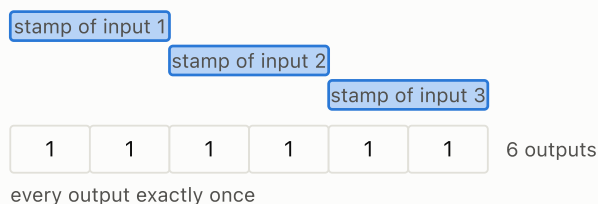


Figure 10. A transposed convolution stamps its kernel once per input pixel of the counter photo, two steps apart: a 3-wide kernel gives the seven outputs 1, 1, 2, 1, 2, 1, 1 stamps (the checkerboard), a 4-wide kernel is even away from the borders, and a 2-wide kernel gives every output exactly one.

7.7 What runs in practice

Nobody builds C. For a 224×224 image and a 3×3 kernel C would have about 2.5 billion entries, almost all zero. Frameworks use one of three strategies: rearrange the input patches into a dense matrix and call a fast matrix multiply (im2col followed by GEMM, a general matrix-multiply routine, at the cost of duplicating overlapping patches in memory); compute those patch indices on the fly inside the matrix-multiply kernel so nothing is duplicated (implicit GEMM, what GPU libraries do); or, for large kernels, multiply in the frequency domain (FFT convolution). Which one wins depends on shapes and hardware, and libraries pick per layer. The matrix view is for understanding, not for implementation.

8. 1×1 convolutions

Absorbs `00_1x1_Convolutions.md`.

8.1 A 1×1 convolution is a Linear applied at every pixel

A 1×1 kernel looks at one pixel at a time, but it looks at *all the channels* of that pixel. With C_{in} input channels and C_{out} output channels its weight has shape $(C_{out}, C_{in}, 1, 1)$, which is a (C_{out}, C_{in}) matrix in disguise. At each pixel it computes $W \cdot [c_1, \dots, c_{C_{in}}] + b$: exactly the Linear layer of section 3.2, with the pixel's channel vector as the input. Height and width are untouched; only the channel count changes. Think of it as a learnable colour-space change: RGB to greyscale is a fixed 1×1 convolution with weights $0.299, 0.587, 0.114$.

The program builds a 1×1 convolution from 64 to 128 channels, applies it to a 32×32 map, then does the same job with a `matmul` on the pixels and shows the two agree.

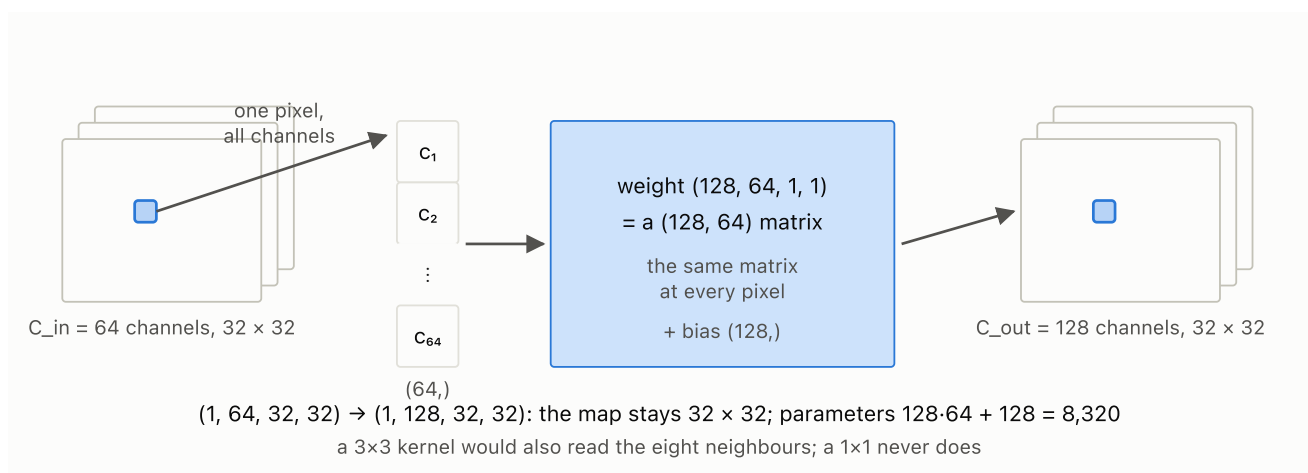


Figure 11. A 1×1 convolution reads one pixel of the counter photo at a time but all 64 of its channels, and multiplies that channel vector by the same $(128, 64)$ matrix at every pixel: the map stays 32×32 , only the channel count changes, and the layer has 8,320 parameters.

```

use candle_core::{DType, Device, Module, Tensor};
use candle_nn::{Conv2dConfig, VarBuilder, VarMap, conv2d};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let cfg = Conv2dConfig { padding: 0, stride: 1, dilation: 1, groups: 1,
        ..Default::default() };
    let mix = conv2d(64, 128, 1, cfg, vb.pp("mix"))?;           // in, out, kernel
                                                                // size, config, names

    let x = Tensor::randn(0f32, 1.0, (1, 64, 32, 32), &dev)?;
    let y = mix.forward(&x)?;
    println!("{:?} -> {:?}; weight {:?}; parameters {}", x.dims(), y.dims(),
        mix.weight().dims(),
        mix.weight().elem_count() + mix.bias().unwrap().elem_count());

    let (b, c, h, w) = x.dims4()?;
    let pixels = x.permute((0, 2, 3, 1))?.reshape((b * h * w, c))?;           // (pixels,
    C_in)
    let wmat = mix.weight().reshape((128, 64))?;                             // (C_out,
    C_in)
    let per_pixel = pixels.matmul(&wmat.t())?.broadcast_add(mix.bias().unwrap())?
        .reshape((b, h, w, 128))?.permute((0, 3, 1, 2))?;                   // back to
    (b, C_out, h, w)
    let diff = (per_pixel - &y)?.abs()?.max_all()?.to_scalar::<f32>()?;
    println!("same as a per-pixel Linear: {}", diff < 1e-5);
    println!("default config: {:?}", Conv2dConfig::default());
    Ok(())
}

```

```

[1, 64, 32, 32] -> [1, 128, 32, 32]; weight [128, 64, 1, 1]; parameters 8320
same as a per-pixel Linear: true
default config: Conv2dConfig { padding: 0, stride: 1, dilation: 1, groups: 1, cudnn_fwd_algo: None }

```

Two candle details are hiding in that program:

- `Conv2dConfig` has a fifth field, `cudnn_fwd_algo`, on both 0.9.1 and 0.11.0. A struct literal that names only padding, stride, dilation and groups, which is what every example in the old note did, does not compile. Write `..Default::default()`, or start from `Conv2dConfig::default()` and change fields.
- `conv2d(in, out, k, cfg, vb)` creates the weight as `(out, in / groups, k, k)` with Kaiming-normal initialisation and a bias with uniform $\pm 1/\sqrt{\text{in}}$. `Conv2d::new(weight, bias, cfg)` wraps tensors you already have.

```

// X expected: missing field `cudnn_fwd_algo` in initializer of `Conv2dConfig`
use candle_nn::Conv2dConfig;

fn main() {
    let cfg = Conv2dConfig { padding: 0, stride: 1, dilation: 1, groups: 1 };
    println!("{cfg:?}");
}

```

8.2 Counting parameters and multiplications

A $k \times k$ convolution from C_{in} to C_{out} channels has $k^2 \cdot C_{in} \cdot C_{out}$ weights plus C_{out} biases, and costs $H \cdot W \cdot k^2 \cdot C_{in} \cdot C_{out}$ multiply-adds per image. A 1×1 convolution is the $k = 1$ case: nine times cheaper than 3×3 , twenty-five times cheaper than 5×5 , for the same channel change.

```
fn params(k: usize, c_in: usize, c_out: usize) -> usize { k * k * c_in * c_out + c_out }

fn main() {
    println!("{:>12} {:>10} {:>10} {:>12}", "channels", "1x1", "3x3", "5x5");
    for (c_in, c_out) in [(3usize, 2usize), (512, 64), (256, 256)] {
        println!("{:>12} {:>10} {:>10} {:>12}", format!("{c_in}->{c_out}"), params(1, c_in,
            c_out), params(3, c_in, c_out), params(5, c_in, c_out));
    }
    let (h, w) = (32usize, 32usize);
    println!("multiply-adds on a 32x32 map, 64->128: 1x1 {} vs 3x3 {}", h * w * 64 * 128, h
        * w * 9 * 64 * 128);
}
```

channels	1x1	3x3	5x5
3->2	8	56	152
512->64	32832	294976	819264
256->256	65792	590080	1638656
multiply-adds on a 32x32 map, 64->128: 1x1 8388608 vs 3x3 75497472			

(The old note gave 295,040 for the 3×3 case of $512 \rightarrow 64$; the correct figure is 294,976.)

8.3 What they are used for

Reducing channels before an expensive layer (the bottleneck). GoogLeNet's authors wrote that " 1×1 convolutions are used to compute reductions before the expensive 3×3 and 5×5 convolutions", adding that they "also include the use of rectified linear activation". Going from 192 to 32 channels with a 5×5 directly costs $25 \cdot 192 \cdot 32 = 153,600$ weights; going $192 \rightarrow 16$ with a 1×1 first and then $16 \rightarrow 32$ with the 5×5 costs $192 \cdot 16 + 25 \cdot 16 \cdot 32 = 15,872$, about a tenth.

The ResNet bottleneck. The 50-, 101- and 152-layer ResNets use a block of " 1×1 , 3×3 , and 1×1 convolutions, where the 1×1 layers are responsible for reducing and then increasing (restoring) dimensions, leaving the 3×3 layer a bottleneck with smaller input/output dimensions". The paper's example is $256 \rightarrow 64 \rightarrow 64 \rightarrow 256$, and the block adds its input back before the final ReLU.

The MobileNetV2 inverted residual. The opposite order: a 1×1 *expands* the channels six-fold (32 to 192 in the block below; the paper uses expansion factor 6 for every block but its first), a cheap depthwise 3×3 filters each channel (section 9), and a 1×1 *projects* back to 32 with no activation after it (a "linear bottleneck", to avoid destroying information in the narrow tensor). ReLU6, $\min(\max(x, 0), 6)$, is used for its robustness in low-precision arithmetic, and the shortcut is added when the stride is 1 and the channel counts match.

Adding depth without spatial cost. Stacking $1 \times 1 \rightarrow \text{ReLU} \rightarrow 1 \times 1 \rightarrow \text{ReLU}$ adds non-linear capacity at every pixel. Without the activations the stack would collapse into a single 1×1 , for the same reason two Linear layers collapse (section 5.2).

Replacing the fully connected classifier. Network in Network (Lin, Chen and Yan, 2013) is the origin of the technique. Their "cross channel parametric pooling layer is also equivalent to a convolution layer with 1×1 convolution kernel", and they replaced the final fully connected layers by global average pooling over each class's feature map. On CIFAR-10 their Table 5 reports 11.59% test

error with fully connected layers, 10.88% with dropout added, and 10.41% with global average pooling instead. The old note's "from 15.99% to 10.41%" compared two different networks: in the paper's section 4.6 a *conventional* CNN reaches 17.56% with a fully connected layer, 15.99% with dropout added, and 16.46% with global average pooling instead; the 10.41% belongs to the mlpconv network of Table 5.

The next program builds both residual blocks and reports shapes and parameter counts; the numbers match the arithmetic above (70,016 for the ResNet bottleneck against 590,080 for a plain 3×3 at 256 channels).

```

use candle_core::{DType, Device, Module, Tensor};
use candle_nn::{Conv2d, Conv2dConfig, VarBuilder, VarMap, conv2d};

fn cfg(padding: usize, groups: usize) -> Conv2dConfig {
    Conv2dConfig { padding, groups, ..Default::default() }
}

/// ResNet bottleneck: 256 -> 64 (1x1) -> 64 (3x3) -> 256 (1x1), plus the input.
struct Bottleneck { reduce: Conv2d, process: Conv2d, expand: Conv2d }

impl Bottleneck {
    fn new(vb: VarBuilder) -> candle_core::Result<Self> {
        Ok(Self {
            reduce: conv2d(256, 64, 1, cfg(0, 1), vb.pp("reduce"))?,
            process: conv2d(64, 64, 3, cfg(1, 1), vb.pp("process"))?,
            expand: conv2d(64, 256, 1, cfg(0, 1), vb.pp("expand"))?,
        })
    }
}

impl Module for Bottleneck {
    fn forward(&self, x: &Tensor) -> candle_core::Result<Tensor> {
        let h = self.reduce.forward(x)?.relu()?;
        let h = self.process.forward(&h)?.relu()?;
        let h = self.expand.forward(&h)?;
        (h + x)?.relu() // residual connection, then ReLU
    }
}

/// MobileNetV2 inverted residual: 32 -> 192 (1x1, ReLU6) -> 192 (depthwise 3x3, ReLU6) ->
/// 32 (1x1, linear), plus the input.
struct InvertedResidual { expand: Conv2d, depthwise: Conv2d, project: Conv2d }

impl InvertedResidual {
    fn new(vb: VarBuilder) -> candle_core::Result<Self> {
        Ok(Self {
            expand: conv2d(32, 192, 1, cfg(0, 1), vb.pp("expand"))?,
            depthwise: conv2d(192, 192, 3, cfg(1, 192), vb.pp("depthwise"))?, // groups =
                channels
            project: conv2d(192, 32, 1, cfg(0, 1), vb.pp("project"))?,
        })
    }
}

impl Module for InvertedResidual {
    fn forward(&self, x: &Tensor) -> candle_core::Result<Tensor> {
        let h = self.expand.forward(x)?.clamp(0f32, 6f32)?; // ReLU6
        let h = self.depthwise.forward(&h)?.clamp(0f32, 6f32)?;
        let h = self.project.forward(&h)?; // no activation:
            linear bottleneck
        h + x
    }
}

fn count(varmap: &VarMap, prefix: &str) -> usize {
    varmap.data().lock().unwrap().iter().filter(|(k, _)| k.starts_with(prefix)).map(|(_, v)|
        v.elem_count()).sum()
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);

```

```

let resnet = Bottleneck::new(vb.pp("resnet"))?;
let mobile = InvertedResidual::new(vb.pp("mobile"))?;
let plain = conv2d(256, 256, 3, cfg(1, 1), vb.pp("plain"))?;

let x = Tensor::randn(0f32, 1.0, (1, 256, 56, 56), &dev)?;
println!("resnet bottleneck : {:?} -> {:?}, {} parameters", x.dims(),
    resnet.forward(&x)?.dims(), count(&varmap, "resnet"));
println!("plain 3x3 256->256: {} parameters", count(&varmap, "plain"));
let x = Tensor::randn(0f32, 1.0, (1, 32, 56, 56), &dev)?;
println!("inverted residual : {:?} -> {:?}, {} parameters (depthwise weight {:?})",
    x.dims(), mobile.forward(&x)?.dims(), count(&varmap, "mobile"),
    mobile.depthwise.weight().dims());
let _ = plain;
Ok(())
}

```

```

resnet bottleneck : [1, 256, 56, 56] -> [1, 256, 56, 56], 70016 parameters
plain 3x3 256->256: 590080 parameters
inverted residual : [1, 32, 56, 56] -> [1, 32, 56, 56], 14432 parameters (depthwise weight [192, 1, 3, 3])

```

8.4 A small classifier for counter photos, ending in global average pooling

The last program of this section is a complete network in the Network-in-Network style: two 3×3 convolutions, a 1×1 reduction between them, a 1×1 expansion after them, global average pooling, and a final 1×1 convolution that maps 128 channels to 10 classes. Because the pooled map is 1×1 , that last convolution is the classifier, and there is no fully connected layer to overfit. `mean_keepdim` over the two spatial axes is the pooling; `squeeze` removes the two size-1 axes at the end.

```
use candle_core::{D, DType, Device, Module, Tensor};
use candle_nn::{Conv2d, Conv2dConfig, VarBuilder, VarMap, conv2d, loss};

struct CounterNet { c1: Conv2d, reduce: Conv2d, c2: Conv2d, expand: Conv2d, classify: Conv2d
}

impl CounterNet {
    fn new(vb: VarBuilder) -> candle_core::Result<Self> {
        let k3 = Conv2dConfig { padding: 1, ..Default::default() };
        let k1 = Conv2dConfig::default();
        Ok(Self {
            c1: conv2d(3, 64, 3, k3, vb.pp("c1"))?,
            reduce: conv2d(64, 32, 1, k1, vb.pp("reduce"))?,
            c2: conv2d(32, 64, 3, k3, vb.pp("c2"))?,
            expand: conv2d(64, 128, 1, k1, vb.pp("expand"))?,
            classify: conv2d(128, 10, 1, k1, vb.pp("classify"))?,
        })
    }
}

impl Module for CounterNet {
    fn forward(&self, x: &Tensor) -> candle_core::Result<Tensor> {
        let h = self.c1.forward(x)?.relu()?;
        let h = self.reduce.forward(&h)?.relu()?;
        let h = self.c2.forward(&h)?.relu()?;
        let h = self.expand.forward(&h)?.relu()?;
        let pooled = h.mean_keepdim(D::Minus2)?.mean_keepdim(D::Minus1)?; // (batch, 128, 1, 1)
        self.classify.forward(&pooled)?.squeeze(3)?.squeeze(2) // (batch, 10)
    }
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let net = CounterNet::new(vb)?;
    let photos = Tensor::randn(0f32, 1.0, (4, 3, 32, 32), &dev)?; // 4 RGB photos, 32x32
    let labels = Tensor::new(&[2u32, 5, 1, 9], &dev)?;
    let logits = net.forward(&photos)?;
    let l = loss::cross_entropy(&logits, &labels)?;
    println!("{:?} -> logits {:?}; loss is finite: {}", photos.dims(), logits.dims(),
        l.to_scalar::<f32>()?.is_finite());
    let n: usize = varmap.all_vars().iter().map(|v| v.elem_count()).sum();
    println!("parameters: {n}");
    Ok(())
}
```

```
[4, 3, 32, 32] -> logits [4, 10]; loss is finite: true
parameters: 31978
```

8.5 When not to use them

A 1×1 convolution cannot see neighbours, so it cannot detect an edge, a corner or a cup rim; that needs a kernel with spatial extent. If a layer's channel count is already small there is nothing to reduce. And aggressive reduction in a shallow network removes capacity the network needs; the deep ResNets tolerate a four-fold bottleneck because they have many blocks in which to recover.

9. Grouped and depthwise convolutions

Absorbs 00_Convolutions_grouped.md, 00_Convolutions_grouped1.md, 00_depthWiseConv.md and 00_depthWiseConv1.md.

9.1 The rule: each filter sees C_{in} / g groups channels

A standard convolution connects every output channel to every input channel: each of the C_{out} kernels has depth C_{in} . A grouped convolution with g groups splits the input channels into g slabs and the output channels into g slabs, and connects slab to slab only. Each kernel now has depth C_{in} / g , so the weight tensor has shape $(C_{out}, C_{in} / g, k, k)$, and both C_{in} and C_{out} must be divisible by g .

groups = 1, $C_{in} = 4$, $C_{out} = 4$
weight $(4, 4, k, k)$

C1 C2 C3 C4 $\rightarrow$ F1 $\rightarrow$ O1
C1 C2 C3 C4 $\rightarrow$ F2 $\rightarrow$ O2
C1 C2 C3 C4 $\rightarrow$ F3 $\rightarrow$ O3
C1 C2 C3 C4 $\rightarrow$ F4 $\rightarrow$ O4

groups = $C_{in} = 4$ (depthwise)
weight $(4, 1, k, k)$

C1 $\rightarrow$ F1 $\rightarrow$ O1
C2 $\rightarrow$ F2 $\rightarrow$ O2
C3 $\rightarrow$ F3 $\rightarrow$ O3
C4 $\rightarrow$ F4 $\rightarrow$ O4

groups = 2, $C_{in} = 4$, $C_{out} = 4$
weight $(4, 2, k, k)$

C1 C2 $\rightarrow$ F1 $\rightarrow$ O1
C1 C2 $\rightarrow$ F2 $\rightarrow$ O2
C3 C4 $\rightarrow$ F3 $\rightarrow$ O3
C3 C4 $\rightarrow$ F4 $\rightarrow$ O4

depthwise separable
depthwise $(4, 1, k, k)$ then pointwise 1×1 $(M, 4, 1, 1)$

C1..C4 $\xrightarrow{\text{depthwise}}$ D1..D4 $\xrightarrow{1 \times 1}$ O1..OM
(mixing happens only here)

The extreme $g = C_{in}$ is a depthwise convolution: one 2-D kernel per input channel, no mixing between channels at all. A depthwise convolution followed by a 1×1 convolution (the "pointwise" step, which does the mixing) is a depthwise separable convolution, the building block of MobileNet. The old note asked whether "a standard convolution has a 3-D kernel and a depthwise one a 2-D kernel": yes, in the sense that a standard kernel is $k \times k \times C_{in}$ and a depthwise kernel is $k \times k \times 1$.

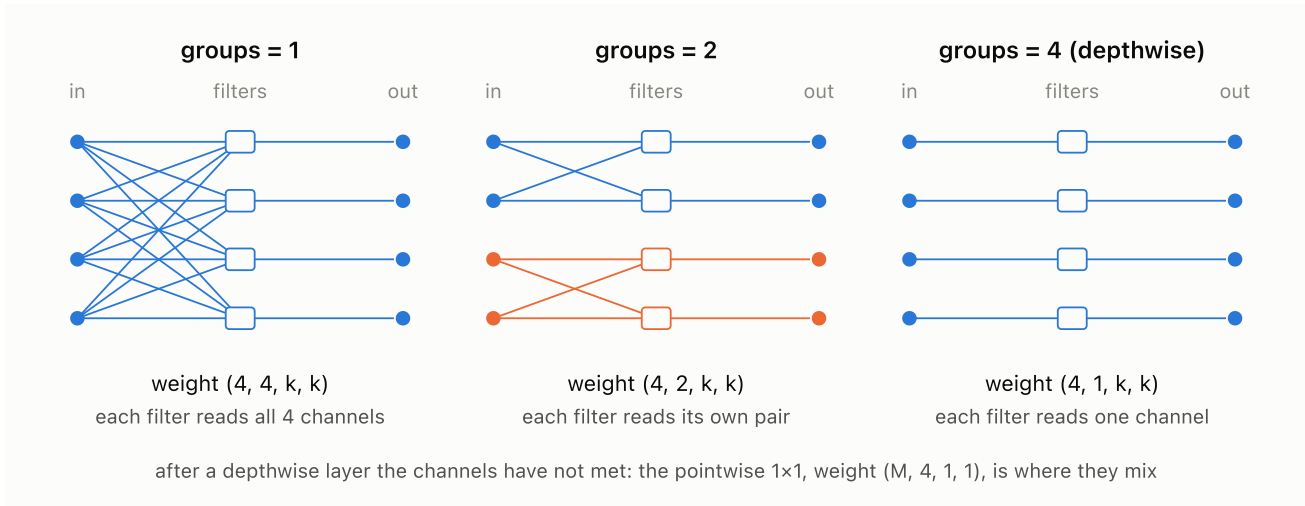


Figure 12. With groups = 1 every filter reads all four channels of the counter photo (weight (4, 4, k, k)); with groups = 2 each filter reads its own pair (4, 2, k, k); with groups = 4 each filter reads one channel (4, 1, k, k), and only the 1x1 that follows mixes the channels again.

```
use candle_core::{Device, Tensor};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let x = Tensor::randn(0f32, 1.0, (1, 4, 5, 5), &dev)?;

    // grouped (g = 2): equals two separate convolutions on the two channel halves,
    // concatenated
    let w = Tensor::randn(0f32, 1.0, (4, 2, 3, 3), &dev)?; // (C_out, C_in /
    // 2, 3, 3)
    let grouped = x.conv2d(&w, 1, 1, 1, 2)?; // padding 1,
    // stride 1, dilation 1, groups 2
    let half0 = x.narrow(1, 0, 2)?.conv2d(&w.narrow(0, 0, 2)?, 1, 1, 1, 1)?;
    let half1 = x.narrow(1, 2, 2)?.conv2d(&w.narrow(0, 2, 2)?, 1, 1, 1, 1)?;
    let manual = Tensor::cat(&[half0, half1], 1)?;
    let same = (&grouped - &manual)?.abs()?.max_all()?.to_scalar:::<f32>()? == 0.0;
    println!("grouped {:?}, equals per-group convolutions: {same}", grouped.dims());
    let bad = Tensor::randn(0f32, 1.0, (4, 4, 3, 3), &dev)?;
    println!("full-depth weight with groups 2: {}", x.conv2d(&bad, 1, 1, 1,
    2).unwrap_err());

    // depthwise (g = C_in = 4): channel 2 of the output depends on channel 2 of the input
    // only
    let wd = Tensor::randn(0f32, 1.0, (4, 1, 3, 3), &dev)?;
    let depthwise = x.conv2d(&wd, 1, 1, 1, 4)?;
    let ch2 = x.narrow(1, 2, 1)?.conv2d(&wd.narrow(0, 2, 1)?, 1, 1, 1, 1)?;
    let same = (depthwise.narrow(1, 2, 1)? - ch2)?.abs()?.max_all()?.to_scalar:::<f32>()? ==
    0.0;
    println!("depthwise {:?}, channel 2 equals a single-channel convolution: {same}",
    depthwise.dims());

    // depthwise separable: depthwise, then a 1x1 that mixes the 4 channels into 8
    let wp = Tensor::randn(0f32, 1.0, (8, 4, 1, 1), &dev)?;
    println!("separable output {:?}", depthwise.conv2d(&wp, 0, 1, 1, 1)?.dims());
    Ok(())
}
```

```
grouped [1, 4, 5, 5], equals per-group convolutions: true
full-depth weight with groups 2: in_channel mismatch between input (4, groups 2) and kernel (4)
depthwise [1, 4, 5, 5], channel 2 equals a single-channel convolution: true
separable output [1, 8, 5, 5]
```

9.2 What it saves

For a $k \times k$ layer from N input channels to N output channels on an $H \times W$ map, the standard cost is $H \cdot W \cdot k^2 \cdot N \cdot N$ multiply-adds. The separable version costs $H \cdot W \cdot k^2 \cdot N$ for the depthwise step plus $H \cdot W \cdot N \cdot N$ for the pointwise step, so the ratio is $1/N + 1/k^2$. The MobileNet paper states the consequence: "MobileNet uses 3×3 depthwise separable convolutions which uses between 8 to 9 times less computation than standard convolutions at only a small reduction in accuracy [...]".

```
fn main() {
    let (k, c_in, c_out) = (3usize, 32usize, 64usize);
    let standard = k * k * c_in * c_out;
    let depthwise = k * k * c_in;
    let pointwise = c_in * c_out;
    println!("3x3, {c_in} -> {c_out} channels, no biases:");
    println!("  standard    {standard}");
    println!("  depthwise    {depthwise} + pointwise {pointwise} = {}", depthwise +
        pointwise);
    println!("  ratio        {:.2}x fewer; 1/N + 1/k^2 = {:.4}", standard as f64 / (depthwise
        + pointwise) as f64, 1.0 / c_out as f64 + 1.0 / (k * k) as f64);
}
```

```
3x3, 32 -> 64 channels, no biases:
standard    18432
depthwise    288 + pointwise 2048 = 2336
ratio        7.89x fewer; 1/N + 1/k^2 = 0.1267
```

9.3 What it costs, and where the idea came from

Expressiveness. Filters in a group cannot see channels outside it. The ShuffleNet authors put it plainly: "Outputs from a certain group only relate to the inputs within the group. This property blocks information flow between channel groups and weakens representation." Their fix, channel shuffle, reshapes the output channels to (g, n) , transposes, and flattens, so the next grouped layer sees a mix of every group. A depthwise separable convolution has the same limitation inside its depthwise step and relies entirely on the 1×1 to mix; it assumes that spatial filtering and channel mixing can be done one after the other, which is an approximation that mobile networks accept for the speed.

Speed on real hardware. Fewer multiply-adds is not the same as less time. A depthwise convolution does very little arithmetic per byte it reads, so on a GPU it is limited by memory bandwidth rather than by compute, and the reduction in wall-clock time is smaller than the reduction in operations. Measure on the target device before assuming.

History. Grouped convolution appeared in AlexNet (2012) for a practical reason, not efficiency: "A single GTX 580 GPU has only 3GB of memory, which limits the maximum size of the networks that can be trained on it. [...] Therefore we spread the net across two GPUs." Half the kernels lived on each GPU and "the GPUs communicate only at certain layers". (The old note said 1.5 GB; the paper says 3 GB.) The efficiency story came later with ResNeXt, MobileNet and ShuffleNet.

In candle a grouped convolution is `Conv2dConfig { groups: g, ..Default::default() }`; `conv2d(in, out, k, cfg, vb)` then allocates the weight as $(out, in / g, k, k)$ for you, as the inverted-residual example in section 8.3 showed (depthwise weight `[192, 1, 3, 3]`).

10. Dilated convolutions, receptive fields and dense prediction

Absorbs `00_dilation.md`, `00_dilation_enhanced.md` and `denseAndSpatial.md`.

10.1 Two goals that pull in opposite directions

Suppose the café camera must label every pixel of a counter photo as cup, saucer, hand, counter or background. This is dense prediction: one decision per pixel. Two things are needed at once.

- **Context.** A patch of white pixels could be a cup, a saucer or a napkin. To decide, the network must see the surroundings: a handle, a rim, a spoon next to it. The region of the input that influences one output value is that unit's *receptive field*, and for context it must be large.
- **Resolution.** To draw the rim of the cup in the right place, the network must keep fine detail all the way to the output. Every time a network halves its feature map with pooling or a stride, that detail is lost and cannot be fully recovered.

The classic image-classification recipe (shrink the map by $32\times$ so that deep units see the whole image) gives context and throws away resolution. Large kernels give context without shrinking, but a 7×7 kernel has 5.4 times the weights of a 3×3 and an 11×11 more than 13 times. Dilated convolutions are the third way.

10.2 Receptive-field arithmetic

Two numbers describe a position in a stack of convolutions: the receptive field rf of one output unit and the $jump$, the distance in input pixels between two neighbouring units of the current map. Starting from $rf = 1$, $jump = 1$, each layer with kernel k , dilation d and stride s updates them as

$$\begin{aligned} rf &\leftarrow rf + (k - 1) \cdot d \cdot jump \\ jump &\leftarrow jump \cdot s \end{aligned}$$

The old notes' 2-D formula $RF_{new} = RF_{old} + 2(d-1)(k-1)$ was wrong, and so was their table for a mixed stack (they gave 9, 17, 33 where the recurrence gives 7, 23, 55). The program computes the tables correctly and reproduces the numbers published by Yu and Koltun for their context module.

```
fn receptive_fields(layers: &[(usize, usize, usize)]) -> Vec<usize> { // (kernel,
    dilation, stride)
    let (mut rf, mut jump) = (1usize, 1usize);
    layers.iter().map(|(k, d, s)| { rf += (k - 1) * d * jump; jump *= s; rf }).collect()
}

fn main() {
    println!("plain 3x3 stack, stride 1           {:?}", receptive_fields(&[(3, 1, 1), (3, 1, 1), (3, 1, 1)]));
    println!("3x3 with dilations 1,2,4,8,16      {:?}", receptive_fields(&[(3, 1, 1), (3, 2, 1), (3, 4, 1), (3, 8, 1), (3, 16, 1)]));
    println!("3x3 with strides 1,2,2,2                {:?}", receptive_fields(&[(3, 1, 1), (3, 1, 2), (3, 1, 2), (3, 1, 2)]));
    println!("mixed: d1 s1, d2 s2, d4 s1, d8 s1          {:?}", receptive_fields(&[(3, 1, 1), (3, 2, 2), (3, 4, 1), (3, 8, 1)]));
    println!("Yu & Koltun 1,1,2,4,8,16,1                 {:?}", receptive_fields(&[(3, 1, 1), (3, 1, 1), (3, 2, 1), (3, 4, 1), (3, 8, 1), (3, 16, 1), (3, 1, 1)]));
}
```

plain 3x3 stack, stride 1	[3, 5, 7]
3x3 with dilations 1,2,4,8,16	[3, 7, 15, 31, 63]
3x3 with strides 1,2,2,2	[3, 5, 9, 17]
mixed: d1 s1, d2 s2, d4 s1, d8 s1	[3, 7, 23, 55]
Yu & Koltun 1,1,2,4,8,16,1	[3, 5, 9, 17, 33, 65, 67]

Three plain 3×3 layers see 7 pixels: the receptive field grows by 2 per layer. Five 3×3 layers with doubling dilations see 63 pixels with the same 45 weights per channel pair, where a single 63×63 kernel would need 3,969. That is the sentence from Yu and Koltun's paper: "The receptive field grows exponentially while the number of parameters grows linearly." Their module's dilations 1, 1, 2, 4, 8, 16, 1 give 3, 5, 9, 17, 33, 65, 67, exactly as they report.

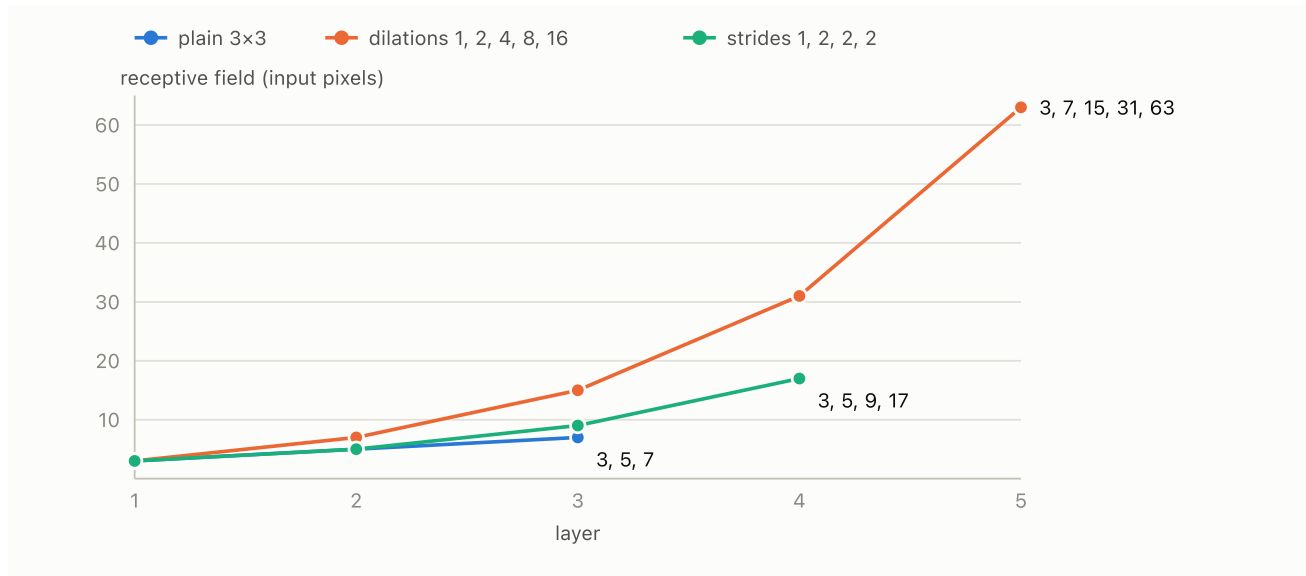


Figure 13. The receptive field of one unit grows by $(k - 1) \cdot d$ -jump per layer: a stack of three undilated 3×3 layers reaches 7 pixels of the counter photo, five layers with dilations 1, 2, 4, 8, 16 reach 63 without adding a single weight (still 45 per channel pair), and strides 1, 2, 2, 2 reach 17 because each stride doubles the jump of every later layer.

10.3 Dilation, effective kernel size, padding and stride

A dilated (or "atrous", from the French for "with holes") convolution keeps the kernel's $k \times k$ weights but spaces them d pixels apart. DeepLab writes it as $y[i] = \sum_k x[i + r \cdot k] \cdot w[k]$ with rate r . The area the kernel covers is

$$k_{\text{eff}} = d \cdot (k - 1) + 1 \quad \text{3x3 with } d = 2 \text{ covers } 5 \times 5; \text{ with } d = 4 \text{ covers } 9 \times 9$$

and the output size along one axis is the ordinary convolution formula with k_{eff} in place of k :

$$L_{\text{out}} = \lfloor (L_{\text{in}} + 2p - d(k - 1) - 1) / s \rfloor + 1$$

To keep $L_{out} = L_{in}$ with stride 1 you need $p = d(k - 1)/2$, which for a 3×3 kernel is $p = d$, not $p = 1$; forgetting this shrinks the map by $2(d - 1)$ pixels per layer. With stride $s > 1$ no padding preserves the size; the symmetric padding $p = \lfloor k_{eff}/2 \rfloor$ (for odd k_{eff}) gives $L_{out} = \lfloor L_{in} / s \rfloor$, which is what most people mean by "same" padding under a stride.

```
use candle_core::{Device, Tensor};

fn out_len(l: usize, k: usize, d: usize, p: usize, s: usize) -> usize { (l + 2 * p - d * (k - 1) - 1) / s + 1 }

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let w = Tensor::ones((1, 1, 3, 3), candle_core::DType::F32, &dev)?;
    println!("L_in d p s | formula candle");
    for (l, d, p, s) in [(128usize, 1usize, 1usize, 1usize), (128, 2, 2, 1), (128, 2, 1, 1),
                        (128, 4, 4, 1), (32, 4, 4, 2), (32, 4, 3, 2), (33, 4, 4, 2)] {
        let x = Tensor::ones((1, 1, l, l), candle_core::DType::F32, &dev)?;
        let y = x.conv2d(&w, p, s, d, 1)?; // padding, stride, dilation, groups
        println!("{l:>4} {d:>2} {p:>2} {s:>2} | {:>7} {:?}" , out_len(l, 3, d, p, s), y.dims());
    }
    Ok(())
}
```

L_in	d	p	s		formula	candle
128	1	1	1		128	[1, 1, 128, 128]
128	2	2	1		128	[1, 1, 128, 128]
128	2	1	1		126	[1, 1, 126, 126]
128	4	4	1		128	[1, 1, 128, 128]
32	4	4	2		16	[1, 1, 16, 16]
32	4	3	2		15	[1, 1, 15, 15]
33	4	4	2		17	[1, 1, 17, 17]

Note the argument order of `Tensor::conv2d`: (kernel, padding, stride, dilation, groups). For the layer form, `Conv2dConfig { padding, stride, dilation, groups, .. }` names them.

10.4 Gridding, and why dilation rates must not share a factor

Stack three layers that all use dilation 4 and look at which input pixels one output unit actually reads. Each layer reads offsets $\{-4, 0, +4\}$ relative to its centre, in units of the previous map, so the stack reads sums of three such offsets: only multiples of 4. Between them are pixels no unit ever sees. Wang and colleagues named this "gridding": "the receptive field of this kernel only covers an area with checkerboard patterns". Their hybrid dilated convolution rule is that "the dilation rate within a group should not have a common factor relationship (like 2,4,8, etc.)", with the formal condition that the largest gap between sampled pixels must not exceed the kernel size; 1, 2, 5 passes and 1, 2, 9 fails.

The program checks that rule in two ways: by computing the set of offsets a stack can reach (the sum of each layer's offset set) and by pushing a single bright pixel through real convolutions with all-ones kernels and counting which outputs light up.

```
use candle_core::{DType, Device, Tensor};

/// Offsets (in input pixels) one output unit reads through a stack of 3-tap layers with
/// these dilations.
fn offsets(dilations: &[usize]) -> Vec<i64> {
    let mut set: Vec<i64> = vec![0];
    for &d in dilations {
        let mut next = Vec::new();
        for &o in &set {
            for step in [-(d as i64), 0, d as i64] { next.push(o + step); }
        }
        next.sort();
        next.dedup();
        set = next;
    }
    set
}

fn main() -> candle_core::Result<()> {
    for dil in [vec![4usize, 4, 4], vec![2, 4, 8], vec![1, 2, 9], vec![1, 2, 5], vec![1, 2,
        4], vec![1, 2, 3]] {
        let o = offsets(&dil);
        let reach = *o.last().unwrap();
        println!("dilations {dil:?}: reach ±{reach}, sees {} of {} positions, no holes: {}",
            o.len(), 2 * reach + 1, o.len() as i64 == 2 * reach + 1);
    }
    let dev = Device::Cpu;
    let ones = Tensor::ones((1, 1, 3, 3), DType::F32, &dev)?;
    for dil in [vec![1usize, 2, 4], vec![4, 4, 4]] {
        let n = 33usize;
        let mut img = vec![0f32; n * n];
        img[(n / 2) * n + n / 2] = 1.0; // one bright
        // pixel in the middle
        let mut t = Tensor::from_slice(&img, (1, 1, n, n), &dev)?;
        for &d in &dil { t = t.conv2d(&ones, d, 1, d, 1)?; } // padding d keeps
        // the size
        let v = t.flatten_all()?.to_vec1::<f32>()?;
        let lit: Vec<usize> = (0..v.len()).filter(|&i| v[i] != 0.0).collect();
        let rows: Vec<usize> = lit.iter().map(|i| i / n).collect();
        let side = rows.iter().max().unwrap() - rows.iter().min().unwrap() + 1;
        println!("impulse through dilations {dil:?}: {} pixels lit inside a {side}x{side}
            box of {} pixels", lit.len(), side * side);
    }
    Ok(())
}
```

```
dilations [4, 4, 4]: reach ±12, sees 7 of 25 positions, no holes: false
dilations [2, 4, 8]: reach ±14, sees 15 of 29 positions, no holes: false
dilations [1, 2, 9]: reach ±12, sees 21 of 25 positions, no holes: false
dilations [1, 2, 5]: reach ±8, sees 17 of 17 positions, no holes: true
dilations [1, 2, 4]: reach ±7, sees 15 of 15 positions, no holes: true
dilations [1, 2, 3]: reach ±6, sees 13 of 13 positions, no holes: true
impulse through dilations [1, 2, 4]: 225 pixels lit inside a 15x15 box of 225 pixels
impulse through dilations [4, 4, 4]: 49 pixels lit inside a 25x25 box of 625 pixels
```

The impulse experiment is the receptive field drawn for you: with dilations 1, 2, 4 every one of the 225 pixels of the 15×15 field is reached; with 4, 4, 4 the field is 25×25 but only 49 pixels of it are ever read.

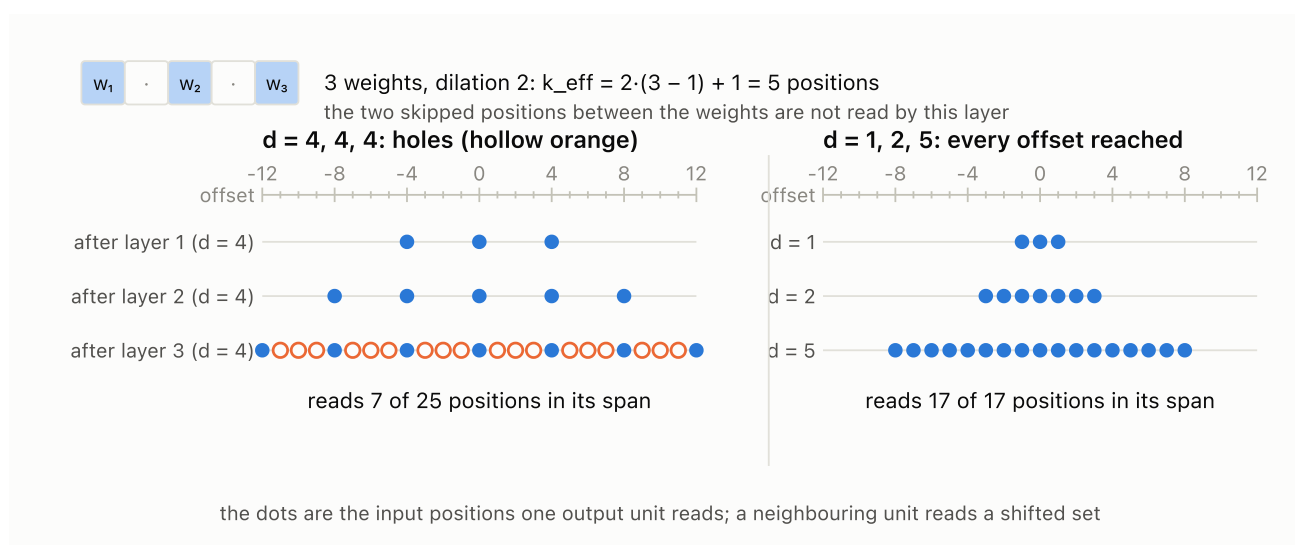


Figure 14. Three layers of dilation 4 read only offsets that are multiples of 4, so one output unit sees 7 of the 25 positions in its span of the counter photo and never the pixels between them, while dilations 1, 2, 5 reach all 17 positions in theirs.

DeepLab's Atrous Spatial Pyramid Pooling sidesteps the problem differently: it runs several dilated 3×3 convolutions *in parallel* on the same map (rates 6, 12 and 18 at output stride 16, doubled at output stride 8; the output stride is the factor by which the feature map is smaller than the image), plus a 1×1 branch and a global-average-pooling branch, and concatenates the results. Each branch sees a different scale; none is stacked on another.

10.5 Where dilation is used

- **Semantic segmentation (DeepLab).** The last pooling stages of a ResNet backbone are replaced by dilated convolutions, so the final map is 8 or 16 times smaller than the image instead of 32, and ASPP adds multi-scale context on top.
- **Audio (WaveNet).** One-dimensional causal convolutions with dilations "1, 2, 4, ..., 512" stacked and repeated; "each 1, 2, 4, ..., 512 block has receptive field of size 1024" samples, with only ten layers per block. Causal means a sample never looks at the future, which is what a generator needs.
- **Time series (temporal convolutional networks).** The same dilated causal stacks applied to sensor or sales data: the till's history over thousands of hours seen by a handful of layers, trainable in parallel unlike a recurrent network.
- **Volumes (medical imaging).** 3-D dilated kernels keep resolution on CT and MRI scans where a small lesion must not vanish under pooling.
- **Real-time segmentation.** Strides in the first layers shrink the map cheaply; dilation in the later layers grows the context without shrinking it further, so the decoder stays small.
- **Mobile networks.** Depthwise separable convolutions (section 9) combined with dilation give a large receptive field for a fraction of the arithmetic.

Dilation is not the only answer to the context-versus-resolution problem. **U-Net** shrinks the map on the way down, grows it again on the way up, and copies the early high-resolution maps across as skip connections, so the decoder has both context and detail. **Vision transformers** let every patch attend

to every other patch (section 14), which is global context from the first layer at quadratic cost; the **Swin Transformer** limits "self-attention computation to non-overlapping local windows while also allowing for cross-window connection", which gives "linear computational complexity with respect to image size".

10.6 Design rules that survive scrutiny

The old notes contained tables of throughput percentages per dilation rate, frame rates, mIoU figures (mean intersection-over-union, the usual segmentation score) and accuracy drops with no source. None of them are repeated here. What can be said with support:

- Start local. Keep the first layers at $d = 1$ so fine detail is captured before context is added; Yu and Koltun's module begins with two undilated layers.
- Increase dilation with depth and avoid a common factor between consecutive rates (1, 2, 5 or 1, 2, 3 rather than 2, 4, 8). Check coverage with the offset-set computation above before trusting a schedule.
- Compute padding from k_{eff} , not k , and verify output shapes with the formula.
- Use stride early, when losing resolution is acceptable and cheap, and dilation late, when it is not. Do not put a large stride and a large dilation in the same layer; both sparsify the sampling and their effects compound.
- A dilated convolution has the same arithmetic cost as its undilated version but reads memory in a scattered pattern; whether it is slower on your hardware is a measurement, not a rule.
- Dilation in a classifier is a trade, not a rule. Dilated Residual Networks (Yu, Koltun and Funkhouser, CVPR 2017) replaced the last strides of a ResNet with dilation and found that the networks "outperform their non-dilated counterparts in image classification without increasing the model's depth or complexity", at the price of larger late feature maps and more computation; they also had to remove gridding artifacts ("degridding") to get there. Strides remain the cheaper default; measure before choosing.

10.7 Formulas at a glance

Quantity	Formula
effective kernel	$k_{\text{eff}} = d(k - 1) + 1$
output length	$\lfloor (L + 2p - d(k - 1) - 1)/s \rfloor + 1$
"same" padding, stride 1	$p = d(k - 1)/2$
"same" under stride (odd k_{eff})	$p = \lfloor k_{\text{eff}}/2 \rfloor$ gives $\lceil L/s \rceil$
receptive field after a layer	$\text{rf} += (k - 1) \cdot d \cdot \text{jump}$, then $\text{jump} *= s$
weights per channel pair	k^2 , independent of d
transposed-convolution output	$(L - 1)s - 2p + d(k - 1) + \text{output_padding} + 1$

Part D: Normalisation and channel attention

11. Batch normalisation

Absorbs 00_Batch Normalization2.md, 00_why_Batch Normalization_works.md, and the normalisation notes in 04_mlp_multi_layer_perceptron2.md.

11.1 What it computes

Batch normalisation (Ioffe and Szegedy, 2015) standardises the numbers flowing between layers so that each channel has mean 0 and variance 1 across the current mini-batch, then lets the network scale and shift them back with two learned numbers per channel. For a channel c , with m values in the batch (for a convolutional map, $m = N \cdot H \cdot W$, every pixel of every image):

μ = mean of the m values	σ^2 = mean of $(x - \mu)^2$ (biased: divide by m)
$\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \epsilon)}$	$\epsilon = 1e-5$ keeps this finite when $\sigma^2 = 0$
$y = \gamma \cdot \hat{x} + \beta$	γ starts at 1, β at 0, both learned

That is training. At inference the batch may be a single café photo, whose "batch mean" is meaningless, so the layer instead uses running estimates of μ and σ^2 accumulated during training with an exponential moving average. The result is deterministic and independent of what else is in the batch. Forgetting to switch to this mode is the classic BatchNorm bug: test predictions that change depending on their neighbours in the batch.

11.2 BatchNorm in candle

`candle_nn::batch_norm(num_features, config, vb)` creates four named tensors, weight (γ), bias (β), `running_mean` and `running_var`, and returns a `BatchNorm`. The defaults are `eps: 1e-5`, `momentum: 0.1`, `affine: true`, `remove_mean: true`. The running statistics are updated as `running * (1 - momentum) + batch * momentum`, PyTorch's convention, and the running variance is updated with the *unbiased* batch variance ($m/(m - 1)$ correction) while the normalisation itself uses the biased one, again as in PyTorch.

BatchNorm implements `ModuleT`, the trait for layers that behave differently in training, so you call `forward_t(&x, true)` or `forward_train(&x)` to train and `forward_t(&x, false)` to infer. It does not implement `Module`, so `bn.forward(&x)` does not compile. The program checks every number by hand: channel 0 of the input holds 1, 2, 3, 4, so its mean is 2.5 and its biased variance 1.25.

```
use candle_core::{DType, Device, ModuleT, Tensor};
use candle_nn::{BatchNormConfig, VarBuilder, VarMap, batch_norm};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let bn = batch_norm(2, BatchNormConfig::default(), vb.pp("bn"))?;
    let mut names: Vec<String> = varmap.data().lock().unwrap().keys().cloned().collect();
    names.sort();
    println!("tensors: {names:?}");
    println!("config: {:?}", BatchNormConfig::default());

    // one image, 2 channels, 2x2 pixels
    let x = Tensor::new(&[[[1.0f32, 2.0], [3.0, 4.0]], [[10.0, 10.0], [10.0, 30.0]]],
        &dev)?;
    let y = bn.forward_train(&x)?;
    println!("train output, channel 0: {:?}", y.narrow(1, 0, 1)?.flatten_all()?.to_vec1::
        <f32>()?);
    println!("running_mean {:?}, running_var {:?}", bn.running_mean().to_vec1::<f32>(),
        bn.running_var().to_vec1::<f32>());

    let mean = 2.5f64;
    let var_biased = 1.25f64;
    let var_unbiased = var_biased * 4.0 / 3.0;
    println!("by hand, channel 0: (1 - 2.5)/sqrt(1.25 + 1e-5) = {:.5}", (1.0 - mean) /
        (var_biased + 1e-5).sqrt());
    println!("by hand: running_mean = 0.9*0 + 0.1*2.5 = {:.3}, running_var = 0.9*1 + 0.1*
        {var_unbiased:.4} = {:.5}", 0.1 * mean, 0.9 + 0.1 * var_unbiased);

    let e = bn.forward_t(&x, false)?; // inference: running statistics
    println!("eval output, channel 0: {:?}", e.narrow(1, 0, 1)?.flatten_all()?.to_vec1::
        <f32>()?);
    println!("by hand: (1 - 0.25)/sqrt(1.06667 + 1e-5) = {:.5}", (1.0 - 0.25) / (0.9 + 0.1 *
        var_unbiased + 1e-5f64).sqrt());
    println!("wrong channel count: {}", bn.forward_t(&Tensor::zeros((1, 3, 2, 2),
        DType::F32, &dev)?, false).unwrap_err());
    Ok(())
}
```

```
tensors: ["bn.bias", "bn.running_mean", "bn.running_var", "bn.weight"]
config: BatchNormConfig { eps: 1e-5, remove_mean: true, affine: true, momentum: 0.1 }
train output, channel 0: [-1.3416355, -0.44721183, 0.44721183, 1.3416355]
running_mean [0.25, 1.5], running_var [1.0666666, 10.900001]
by hand, channel 0: (1 - 2.5)/sqrt(1.25 + 1e-5) = -1.34164
by hand: running_mean = 0.9*0 + 0.1*2.5 = 0.250, running_var = 0.9*1 + 0.1*1.6667 = 1.06667
eval output, channel 0: [0.72618103, 1.6944224, 2.6626637, 3.6309052]
by hand: (1 - 0.25)/sqrt(1.06667 + 1e-5) = 0.72618
wrong channel count: shape mismatch in reshape, lhs: [2], rhs: [1, 3, 1, 1]
```

```
// X expected: the method `forward` exists for struct `BatchNorm`, but its trait bounds were
// not satisfied
use candle_core::{DType, Device, Module, Tensor};
use candle_nn::{BatchNormConfig, VarBuilder, VarMap, batch_norm};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let bn = batch_norm(2, BatchNormConfig::default(), vb.pp("bn"))?;
    let x = Tensor::zeros((1, 2, 2, 2), DType::F32, &dev)?;
    println!("{}", bn.forward(&x)?);
    Ok(())
}
```

11.3 Why it helps: not the reason the paper gave

The 2015 paper explained BatchNorm by "internal covariate shift": as early layers learn, the distribution of what later layers receive keeps moving, and normalising was supposed to hold it still. Santurkar, Tsipras, Ilyas and Madry tested this in 2018 and found it did not hold up. They injected random noise with non-zero mean and non-unit variance *after* every BatchNorm layer, deliberately destroying the distributional stability, and "the 'noisy' BatchNorm model nearly matches the performance of standard BatchNorm model, despite complete distributional instability". What BatchNorm does instead, they showed, is make "the optimization landscape significantly smoother", which "induces a more predictive and stable behavior of the gradients, allowing for faster training". Concretely, the loss changes less abruptly as the parameters move (a smaller Lipschitz constant) and the gradient at one point is a better predictor of the gradient a step away (β -smoothness). A smoother landscape is why larger learning rates work and why initialisation matters less. Santurkar and colleagues found the same smoothing in the other normalisation variants they tested; that layer normalisation works for the same reason is widely assumed but was not measured in that paper.

11.4 Where to put it, and why the bias before it is dead weight

The paper's order is linear or convolution, then BatchNorm, then the activation, and it remains the common choice; some practitioners report better results with BatchNorm after the activation, and the question is settled empirically per architecture rather than by argument.

Whatever the order, a bias in the layer immediately before BatchNorm does nothing in training mode: the layer adds a constant per channel, and BatchNorm's mean subtraction removes exactly that constant. β plays the bias's role instead. This is why convolutions followed by BatchNorm are created without a bias (conv2d_no_bias in candle, bias=False in PyTorch). The check:

```
use candle_core::{DType, Device, Module, Tensor};
use candle_nn::{BatchNormConfig, Conv2d, Conv2dConfig, VarBuilder, VarMap, batch_norm,
conv2d};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let with_bias = conv2d(2, 3, 3, Conv2dConfig { padding: 1, ..Default::default() },
vb.pp("conv"))?;
    let without = Conv2d::new(with_bias.weight().clone(), None, *with_bias.config()); //
    same weights, no bias
    let bn = batch_norm(3, BatchNormConfig::default(), vb.pp("bn"))?;

    let x = Tensor::randn(0f32, 1.0, (4, 2, 6, 6), &dev)?;
    let a = bn.forward_train(&with_bias.forward(&x)?)?;
    let b = bn.forward_train(&without.forward(&x)?)?;
    let diff = (a - b)?.abs()?.max_all()?.to_scalar::<f32>()?;
    println!("BN(conv + bias) equals BN(conv) in training mode: {}", diff < 1e-5);
    Ok(())
}
```

```
BN(conv + bias) equals BN(conv) in training mode: true
```

Training with batch statistics also injects a little noise (each example is normalised against whatever else landed in its batch), which acts as a mild regulariser; networks with BatchNorm often need less dropout.

11.5 When BatchNorm is the wrong tool

Everything above depends on the batch being a fair sample. It fails when it is not:

- **Small batches.** With one or two images per device the batch statistics are noise. The Weight Standardization paper (Qiao and colleagues) is about exactly this "micro-batch" regime, where "each GPU typically has only 1-2 images for training"; standardising the *weights* of each convolution "to smooth the loss landscape by reducing the Lipschitz constants", combined with a batch-independent normaliser, matches or outperforms BatchNorm trained with large batches.
- **Sequences and transformers.** Variable lengths and padding make batch statistics inconsistent; layer normalisation, which normalises each token over its own features, is standard there.
- **Video, where the batch spans time.** Rivoir, Funke and Speidel showed that BatchNorm's "unique property of depending on other samples in a batch" causes "a 'cheating' effect in anticipation": a model asked to predict the next surgical phase can read the future through the batch statistics. With BN-free backbones, "even simple CNN-LSTMs beat the state of the art on three surgical workflow benchmarks".
- **Federated learning.** Each client's data has its own distribution; Guerraoui and colleagues identify "external covariate shifts and inconsistent statistics across clients" and propose a federated BatchNorm that keeps the statistics consistent with centralised training.

- **Several GPUs.** Plain BatchNorm computes statistics per device; if the per-device batch is small the same noise problem appears even though the global batch is large. Synchronised BatchNorm averages across devices at a communication cost.

The alternatives differ only in *which numbers are averaged together*:

Normaliser	Averages over	Depends on the batch?	Typical home
Batch norm	all images and pixels, per channel	yes	CNNs with large batches
Layer norm	all features of one token or sample	no	transformers, RNNs
Group norm	groups of channels of one sample	no	CNNs with small batches
Instance norm	the pixels of one channel of one sample	no	style transfer
Weight standardisation	the weights of a convolution, not activations	no	micro-batch training

candle provides `layer_norm`, `rms_norm` and `group_norm` in `candle-nn` alongside `batch_norm`.

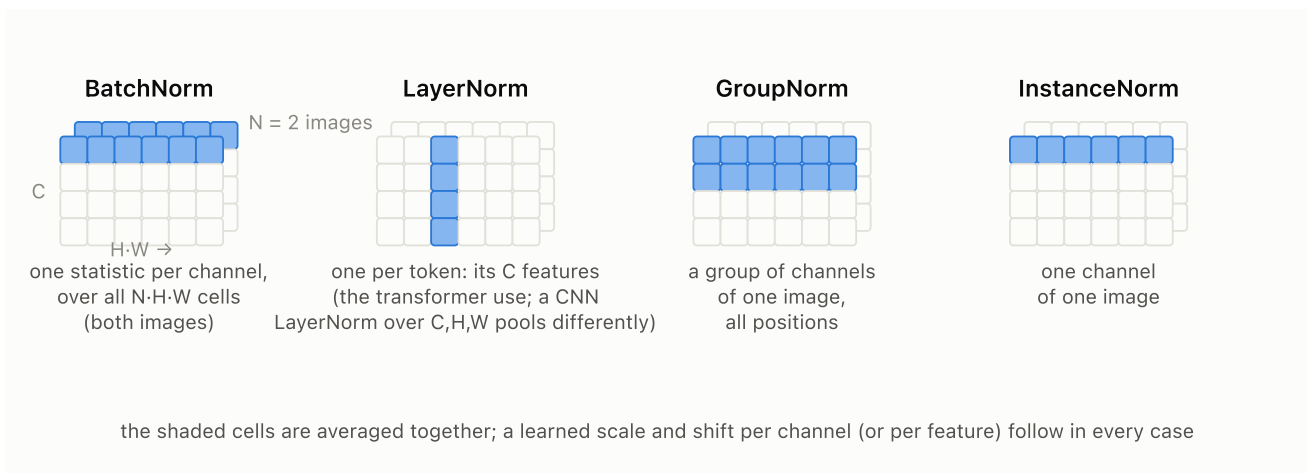


Figure 15. The normalisers differ only in which cells share a mean and variance: BatchNorm averages one channel over every photo and pixel in the batch, LayerNorm averages the features of one token, GroupNorm a group of channels of one photo, and InstanceNorm one channel of one photo.

11.6 Conventions differ between frameworks

	PyTorch	Keras	candle
running update	$\text{new} = (1 - m) \cdot \text{old} + m \cdot \text{batch}, m = 0.1$	$\text{new} = m \cdot \text{old} + (1 - m) \cdot \text{batch}, m = 0.99$	PyTorch's, $m = 0.1$
eps	1e-5	1e-3	1e-5
running variance	unbiased	not stated in the Keras docs	unbiased

The conventions are complementary: PyTorch's m is Keras's $1 - m$. The default values are not equivalent: PyTorch's 0.1 corresponds to Keras 0.9, and Keras's 0.99 corresponds to PyTorch 0.01, an averaging window about ten times longer. Porting weights between frameworks is safe; porting the *momentum number* is not.

12. Fusing a convolution with its BatchNorm

Absorbs `00_conv2d-batchnorm-fusion-guide.md`.

12.1 The algebra

At inference BatchNorm is a fixed affine map per channel, and a convolution is linear, so the two can be folded into one convolution with new weights and a new bias. With $\alpha = \gamma / \sqrt{(\sigma^2 + \epsilon)}$ per output channel:

$$\begin{aligned} \text{BN}(W * x + b) &= \gamma \cdot (W * x + b - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta \\ &= (\alpha \cdot W) * x + (\alpha \cdot (b - \mu) + \beta) \end{aligned}$$

$$\begin{aligned} W' &= \alpha \cdot W & (\alpha \text{ scales every weight of output channel } c \text{ by } \alpha_c) \\ b' &= \alpha \cdot (b - \mu) + \beta \end{aligned}$$

The fused layer computes exactly the same function, one pass over the activations instead of two. A convolution created without a bias gets one from the fold, because $\beta - \alpha \cdot \mu$ is rarely zero.

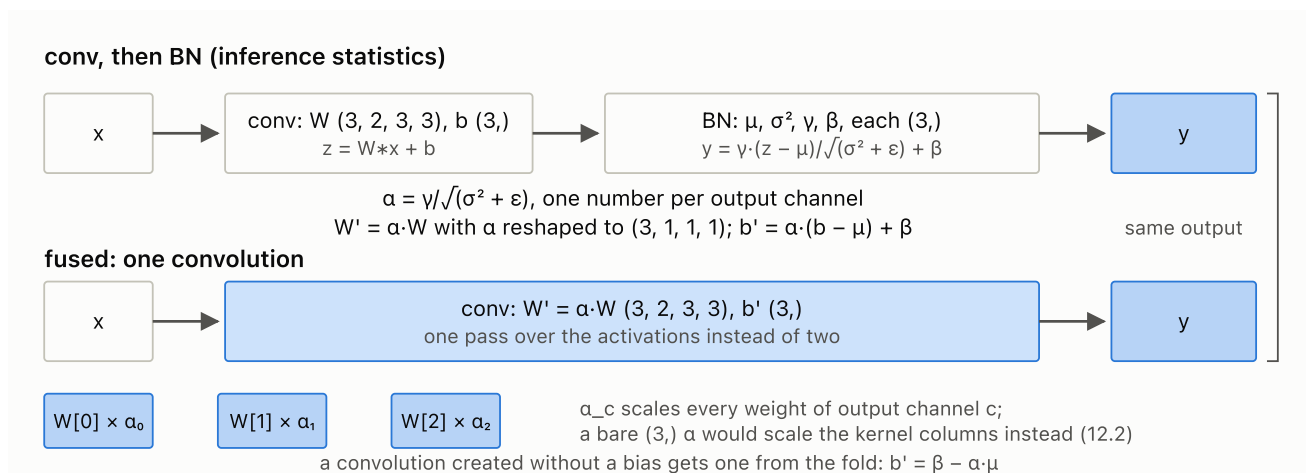


Figure 16. At inference BatchNorm is a fixed scale $\alpha = \gamma / \sqrt{(\sigma^2 + \epsilon)}$ and shift per channel, so it folds into the convolution in front of it: every weight of output channel c is multiplied by α_c (α reshaped to $(3, 1, 1, 1)$) and the bias becomes $\alpha \cdot (b - \mu) + \beta$, one pass over the counter photo instead of two.

12.2 In candle: absorb_bn, and the broadcasting trap

`Conv2d::absorb_bn(&bn)` performs the fold. Its source reshapes α to $(C_{\text{out}}, 1, 1, 1)$ before multiplying the weight, and that reshape is the step the old note's hand-written version forgot. Broadcasting aligns shapes from the right, so a bare $(C_{\text{out}},)$ against a weight of shape $(C_{\text{out}},$

`C_in, 3, 3)` lines α up with the *last* axis (the kernel's columns), and if `C_out` happens to equal `k` the multiplication succeeds and quietly scales the wrong thing. The program shows the correct fold, the `absorb_bn` result, and the silent failure.

```
use candle_core::{DType, Device, Module, ModuleT, Tensor};
use candle_nn::{BatchNormConfig, Conv2d, Conv2dConfig, VarBuilder, VarMap, batch_norm,
conv2d};

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let conv = conv2d(2, 3, 3, Conv2dConfig { padding: 1, ..Default::default() },
vb.pp("conv"))?;
    let bn = batch_norm(3, BatchNormConfig::default(), vb.pp("bn"))?;

    // a few training steps so that the running statistics are not the trivial 0 and 1
    for _ in 0..3 {
        let batch = Tensor::randn(0f32, 1.0, (4, 2, 6, 6), &dev)?;
        let _ = bn.forward_train(&conv.forward(&batch))?;
    }

    let x = Tensor::randn(0f32, 1.0, (2, 2, 6, 6), &dev)?;
    let reference = bn.forward_t(&conv.forward(&x)?, false)?; // conv, then BN in
inference mode

    let fused = conv.absorb_bn(&bn)?;
    let d1 = (fused.forward(&x)? - &reference)?.abs()?.max_all()?.to_scalar::<f32>()?;
    println!("absorb_bn matches conv->bn: {}", fused.bias().is_some());

    let (gamma, beta) = bn.weight_and_bias().unwrap();
    let alpha = (gamma / (bn.running_var() + bn.eps())?.sqrt())?; //
(C_out,)
    let w = conv.weight().broadcast_mul(&alpha.reshape((3, 1, 1, 1)))?; //
scale each output channel
    let b = (beta + alpha.mul(&(conv.bias().unwrap() - bn.running_mean())?))?;
    let manual = Conv2d::new(w, Some(b.clone()), *conv.config());
    let d2 = (manual.forward(&x)? - &reference)?.abs()?.max_all()?.to_scalar::<f32>()?;
    println!("manual fold matches: {}", d2 < 1e-4);

    let wrong_w = conv.weight().broadcast_mul(&alpha)?; // no
reshape: aligns with the kernel columns
    let wrong = Conv2d::new(wrong_w.clone(), Some(b), *conv.config());
    let d3 = (wrong.forward(&x)? - &reference)?.abs()?.max_all()?.to_scalar::<f32>()?;
    println!("without the reshape: shape still {:?}, but wrong: {}", wrong_w.dims(), d3 >
1e-2);
    Ok(())
}
```

```
absorb_bn matches conv->bn: true; fused bias present: true
manual fold matches: true
without the reshape: shape still [3, 2, 3, 3], but wrong: true
```

Two candle idioms in that program are worth naming: `bn.running_var() + bn.eps()` adds a scalar to every element, and `gamma / tensor` divides element-wise because the shapes match. PyTorch ships the same fold as `torch.nn.utils.fusion.fuse_conv_bn_eval(conv, bn)`, which likewise requires both modules to be in evaluation mode.

12.3 When not to fuse, and what to expect

- **Never during training.** The batch statistics change every step and γ , β , μ , σ^2 must remain separate parameters. Fuse a finished model, once, for deployment.
- **Precision.** The fold divides by $\sqrt{(\sigma^2 + \epsilon)}$. A channel whose running variance is tiny produces a large α , and in f16 that can overflow or lose precision; check the fused model against the original on real inputs, as the program above does, before shipping it.
- **What it buys.** One fewer read and write of the whole activation tensor per layer, one fewer kernel launch, and a smaller model file. How much wall-clock time that saves depends on whether the layer was memory-bound; the old note's speed-up tables had no source and are not repeated. Measure.

13. Squeeze-and-Excitation

Absorbs `00_Squeeze-and-Excitation(SE).md` and `00_Squeeze-and-Excitation(SE)2.md`.

13.1 The idea

After a few convolutions, the counter photo is a stack of 64 channels: one responds to horizontal edges, one to shiny curved surfaces, one to steam-like texture, and so on. Which channels matter depends on the photo. A Squeeze-and-Excitation block (Hu, Shen, Albanie, Sun and Wu) lets the network turn each channel up or down *per image*: it summarises each channel into one number (squeeze), passes those numbers through a tiny two-layer network that outputs one weight between 0 and 1 per channel (excite), and multiplies each channel by its weight (scale). It is attention over channels. It shares nothing with the query-key-value attention of section 14 except the word: there are no tokens and no pairwise similarities, only a learned gate per channel.

```

X (B, C, H, W)
|  squeeze: mean over H and W          s = (B, C)
▼
|  excite:  z = sigmoid( W2 · relu( W1 · s ) )   W1: (C/r, C)   W2: (C, C/r)   z in (0,1)
▼
|  scale:  Y[:, c, :, :] = z[:, c] · X[:, c, :, :]
▼
Y (B, C, H, W)

```

The reduction ratio r (16 in the paper) keeps the excitation network small: $2C^2/r$ weights plus $C/r + C$ biases. SENet won the 2017 ImageNet Large Scale Visual Recognition Challenge (ILSVRC) classification task and "reduced the top-5 error to 2.251%", at what the paper calls "slight additional computational cost".

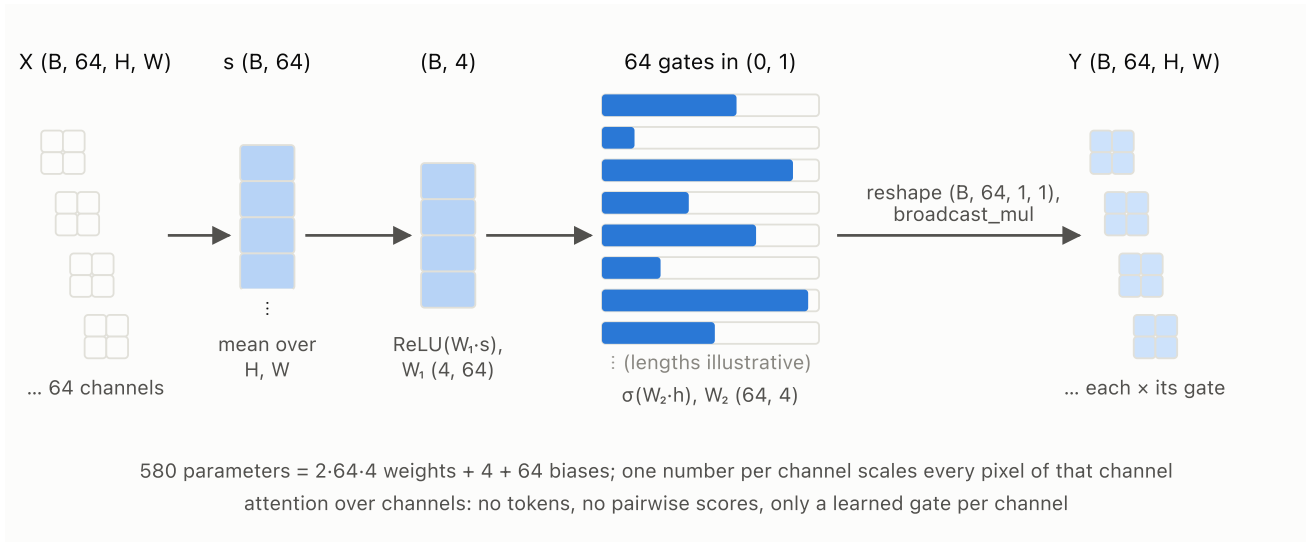


Figure 17. Squeeze-and-Excitation averages each of the 64 channels of the counter photo to one number, turns those 64 numbers into 64 gates between 0 and 1 through a $64 \rightarrow 4 \rightarrow 64$ network (580 parameters), and multiplies every pixel of a channel by its gate: attention over channels, with no pixel moved.

13.2 The block in candle, corrected

Both old notes contained the same four bugs, each a compile or run-time error on every candle version: `VarBuilder::from_varmap(VarMap::new(), ..)` needs a reference; `Tensor` has no `sigmoid` method (the function is `candle_nn::ops::sigmoid`); `xs * z` fails because `*` does not broadcast (section 3.3); and `z.i(0)` needs use `candle_core::IndexOp`. This version runs.

```

use candle_core::{DType, Device, IndexOp, Module, Tensor};
use candle_nn::{Linear, VarBuilder, VarMap, linear, ops};

/// Squeeze-and-Excitation for (B, C, H, W) inputs.
pub struct SeBlock {
    fc1: Linear,
    fc2: Linear,
    channels: usize,
}

impl SeBlock {
    pub fn new(channels: usize, reduction: usize, vb: VarBuilder) ->
        candle_core::Result<Self> {
        if reduction == 0 || channels % reduction != 0 {
            candle_core::bail!("channels {channels} must be divisible by reduction
                                {reduction}");
        }
        let mid = channels / reduction;
        Ok(Self { fc1: linear(channels, mid, vb.pp("fc1"))?, fc2: linear(mid, channels,
                                vb.pp("fc2"))?, channels })
    }

    /// The per-channel gate, shape (B, C), every entry in (0, 1).
    pub fn gate(&self, xs: &Tensor) -> candle_core::Result<Tensor> {
        let (b, c, _h, _w) = xs.dims4()?;
        if c != self.channels {
            candle_core::bail!("expected {} channels, got {c}", self.channels);
        }
        let s = xs.mean_keepdim((2, 3)).reshape((b, c))?; // squeeze
        let z = self.fc1.forward(&s)?.relu()?;             // excite
        ops::sigmoid(&self.fc2.forward(&z)?)
    }
}

impl Module for SeBlock {
    fn forward(&self, xs: &Tensor) -> candle_core::Result<Tensor> {
        let (b, c, _h, _w) = xs.dims4()?;
        let z = self.gate(xs)?.reshape((b, c, 1, 1))?;
        xs.broadcast_mul(&z) // scale
    }
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let se = SeBlock::new(64, 16, vb.pp("se"))?;

    let xs = Tensor::randn(0f32, 1.0, (2, 64, 32, 32), &dev)?; // 2 counter photos,
        64 channels
    let ys = se.forward(&xs)?;
    let z = se.gate(&xs)?;
    let flat = z.flatten_all()?.to_vec1:::<f32>()?;
    println!("in {:?} out {:?} gate {:?}; every gate in (0,1): {:?}", xs.dims(), ys.dims(),
        z.dims(), flat.iter().all(|g| *g > 0.0 && *g < 1.0));

    let n: usize = varmap.all_vars().iter().map(|v| v.elem_count()).sum();
    println!("parameters {n} = 2*64*64/16 weights + (4 + 64) biases = {:?}", 2 * 64 * 64 / 16
        + 4 + 64);

    let ratio = (ys.i((0, 5))?. / xs.i((0, 5))?.).flatten_all()?.to_vec1:::<f32>()?; //
        output / input on one channel
}

```

```

let (lo, hi) = ratio.iter().fold((f32::INFINITY, f32::NEG_INFINITY), |(lo, hi), r|
    (lo.min(*r), hi.max(*r)));
println!("channel 5 of photo 0 is scaled by one number: {}", (hi - lo).abs() < 1e-5 &&
    (lo - z.i((0, 5)).to_scalar::<f32>()).abs() < 1e-5);

let zz = z.reshape((2, 64, 1, 1))?;
println!("plain multiply: {}", (&xs * &zz).unwrap_err());
println!("wrong channel count: {}", se.forward(&Tensor::zeros((1, 32, 4, 4), DType::F32,
    &dev)).unwrap_err());
println!("bad reduction: {}", SeBlock::new(64, 12, vb.pp("bad")).err().unwrap());
Ok(())
}

```

```

in [2, 64, 32, 32] out [2, 64, 32, 32] gate [2, 64]; every gate in (0,1): true
parameters 580 = 2*64*64/16 weights + (4 + 64) biases = 580
channel 5 of photo 0 is scaled by one number: true
plain multiply: shape mismatch in mul, lhs: [2, 64, 32, 32], rhs: [2, 64, 1, 1]
wrong channel count: expected 64 channels, got 32
bad reduction: channels 64 must be divisible by reduction 12

```

```

// X expected: no method named `sigmoid` found for struct `candle_core::Tensor`
use candle_core::{Device, Tensor};

fn main() -> candle_core::Result<()> {
    let t = Tensor::new(&[0.0f32, 1.0], &Device::Cpu)?;
    println!("{}", t.sigmoid()?);
    Ok(())
}

```

`mean_keepdim((2, 3))` averages over both spatial axes in one call and keeps them as size-1 dimensions, which is why the `reshape((b, c))` follows. `SeBlock::new` returns a candle error rather than panicking on a bad reduction ratio, so a wrong configuration surfaces as a `Result` like everything else.

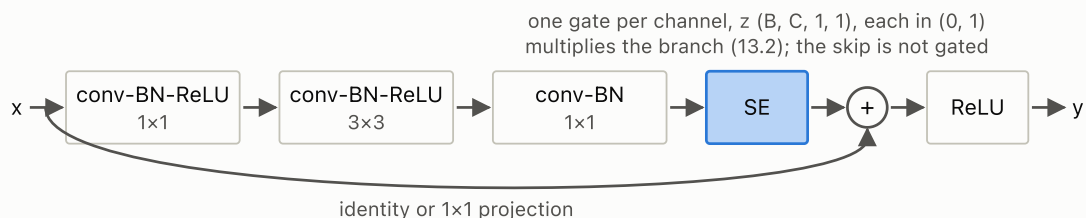
13.3 Where it goes, and its relatives

In a residual block the SE block sits at the end of the residual branch, after the last convolution and its BatchNorm, before the addition with the skip:

```

x → conv-BN-ReLU → conv-BN-ReLU → conv-BN → SE → (+) → ReLU → y
    (1×1)           (3×3)           (1×1)
    |               |               |
    └────────── identity or 1×1 projection ─────────┘

```



SE is the last stage of the branch, after the last conv-BN and before the addition; the ReLU follows the addition, and the skip joins at the addition without passing the gates

Figure 18. In a residual block the counter photo's branch runs conv-BN-ReLU (1×1), conv-BN-ReLU (3×3) and conv-BN (1×1), then the SE block scales those channels with one gate each, and only then is the skip (identity or a 1×1 projection) added and the ReLU applied: the gates act on the branch, not on the skip.

Two later modules generalise the idea. **CBAM** (Woo and colleagues) "sequentially infers attention maps along two separate dimensions, channel and spatial, then the attention maps are multiplied to the input feature map", adding a *where* gate after the *what* gate, with "negligible overheads". **ECA-Net** (Wang and colleagues) keeps only the channel gate but replaces the two fully connected layers with a one-dimensional convolution across neighbouring channels, avoiding the dimensionality reduction; on a ResNet-50 backbone of 24.37M parameters their module adds 80 parameters and still improves top-1 accuracy by more than 2%.

Practical notes that survive: keep C divisible by r ; log the gate statistics during training, because gates stuck at 0 or 1 mean the block has stopped adapting; and remember that SE, CBAM and ECA modulate *existing* channels, so they help most where the channel count is large and the channels are diverse.

Part E: Attention

14. Attention, self-attention and multi-head attention

Absorbs 2_attention.md, 2_self_attention_detailed.md, 21_self_attention_detailed.md, 2_self_attention_detailed2.md, 2_self_attention_detailed3.md and 3_attention_misc_2.md.

14.1 Reading an order slip

An order slip says: *two large lattes, oat milk, no sugar*. To understand the word *lattes* a reader glances at *two* (how many), *large* (what size) and *oat milk* (which variant), and ignores *no sugar* less than you might think, because it also applies to the lattes. Attention gives every word that ability. Each word produces three vectors:

- a **query**: what am I looking for? (*lattes*: "what modifies me?")
- a **key**: what do I offer to others? (*large*: "I am a size")
- a **value**: what do I contribute if someone attends to me? (*large*: the meaning "large")

The score between a query and a key, their dot product, says how relevant that word is to this one. Softmax turns one word's scores over all the keys into weights that sum to 1. The word's new representation is the weighted sum of the values. Every word does this at once, so the whole slip is re-read in one matrix operation, and after a few such layers *lattes* carries the information that there are two of them, large, with oat milk.

The library analogy in the old notes is sound: the query is the question you bring to the desk, the keys are the index cards, the values are the books, and you leave with a blend of the books whose cards matched.

14.2 Scaled dot-product attention, with the numbers

With the queries stacked as rows of Q , keys as rows of K , values as rows of V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$

$Q K^T$ holds every query-key score. The division by $\sqrt{d_k}$ is there because, as the Transformer paper puts it, "for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients". Softmax is taken along each row, over the keys, so each row of weights sums to 1. The program uses three two-dimensional tokens as their

own queries, keys and values, so every number can be checked by hand: the scores are $X X^T$, token 0 and token 2 point in related directions, and token 0 ends up attending equally to itself and to token 2.

```
use candle_core::{D, Device, Tensor};
use candle_nn::ops::softmax;

fn rows(t: &Tensor) -> candle_core::Result<String> {
    Ok(t.to_vec2:::<f32>()?.iter().map(|r| format!("{}", r.iter().map(|v| format!("{}", v:6.3f32)).collect:::<Vec<_>>().join(",")).collect:::<Vec<_>>().join(" ")))
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let x = Tensor::new(&[[1.0f32, 0.0], [0.0, 1.0], [1.0, 1.0]], &dev)?; // 3 tokens, d_k = 2
    let scores = x.matmul(&x.t())?; // Q K^T with Q = K = X
    let scaled = (&scores / (2.0f64).sqrt())?;
    let weights = softmax(&scaled, D::Minus1)?; // softmax over the keys (last dim)
    let out = weights.matmul(&x)?; // weights · V with V = X
    println!("scores {}", rows(&scores)?);
    println!("scaled {}", rows(&scaled)?);
    println!("weights {}", rows(&weights)?);
    println!("row sums {:?}", weights.sum(D::Minus1)?.to_vec1:::<f32>()?);
    println!("output {}", rows(&out)?);
    Ok(())
}
```

```
scores [ 1.000, 0.000, 1.000] [ 0.000, 1.000, 1.000] [ 1.000, 1.000, 2.000]
scaled [ 0.707, 0.000, 0.707] [ 0.000, 0.707, 0.707] [ 0.707, 0.707, 1.414]
weights [ 0.401, 0.198, 0.401] [ 0.198, 0.401, 0.401] [ 0.248, 0.248, 0.503]
row sums [1.0, 1.0, 1.0]
output [ 0.802, 0.599] [ 0.599, 0.802] [ 0.752, 0.752]
```

Row 0 of the weights: token 0 scores 1 against itself, 0 against token 1 and 1 against token 2, so after scaling and softmax it gives about 0.40 to itself, 0.40 to token 2 and 0.20 to token 1. Its output is $0.40 \cdot [1, 0] + 0.20 \cdot [0, 1] + 0.40 \cdot [1, 1] = [0.80, 0.60]$: the token has pulled in some of token 2's second coordinate.

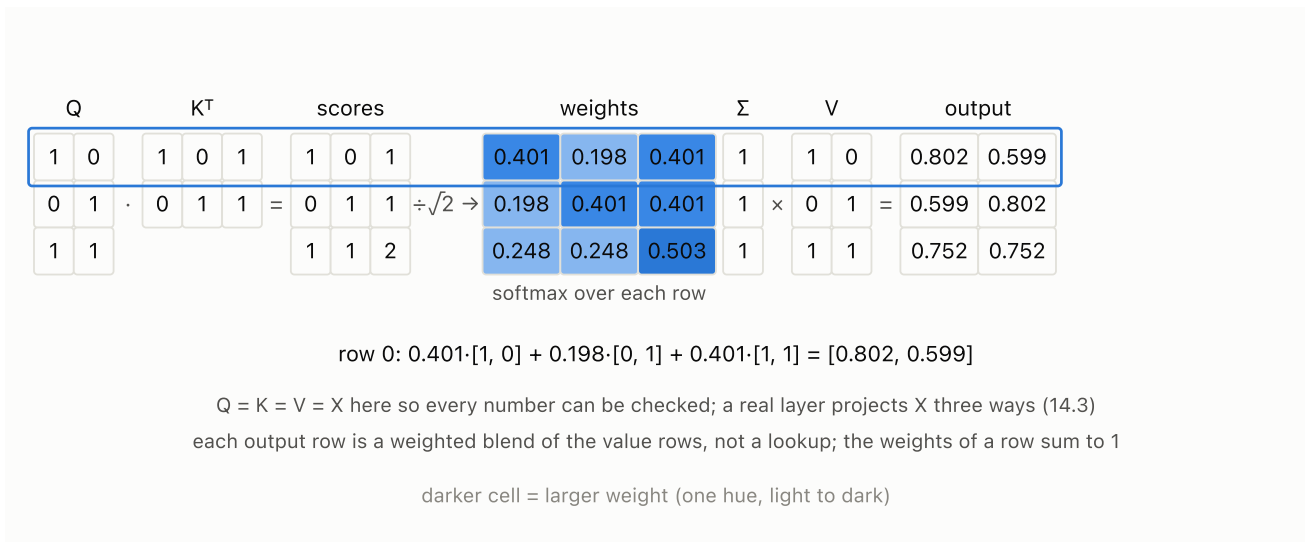


Figure 19. Scaled dot-product attention on three order-slip tokens used as their own queries, keys and values: the scores $X \cdot X^T$ are divided by $\sqrt{2}$, softmax along each row turns them into weights that sum to 1, and token 0's output $0.401 \cdot [1, 0] + 0.198 \cdot [0, 1] + 0.401 \cdot [1, 1] = [0.802, 0.599]$ is a blend of the value rows.

14.3 Self-attention and cross-attention

In *self-attention* the queries, keys and values are all computed from the same sequence: $Q = X W_Q$, $K = X W_K$, $V = X W_V$, with three different learned matrices. The old note's shorthand " $Q = K = V$ " is wrong as written; what is shared is the *source* X , not the three projections, and the whole point of the separate matrices is that a word can ask for one thing, advertise another and contribute a third. In *cross-attention*, used in encoder-decoder models, the queries come from one sequence (the decoder's) and the keys and values from another (the encoder's output).

	self-attention	cross-attention
queries from	the sequence itself	the decoder's sequence
keys and values from	the sequence itself	the encoder's output
used in	encoders; decoders (with a causal mask)	the decoder's second attention block
mask	none in an encoder; causal in a decoder	usually none (padding only)

14.4 Multi-head attention

One attention pattern per layer is limiting: the pattern that links *lattes* to *two* is not the one that links it to *oat milk*. Multi-head attention runs h attention operations side by side, each in a subspace of size $d_k = d_{\text{model}} / h$, so that, in the paper's words, the model can "jointly attend to information from different representation subspaces at different positions". The heads' outputs are concatenated and mixed by one more linear layer W_O .

In code, nobody keeps h separate small projections. One Linear maps $d_{\text{model}} \rightarrow d_{\text{model}}$, and the result is *reshaped* to $(\text{batch}, \text{seq}, h, d_k)$ and *transposed* to $(\text{batch}, h, \text{seq}, d_k)$: head i is simply columns $i \cdot d_k \dots (i+1) \cdot d_k$ of the projected vector, and the transpose puts the head axis in front so that one batched `matmul` computes all heads at once. The total arithmetic equals that of a single head at full width; multi-head attention is not "free diversity", it is the same budget split h ways.

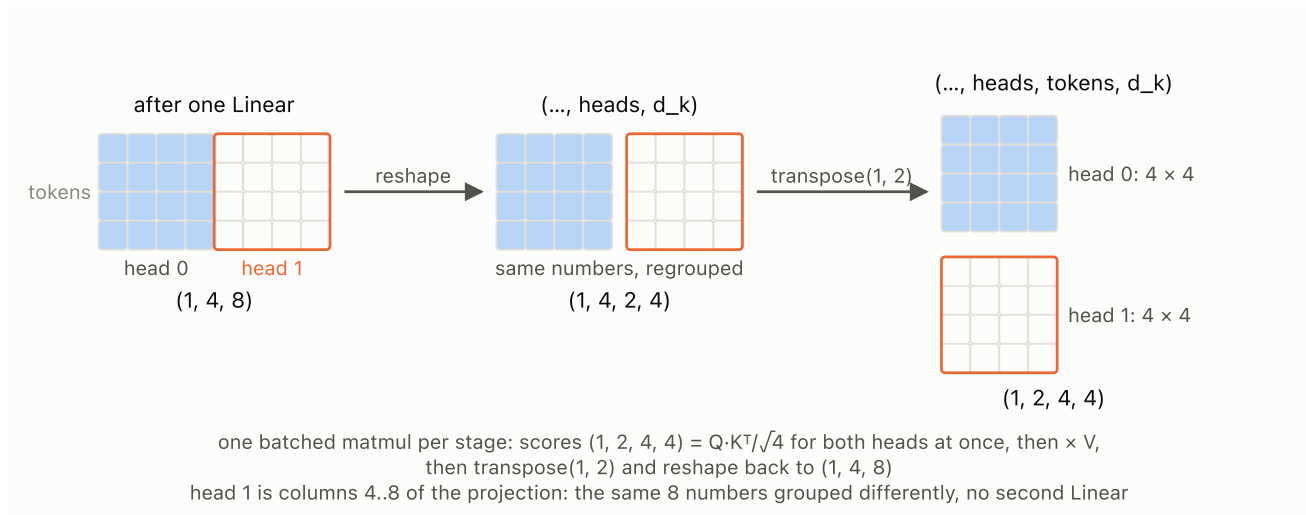


Figure 20. One Linear maps each of the slip's 4 words from 8 numbers to 8, reshape splits those 8 into 2 heads of 4 (head 1 is columns 4..8), and transpose(1, 2) puts the head axis first, $(1, 4, 8) \rightarrow (1, 4, 2, 4) \rightarrow (1, 2, 4, 4)$, so one batched matmul computes both heads' scores at once.

model	d_model	heads	d_k	layers
Transformer base (2017)	512	8	64	6 + 6
BERT base	768	12	64	12
BERT large	1024	16	64	24
GPT-2 small	768	12	64	12
GPT-2 medium	1024	16	64	24
GPT-2 large	1280	20	64	36
GPT-2 XL	1600	25	64	48
GPT-3 175B	12288	96	128	96

(The old note listed a GPT-2 with 1024 dimensions, 12 heads and $d_k = 85$; no GPT-2 has that shape.)

What do the heads learn? Voita and colleagues (ACL 2019) found that in a trained translation model "the most important and confident heads play consistent and often linguistically-interpretable roles" (attending to an adjacent token, the previous or the next; to a token in a particular syntactic relation; to the rarest words in the sentence), and that most of the others can go: "pruning 38 out of 48 encoder heads results in a drop of only 0.15 BLEU" (BLEU is the usual machine-translation score; higher is better). Michel, Levy and Neubig (NeurIPS 2019) reported the same for BERT-style models: "a large percentage of attention heads can be removed at test time without significantly impacting performance" and "some layers can even be reduced to a single head". Heads are a training-time convenience more than a fixed requirement of the architecture.

14.5 The implementation

This is the multi-head self-attention from this project's `src/models/transformer.rs`, cleaned up: the unused `k_dim/v_dim` fields are gone, the checks use `bail!`, an optional mask is supported, and a helper exposes the attention weights so they can be inspected. The parameter count is $4 \cdot d^2 + 4 \cdot d$: four square projections with biases.

```

use candle_core::{D, DType, Device, IndexOp, Module, Tensor};
use candle_nn::{Linear, VarBuilder, VarMap, linear_b, ops::softmax};

pub struct Mhsa {
    dim: usize,
    heads: usize,
    q: Linear,
    k: Linear,
    v: Linear,
    o: Linear,
}

impl Mhsa {
    pub fn new(vb: &VarBuilder, dim: usize, heads: usize, bias: bool) ->
        candle_core::Result<Self> {
        if dim == 0 || heads == 0 || dim % heads != 0 {
            candle_core::bail!("dim {dim} must be a positive multiple of heads {heads}");
        }
        OK(Self {
            dim, heads,
            q: linear_b(dim, dim, bias, vb.pp("q_linear"))?,
            k: linear_b(dim, dim, bias, vb.pp("k_linear"))?,
            v: linear_b(dim, dim, bias, vb.pp("v_linear"))?,
            o: linear_b(dim, dim, bias, vb.pp("output_linear"))?,
        })
    }

    /// (batch, seq, dim) -> (batch, heads, seq, d_k)
    fn split_heads(&self, t: &Tensor) -> candle_core::Result<Tensor> {
        let (b, n, _) = t.dims3()?;
        t.reshape((b, n, self.heads, self.dim / self.heads))?.transpose(1, 2)
    }

    /// Attention weights (batch, heads, seq, seq); `mask` is 1 where attention is
    /// forbidden.
    pub fn weights(&self, x: &Tensor, mask: Option<&Tensor>) -> candle_core::Result<Tensor>
    {
        let (_, _, d) = x.dims3()?;
        if d != self.dim {
            candle_core::bail!("input dim {d} != model dim {}", self.dim);
        }
        let q = self.split_heads(&self.q.forward(x)?);
        let k = self.split_heads(&self.k.forward(x)?);
        let d_k = (self.dim / self.heads) as f64;
        let scores = (q.matmul(&k.transpose(2, 3)?.contiguous()?)) / d_k.sqrt()?;
        let scores = match mask {
            None => scores,
            Some(m) => {
                let m = m.broadcast_as(scores.shape())?.to_dtype(DType::U8)?;
                let neg_inf = Tensor::full(f32::NEG_INFINITY, scores.shape(), x.device())?;
                m.where_cond(&neg_inf, &scores)? // forbidden positions become -
                inf
            }
        };
        softmax(&scores, D::Minus1)
    }

    pub fn forward_masked(&self, x: &Tensor, mask: Option<&Tensor>) ->
        candle_core::Result<Tensor> {
        let (b, n, _) = x.dims3()?;
        let w = self.weights(x, mask)?;
        let v = self.split_heads(&self.v.forward(x)?).contiguous()?;

```

```

        let context = w.matmul(&v)? // (batch, heads,
            seq, d_k)
        .transpose(1, 2)? // (batch, seq,
            heads, d_k)
        .reshape((b, n, self.dim))?; // concatenate the
            heads
        self.o.forward(&context)
    }
}

impl Module for Mhsa {
    fn forward(&self, x: &Tensor) -> candle_core::Result<Tensor> {
        self.forward_masked(x, None)
    }
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let attn = Mhsa::new(&vb, 8, 2, true)?; // 8 numbers per
        word, 2 heads
    let slip = Tensor::randn(0f32, 1.0, (1, 4, 8), &dev)?; // 1 slip of 4
        words

    let y = attn.forward(&slip)?;
    let w = attn.weights(&slip, None)?;
    let sums = w.sum(D::Minus1)?.flatten_all()?.to_vec1::<f32>()?;
    println!("in {:?} out {:?} weights {:?}; rows sum to 1: {}", slip.dims(), y.dims(),
        w.dims(), sums.iter().all(|s| (s - 1.0).abs() < 1e-5));
    let n: usize = varmap.all_vars().iter().map(|v| v.elem_count()).sum();
    println!("parameters {n} = 4*8*8 + 4*8 = {}", 4 * 64 + 4 * 8);

    // head 1 is columns 4..8 of the projection
    let q_full = attn.q.forward(&slip)?;
    let q_heads = attn.split_heads(&q_full)?;
    let same = (q_heads.i((0, 1))?.narrow(1, 4,
        4)?.abs()?.max_all()?.to_scalar::<f32>()? == 0.0;
    println!("head 1 == columns 4..8: {same}; split_heads contiguous: {}, after
        contiguous(): {}", q_heads.is_contiguous(), q_heads.contiguous()?.is_contiguous());

    // causal mask: 1 strictly above the diagonal = the future
    let allowed = Tensor::tril2(4, DType::F32, &dev)?; // 1 on and below
        the diagonal
    let causal = ((allowed * -1.0)? + 1.0)?;
    println!("causal mask\n{causal}");
    let wc = attn.weights(&slip, Some(&causal))?;
    let upper_zero = (wc.i((0, 0))?.abs()?.max_all()?.to_scalar::<f32>()? ==
        0.0;
    let sums = wc.sum(D::Minus1)?.flatten_all()?.to_vec1::<f32>()?;
    println!("with the causal mask: future weights are 0: {upper_zero}; rows still sum to 1:
        {}", sums.iter().all(|s| (s - 1.0).abs() < 1e-5));

    // padding mask: the last word is padding, nobody may attend to it
    let pad = Tensor::new(&[[[0u8, 0, 0, 1]]], &dev)?; // (1, 1, 1, 4)
        broadcasts over heads and queries
    let wp = attn.weights(&slip, Some(&pad))?;
    let last_col = wp.i((0, 0))?.t()?.i(3)?.to_vec1::<f32>()?;
    println!("with the padding mask: weights on the padded word {last_col:?}");

    println!("bad config: {}", Mhsa::new(&vb.pp("bad"), 10, 4, true).err().unwrap());
    println!("bad input : {}", attn.forward(&Tensor::zeros((1, 4, 6), DType::F32,
        &dev)?.unwrap_err());

```

```

    Ok(())
}

```

```

in [1, 4, 8] out [1, 4, 8] weights [1, 2, 4, 4]; rows sum to 1: true
parameters 288 = 4*8*8 + 4*8 = 288
head 1 == columns 4..8: true; split_heads contiguous: false, after contiguous(): true
causal mask
[[0., 1., 1., 1.],
 [0., 0., 1., 1.],
 [0., 0., 0., 1.],
 [0., 0., 0., 0.]]
Tensor[[4, 4], f32]
with the causal mask: future weights are 0: true; rows still sum to 1: true
with the padding mask: weights on the padded word [0.0, 0.0, 0.0, 0.0]
bad config: dim 10 must be a positive multiple of heads 4
bad input : input dim 6 != model dim 8

```

Reading the program against the old notes:

- **Masking.** The mask is 1 where attention is *forbidden*; where_cond replaces those scores with $-\infty$, and softmax turns $-\infty$ into exactly 0 (section 6.5). The causal mask comes from `Tensor::tril2`, which builds a lower-triangular matrix of ones. A padding mask of shape (1, 1, 1, seq) broadcasts over batch, heads and queries. A row that is entirely masked yields NaN, so never mask every key of a query.
- **contiguous().** transpose returns a strided view. The `matmul` calls in this program accepted the transposed queries without complaint on both candle versions; the explicit `contiguous()` on the transposed keys and values is the safe habit for backends and custom kernels that require a plain layout, and it costs one copy.
- **Scaling.** The scores are divided by $\sqrt{d_k}$ of one head ($\sqrt{4} = 2$ here), not of `d_model`.
- **Precision.** When the model runs in f16 or bf16, compute the scores and the softmax in f32 (`to_dtype(DType::F32)` before, and back after); the old notes' upcast was about this, and it is correct.
- **Cost.** The score tensor is (batch, heads, seq, seq): memory grows with the square of the sequence length, and time as $\text{seq}^2 \cdot d$. That is why long-context models use fused attention kernels (candle-nn has a CPU flash-attention module, `candle_nn::ops::sdpa` with a fused Metal path, whose argument list on 0.11.0 also takes an optional mask and a causal flag, and a separate CUDA flash-attention crate), windowed attention (section 10.5's Swin), or approximations. None of them change the equation above, only how it is evaluated.

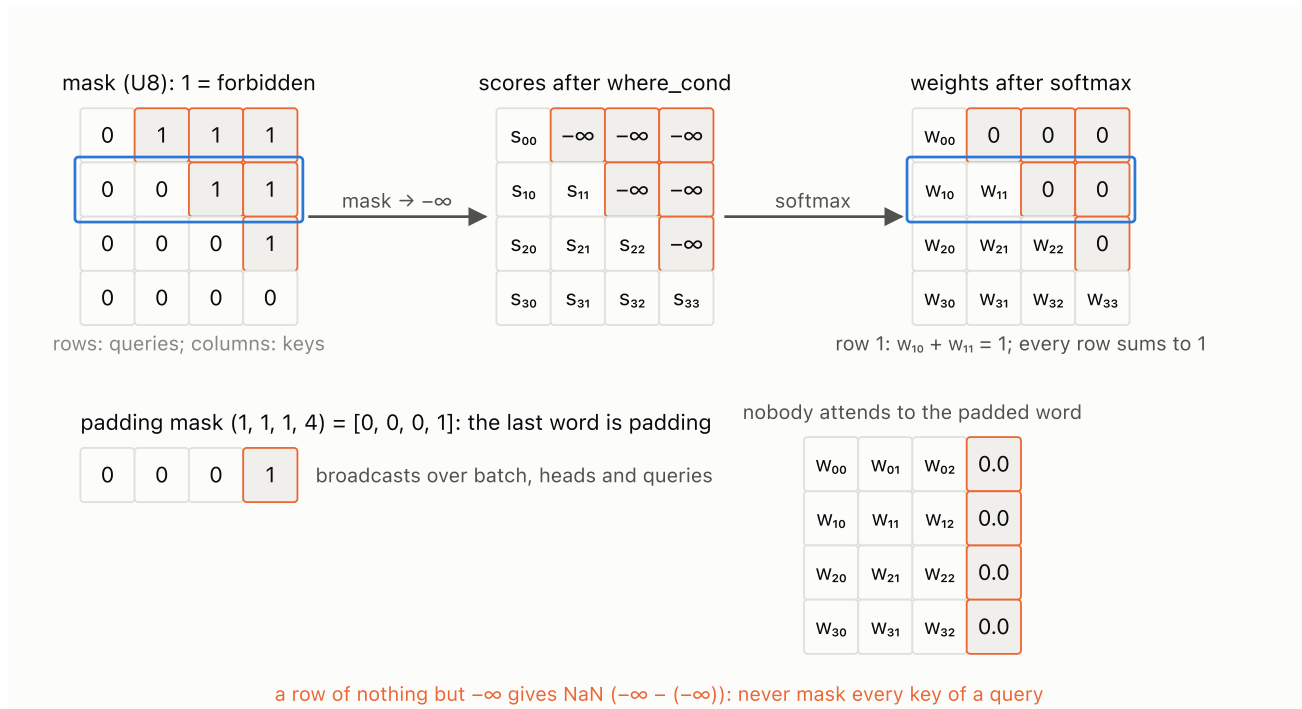


Figure 21. A mask names the words of the slip a query may not read: the causal mask (1 above the diagonal = the future) and the padding mask (the last word) replace those scores with $-\infty$ before softmax, so their weights are exactly 0 while each row of the rest still sums to 1; masking a whole row would give NaN.

14.6 Where attention sits: the Transformer block

Attention on its own is one *sub-layer*. A Transformer block wraps it with three more ideas: a residual connection (add the sub-layer's output to its input, so that depth never makes things worse), layer normalisation (normalise each token over its own features, section 11.5, so that scales stay tame), and a position-wise feed-forward network (two Linear layers with an activation between, applied to every token separately: the MLP of section 5 acting per token). The original paper applies the normalisation *after* each residual addition (post-LN); GPT-2 and most later models apply it *before* each sub-layer (pre-LN). Xiong and colleagues (ICML 2020) showed why the second is easier to train: in a post-LN Transformer "the expected gradients of the parameters near the output layer are large" at initialisation, which is what the learning-rate warm-up stage (starting with a small learning rate and raising it over the first few thousand steps) compensates for, whereas "if the layer normalization is put inside the residual blocks (recently proposed as Pre-LN Transformer), the gradients are well-behaved at initialization" and warm-up can be dropped.

pre-LN block: $x \rightarrow \text{LN} \rightarrow \text{attention} \rightarrow (+) \rightarrow \text{LN} \rightarrow \text{feed-forward} \rightarrow (+) \rightarrow y$

The program builds a pre-LN block with a stand-in for the attention of section 14.5 (plain attention with no projections, enough to show the wiring) and checks that layer normalisation really gives each token mean 0 and variance 1 before the sub-layer sees it.

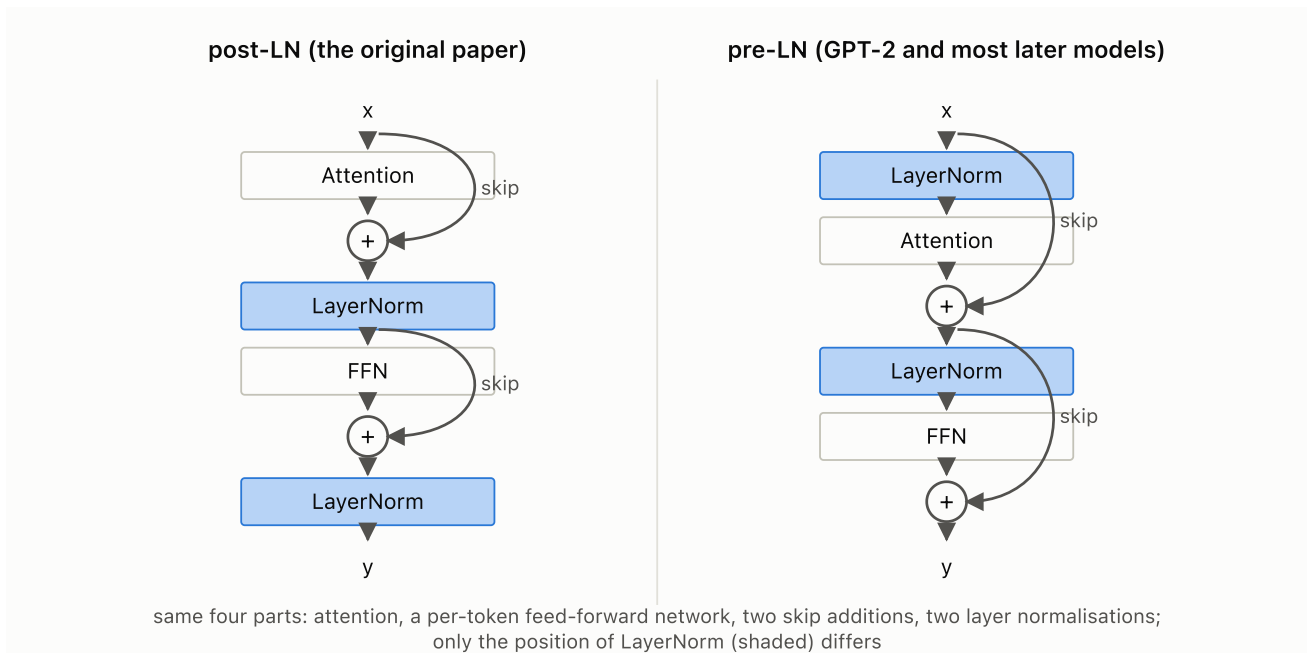


Figure 22. A Transformer block is attention, a per-token feed-forward network, two skip additions and two layer normalisations; the original post-LN block normalises after each addition, while the pre-LN block used by GPT-2 and most later models normalises each sub-layer's input, which is why its gradients behave at initialisation.

```

use candle_core::{D, DType, Device, Module, Tensor};
use candle_nn::{LayerNorm, Linear, VarBuilder, VarMap, layer_norm, linear, ops::softmax};

/// Stand-in for the Mhsa of section 14.5: attention with  $q = k = v = x$ , one head.
fn attention(x: &Tensor) -> candle_core::Result<Tensor> {
    let scores = (x.matmul(&x.transpose(1, 2)?)? / (x.dim(2)? as f64).sqrt())?.
        softmax(&scores, D::Minus1)?.matmul(x)
}

struct TransformerBlock { ln1: LayerNorm, ln2: LayerNorm, ff1: Linear, ff2: Linear }

impl TransformerBlock {
    fn new(vb: VarBuilder, dim: usize, hidden: usize) -> candle_core::Result<Self> {
        Ok(Self {
            ln1: layer_norm(dim, 1e-5, vb.pp("ln1"))?, // size, eps, names
            ln2: layer_norm(dim, 1e-5, vb.pp("ln2"))?,
            ff1: linear(dim, hidden, vb.pp("ff1"))?,
            ff2: linear(hidden, dim, vb.pp("ff2"))?,
        })
    }
}

impl Module for TransformerBlock {
    fn forward(&self, x: &Tensor) -> candle_core::Result<Tensor> {
        let x = (x + attention(&self.ln1.forward(x)?)?)?; // normalise, mix
        // tokens, add back
        let ff = self.ff2.forward(&self.ff1.forward(&self.ln2.forward(&x)?)?.gelu_erf()?)?.
            x + ff // normalise,
        // transform each token, add back
    }
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let varmap = VarMap::new();
    let vb = VarBuilder::from_varmap(&varmap, DType::F32, &dev);
    let block = TransformerBlock::new(vb.pp("block0"), 8, 32)?; // d_model 8, feed-
    // forward width 32

    let x = Tensor::randn(0f32, 1.0, (1, 4, 8), &dev)?;
    let normed = block.ln1.forward(&x)?;
    let means = normed.mean_keepdim(D::Minus1)?.flatten_all()?.to_vec1::<f32>()?;
    let vars = normed.sqr()?.mean_keepdim(D::Minus1)?.flatten_all()?.to_vec1::<f32>()?;
    println!("after layer norm, per token: mean 0: {}, variance 1: {}",
        means.iter().all(|m| m.abs() < 1e-5), vars.iter().all(|v| (v - 1.0).abs() < 1e-3));
    println!("{:?} -> {:?}", x.dims(), block.forward(&x)?.dims());
    let n: usize = varmap.all_vars().iter().map(|v| v.elem_count()).sum();
    println!("parameters: {n} = 2 layer norms (8 + 8 each) + (8*32 + 32) + (32*8 + 8) = {}",
        2 * 16 + 8 * 32 + 32 + 32 * 8 + 8);
    Ok(())
}

```

```

after layer norm, per token: mean 0: true, variance 1: true
[1, 4, 8] -> [1, 4, 8]
parameters: 584 = 2 layer norms (8 + 8 each) + (8*32 + 32) + (32*8 + 8) = 584

```

candle_nn::layer_norm(size, eps, vb) creates weight and bias of length size (the f64 converts into a LayerNormConfig with remove_mean and affine true). Stack a dozen such blocks with the real Mhsa, add the positional encoding of section 15 at the bottom and a Linear classifier at the top, and you have a Transformer encoder.

14.7 From a photo to a sequence

Attention works on any set of tokens. A convolutional feature map (batch, channels, height, width) becomes a sequence of height·width tokens with channels features each by the `flatten_from(2).transpose(1, 2)` transform of section 3.4. That is how vision transformers see an image (with patches as tokens) and how YOLOv10's partial self-attention block feeds a convolutional map into the Mhsa above (section 16).

15. Positional encoding

Absorbs 2_self_attention_detailed4.md.

15.1 Attention does not know the order of the words

Nothing in section 14 used the position of a token. Shuffle the words of the slip and every token's output is the same vector it was before, just shuffled along with it. The program checks this with the plain attention of section 14.2: permuting the rows of X permutes the rows of the output and changes nothing else.

```
use candle_core::{D, Device, Tensor};
use candle_nn::ops::softmax;

fn attention(x: &Tensor) -> candle_core::Result<Tensor> {
    let scores = (x.matmul(&x.t())?)? / (x.dim(1)? as f64).sqrt()?;
    softmax(&scores, D::Minus1)?.matmul(x)
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let x = Tensor::new(&[[1.0f32, 0.0, 2.0], [0.0, 1.0, 0.5], [1.0, 1.0, 0.0], [0.3, 0.7, 0.1]], &dev)?;
    let order = Tensor::new(&[2u32, 0, 3, 1], &dev)?; // a new word order
    let shuffled = x.index_select(&order, 0)?;
    let a = attention(&shuffled)?;
    let b = attention(&x)?.index_select(&order, 0)?;
    let diff = (a - b)?.abs()?.max_all()?.to_scalar::<f32>()?;
    println!("attention(shuffled words) == shuffled attention(words): {}", diff < 1e-6);
    Ok(())
}
```

```
attention(shuffled words) == shuffled attention(words): true
```

"Two large lattes" and "large two lattes" would be the same order. So position must be put into the tokens explicitly.

15.2 Sinusoidal encodings

The Transformer paper adds a fixed vector to each token's embedding, built from sines and cosines of different wavelengths. For position `pos` and dimension pair `i` in a model of width `d_model`:

```
PE(pos, 2i)   = sin( pos / 10000^(2i / d_model) )
PE(pos, 2i+1) = cos( pos / 10000^(2i / d_model) )
```

Low dimensions oscillate fast (they distinguish neighbours), high dimensions slowly (they distinguish regions of the sequence), and the combination is unique per position. The encoding is *added* to the embedding, not concatenated, so the width stays `d_model`. Because it is a formula, it has no parameters and extends to positions longer than any seen in training. The program builds the table for five positions and eight dimensions; row 0 is `[0, 1, 0, 1, ...]` because $\sin 0 = 0$ and $\cos 0 = 1$.

```
use candle_core::{Device, IndexOp, Tensor};

fn sinusoidal(seq: usize, d_model: usize, dev: &Device) -> candle_core::Result<Tensor> {
    let mut pe = vec![0f32; seq * d_model];
    for pos in 0..seq {
        for i in 0..d_model / 2 {
            let angle = pos as f64 / 10000f64.powf(2.0 * i as f64 / d_model as f64);
            pe[pos * d_model + 2 * i] = angle.sin() as f32;
            pe[pos * d_model + 2 * i + 1] = angle.cos() as f32;
        }
    }
    Tensor::from_slice(&pe, (seq, d_model), dev)
}

fn main() -> candle_core::Result<()> {
    let dev = Device::Cpu;
    let pe = sinusoidal(5, 8, &dev)?;
    for pos in 0..5 {
        let row: Vec<String> = pe.i(pos)?.to_vec1::<f32>()?.iter().map(|v| format!("{v:6.3}")).collect();
        println!("pos {pos}: {}", row.join(" "));
    }
    let embeddings = Tensor::zeros((1, 5, 8), candle_core::DType::F32, &dev)?; // (batch, seq, d_model)
    let with_position = embeddings.broadcast_add(&pe)?; // added, not concatenated
    println!("embeddings {:?} + pe {:?} -> {:?}", embeddings.dims(), pe.dims(), with_position.dims());
    Ok(())
}
```

```
pos 0:  0.000  1.000  0.000  1.000  0.000  1.000  0.000  1.000
pos 1:  0.841  0.540  0.100  0.995  0.010  1.000  0.001  1.000
pos 2:  0.909 -0.416  0.199  0.980  0.020  1.000  0.002  1.000
pos 3:  0.141 -0.990  0.296  0.955  0.030  1.000  0.003  1.000
pos 4: -0.757 -0.654  0.389  0.921  0.040  0.999  0.004  1.000
embeddings [1, 5, 8] + pe [5, 8] -> [1, 5, 8]
```

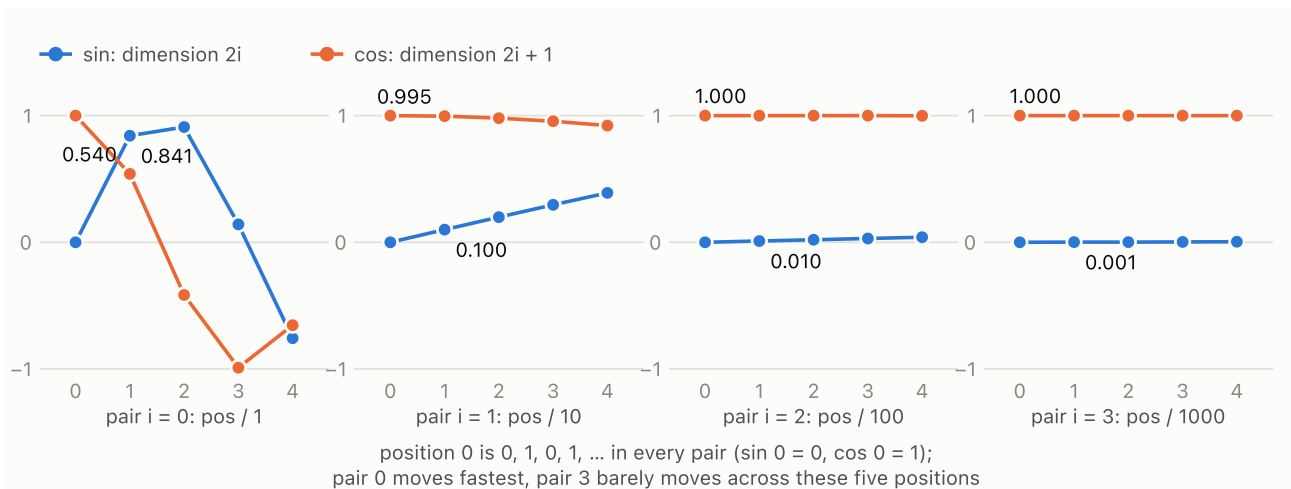


Figure 23. Each pair of dimensions of the slip's positional encoding is a sine and a cosine of $\text{pos} / 10000^{(2i/8)}$: the first pair moves fast (0.841 at position 1), the fourth barely moves (0.001), so every position gets its own pattern and position 0 is 0, 1, 0, 1,

15.3 The other families, briefly

- **Learned absolute positions.** A table of $\text{max_len} \times \text{d_model}$ trainable vectors, one per position, added like the sinusoids. BERT ($\text{max_position_embeddings} = 512$) and GPT-2 ($\text{n_positions} = 1024$) use this. It cannot represent a position beyond the table.
- **Relative and rotary encodings.** Instead of tagging tokens, these change the score: a learned bias depending on the distance $i - j$ is added to $Q \cdot K^T$ (T5), or the queries and keys are rotated by an angle proportional to their position so that their dot product depends only on the relative offset (rotary embeddings, RoPE; candle-nn has a `rotary_emb` module). Both generalise better to lengths not seen in training, which is why most recent language models use them.

Whatever the family, the injection point is the same: before or inside the first attention layer, so that every projection downstream sees position-aware vectors.

16. YOLOv10 notes: where the pieces meet

Absorbs 1_yolov10.md.

The original note was a bookmark: the paper's address (arXiv 2405.14458), the abbreviations SA, MHSA and N_PSA, and a few sentences about the partial self-attention (PSA) module. The sentences check out against the paper, and every component they mention is something this guide has now built.

The PSA module, in the authors' words: "We evenly partition the features across channels into two parts after the 1×1 convolution. We only feed one part into the N_PSA blocks comprised of multi-head self-attention module (MHSA) and feed-forward network (FFN). Two parts are then concatenated and fused by a 1×1 convolution." Inside the attention block they "assign the dimensions of the query and key to half of that of the value in MHSA and replace the LayerNorm with BatchNorm for fast inference", and "PSA is only placed after the Stage 4 with the lowest resolution". N_PSA is set to 1 by default; the paper reports that 2 blocks add 0.2 AP (average precision, the detection score) for 0.1 ms of latency. The B model is obtained from M "by simply increasing the width scale factor".

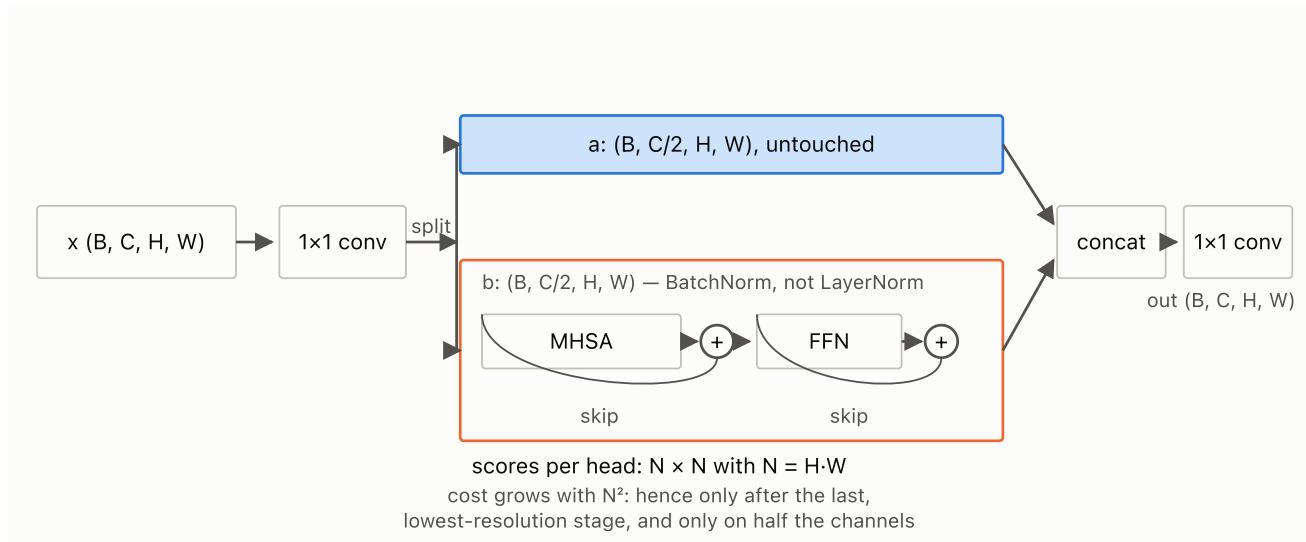


Figure 24. YOLOv10's partial self-attention: a 1×1 convolution splits the counter photo's channels in half, only one half passes through attention and a feed-forward network (each with a skip), and a 1×1 fuses the halves again; it sits only after the lowest-resolution stage because the score matrix has one entry per pair of pixels.

Mapped onto this guide:

in the paper	in this guide
the 1×1 convolutions that split and fuse channels	section 8: channel mixing at every pixel, cheap
MHSA over the feature map	section 14, applied to the flattened map of section 3.4; cost grows with the square of the number of pixels, hence "only after Stage 4", the smallest map, and only on half the channels
BatchNorm instead of LayerNorm	section 11 for the layer, section 12 for why it is attractive at inference: it folds into the neighbouring convolution, LayerNorm does not
query and key at half the value width	d_k and d_v need not be equal; the score cost is $\text{seq}^2 \cdot d_k$, so a smaller d_k cuts it while the values keep their width
the FFN after attention	two Linear layers with an activation between, the MLP of section 5 applied per token
width scale factor	the channel counts of every layer multiplied by one number, which is where the parameter arithmetic of sections 8 and 9 comes in

If you implement PSA in candle, the parts are `conv2d` with `Conv2dConfig::default()` for the 1×1 layers, `batch_norm` with `forward_t`, the `MhSa` of section 14.5 changed so that the query and key projections output heads $\cdot d_k$ with d_k half of d_v (the code as shown fixes $d_k = \text{dim} / \text{heads}$), the flatten and un-flatten transforms (`flatten_from(2).transpose(1, 2)` and its inverse `transpose(1, 2).reshape((b, c, h, w))`), and `Tensor::cat` on the channel axis to rejoin the two halves.

Part F: Appendices

17. Error messages you will actually see

Every message below was produced during the checks for this guide. Compile-time messages come from `rustc`; the others are the `Display` text of a `candle_core::Error`.

Message	Cause	Fix
error[E0284]: type annotations needed on <code>parse()</code> , error[E0283] on <code>collect()</code> or <code>to_vec2()</code>	the type parameter appears only in the return type	turbofish (<code>::<T></code>) or annotate the binding (section 2)
the trait bound <code>i32: WithDType</code> is not satisfied	<code>i32</code> literals or <code>to_vec::<i32></code> on candle 0.9.1	type the first literal (<code>1i64</code> , <code>1u32</code> , <code>1.0f32</code>); 0.11.0 has <code>I32</code> (section 2.3)
unexpected dtype, expected: <code>I64</code> , got: <code>F32</code>	<code>to_vec1::<i64>()</code> on an <code>F32</code> tensor	ask for the tensor's dtype, or <code>to_dtype</code> first (section 2.2)
unexpected rank, expected: <code>1</code> , got: <code>2</code> (<code>[2, 3]</code>)	<code>to_vec1 / dims4 / to_scalar</code> on a tensor of another rank	use the method for the actual rank (section 2.2, 3.4)
shape mismatch in <code>mul</code> , lhs: <code>[2, 64, 32, 32]</code> , rhs: <code>[2, 64, 1, 1]</code>	<code>*</code> , <code>+</code> , <code>-</code> , <code>/</code> need identical shapes	<code>broadcast_mul</code> and friends (section 3.3)
shape mismatch in <code>matmul</code> , lhs: <code>[1, 3]</code> , rhs: <code>[2, 3]</code>	inner dimensions differ	transpose the weight or fix the input width (section 3.2)
dtype mismatch in <code>matmul</code> , lhs: <code>F32</code> , rhs: <code>F64</code>	<code>Var::randn(0.0, 1.0, ..)</code> made an <code>F64</code> tensor	write <code>0f32</code> (section 5.4)
shape mismatch on shared: <code>[3, 3] <=> [2, 2]</code>	the same name requested twice with different shapes	use distinct names or the same shape (section 4.4)
cannot find tensor <code>till.out.weight</code>	the file or backend has no tensor of that name	check prefixes; <code>rename_f</code> for foreign naming (section 4.8)
<code>in_channel</code> mismatch between input (<code>32</code> , groups <code>1</code>) and kernel (<code>64</code>)	input channels do not match the kernel's second dimension times groups	fix the channel count or the weight shape (sections 8, 9)
expected <code>&VarMap</code> , found <code>VarMap``</code>	<code>from_varmap</code> takes a reference	<code>&varmap</code> (section 4.5)
cannot borrow <code>varmap</code> as mutable	<code>load</code> and <code>set_one</code> need <code>&mut self</code>	<code>let mut varmap</code> (section 4.7)
missing field <code>cudnn_fwd_algo</code> in initializer of <code>Conv2dConfig``</code>	struct literal with four fields	<code>..Default::default()</code> (section 8.1)
expected <code>ParamsAdamW</code> , found floating-point number	<code>AdamW::new(vars, 0.001)</code>	<code>AdamW::new_lr(vars, 0.001)</code> (section 5.4)
no method named <code>log_softmax</code> found / no method named <code>sigmoid</code> found	they are functions, not methods	<code>candle_nn::ops::log_softmax(&t, dim)</code> , <code>ops::sigmoid(&t)</code> (sections 6.5, 13.2)

Message	Cause	Fix
the method <code>forward</code> exists for <code>struct BatchNorm</code> , but its trait bounds were not satisfied	<code>BatchNorm</code> implements <code>ModuleT</code> , not <code>Module</code>	<code>forward_t(&x, train)</code> or <code>forward_train</code> (section 11.2)
cannot find function <code>kaiming_normal</code> in <code>module::init`</code>	no such function	<code>init::DEFAULT_KAIMING_NORMAL</code> or <code>Init::Kaiming { .. }</code> (section 4.3)
shape mismatch in <code>gather</code> , lhs: <code>[2, 3]</code> , rhs: <code>[2]</code>	<code>gather</code> wants an index tensor of the same rank	<code>targets.unsqueeze(1)</code> , or use <code>loss::cross_entropy</code> (section 6.5)
<code>cross_entropy</code> expects an input tensor of rank 2 / the target tensor should have a single dimension	wrong shapes for the loss	<code>logits (items, classes), targets (items,)</code> (section 6.5)
the candle crate has not been built with metal support	<code>Device::new_metal</code> without the metal feature	enable the feature in <code>Cargo.toml</code> (section 3.5)
<code>dyn std::error::Error</code> cannot be sent between threads safely	<code>Box<dyn Error></code> moved into a thread	<code>Box<dyn Error + Send + Sync></code> or <code>anyhow::Error</code> (section 1.4)
NaN from softmax	a fully masked row, or $+\infty$ in the input	never mask every key; check inputs (sections 6.4, 14.5)

18. How this guide was verified

Toolchain. `rustc` 1.98.1 (48a229cea 2026-09-01) and `cargo` 1.98.1 on macOS, aarch64, edition 2024. `candle-core` and `candle-nn` at two versions: 0.9.1, the git commit `f5838914f788d3950d0a25042cffe199d9325a9e` of 22 September 2025 that this project's `Cargo.lock` pins; and 0.11.0, the crates.io release of 26 June 2026 (still the newest on 27 September 2026; 0.10.0 to 0.10.2 appeared between the end of March and the start of April 2026).

Method. A script extracted every ```rust block of this file. Blocks that mention neither candle nor tracing were compiled with `rustc --edition 2024 -O` and run. The others were written into `src/bin/` of two scratch crates (both depend on tracing), one per candle version, built with `cargo build` and run. A ```text block directly under a program had to equal the program's standard output line for line. Blocks beginning `// ✗ expected:` had to fail to compile with the quoted text in the compiler's output; the one block marked `// ✓ on: candle 0.11.0` had to fail on 0.9.1 and run on 0.11.0. Blocks beginning `// ~` were run but their output was not compared. Counts on 19 September 2026: 60 ```rust blocks; 108 compile-and-run checks (every candle and tracing block on both versions, every other block once); 0 mismatches. The same run on 27 September 2026, from freshly built scratch crates, gave the same counts: 60 blocks, 108 checks, 0 mismatches.

Differences between the two candle versions that this guide touches. 0.11.0 adds the dtypes `I16`, `I32`, `F6E2M3`, `F6E3M2`, `F4` and `F8E8M0` (only `i16` and `i32` gain `WithDType`, so `i32` literals work there and not on 0.9.1), and `VarBuilder` gains `get_unchecked`, `get_unchecked_dtype`, `set_device` and `set_dtype`. Every other API and every number in this guide behaved identically on both. Runs that start from random weights (the training loop of section 5.4 and the gate values of section 13) differ between runs and versions because the CPU random generator cannot be seeded (`Device::set_seed` returns an error on the CPU).

What was checked against which authority. candle behaviour: the two crates above, plus the source files `candle-nn/src/{var_builder,var_map,init,linear,conv,batch_norm,optim,loss,ops}.rs` and `candle-core/src/{dtype,variable,tensor,error}.rs` at the pinned commit. Rust behaviour: the programs themselves and the anyhow documentation. Machine-learning claims: the original papers, read on 19 September 2026 (Lin, Chen and Yan 2013 for Network in Network; Krizhevsky, Sutskever and Hinton 2012; Szegedy et al. 2014; He et al. 2015; Howard et al. 2017; Sandler et al. 2018; Zhang et al. 2017 for ShuffleNet; Yu and Koltun 2015; van den Oord et al. 2016; Wang et al. 2018 for hybrid dilated convolution; Chen et al. 2017 for DeepLabv3; Dumoulin and Visin 2016; Odena, Dumoulin and Olah 2016; Ioffe and Szegedy 2015; Santurkar et al. 2018; Rivoir, Funke and Speidel 2022; Guerraoui et al. 2024; Qiao et al. 2019; Hu et al. 2017; Woo et al. 2018; Wang et al. 2020 for ECA; Vaswani et al. 2017; Devlin et al. 2018; Brown et al. 2020; Liu et al. 2021 for Swin; Wang et al. 2024 for YOLOv10), the PyTorch documentation for `ConvTranspose2d`, `BatchNorm2d` and `fuse_conv_bn_eval`, the Keras documentation for `BatchNormalization`, and the published GPT-2 configuration files.

Dropped from the old notes as unverifiable. Throughput percentages per dilation rate; frame rates, mIoU and sensitivity figures per use case; "a bottleneck ratio of 2 is optimal"; fusion speed-up and memory tables; "learning rates 10 to 100 times larger"; a "Hybrid Batch Normalization (2025)" reference; the AlexNet GPU memory of 1.5 GB; the GPT-2 row with 85-dimensional heads; the 2-D receptive-field formula and the mixed-stride table; the softmax expected values 0.93587624 and 0.04697593.

Independent review. An independent Fable 5.1 reviewer audited the complete draft on 19 September 2026: it re-ran the block checker (same result), re-read the candle source at both versions and re-fetched about thirty primary sources, and returned 1 blocker, 7 major, 17 minor and 6 optional findings. All were applied: the `vb.get` default (zeros, not Kaiming), scalars on the left of an operator, detach sharing storage, the BERT position count, the Keras/PyTorch momentum correspondence, the NIN 15.99% figure, the dilation-in-classifiers rule, the 0.11.0 dtype list, the missing material on Transformer blocks, head pruning, binary cross-entropy and tracing spans, and the wording items in its report. A focused re-audit by the same reviewer of every changed passage found all corrections applied and no new error, and its own checker run matched; its remaining notes (stale block counts in this section, four one-sentence rewordings and glosses, the order of the term index) were applied last.

Re-verification of 27 September 2026. Eight days after the first edition, every block was compiled and run again on `rustc 1.98.1 (48a229cea 2026-09-01)` with the same two candle versions, and `crates.io` still listed 0.11.0 as the newest release. Every quoted sentence was re-read in its primary source (the papers' arXiv text or PDF, the PyTorch, Keras and anyhow documentation, the Hugging Face model configuration files, the `distill.pub` article), every candle API statement was re-read in the source at both versions, and the project files this guide names (`main.rs`, `mlp.rs`, `transformer.rs`) were re-read. Nothing in the text was found wrong. Two small changes were made: the `// ✓` on: marker of section 2.3 now names the candle version instead of the scratch crate it was checked in, and the convention is stated in section 0; and section 14.5 notes that `sdpa` on 0.11.0 also takes a mask and a causal flag. An independent Fable 5.1 reviewer then re-ran the block checker on both candle versions (60 blocks, 108 checks, 0 mismatches), recomputed every worked number, re-fetched about forty-five primary sources and re-read the candle source at both versions, and found no wrong statement, number or code block and no quotation wrong in substance; its four wording notes were

applied (the exact spacing of the Linear documentation quote in section 3.2, the 1x1 spelling inside the Network in Network quotation in section 8.3, the old note's "Runtime Error" wording in section 2.3, and the 'static bound in section 1.1).

To re-run any block. Create a crate with the Cargo.toml of section 0 (or this project's, for 0.9.1), paste the block into src/main.rs, and cargo run. The std-only blocks need only rustc --edition 2024 file.rs.

Figures. The 24 figures in this guide are drawn by ml/figures/make_figures.rs, a dependency-free Rust program compiled with rustc --edition 2024 and run with the ml/figures folder as its argument; running it again rewrites every .svg byte for byte. It computes every number it draws (the softmax values, the attention weights and outputs, the positional encodings, the receptive fields, the dilation offsets, the transposed-convolution stamp counts, the convolution output sizes and window positions, the BatchNorm fold and the backward example) from the same inputs as the programs above instead of copying them. Each file was checked to be well-formed XML with explicit colours and its own light background (so a dark theme leaves it readable), rendered at full width and inspected, and every number shown in it was compared with the ```text block or sentence beside it; the eight densest figures were also inspected at 380 pixels wide and two on a dark page. The figures are illustrations: no code block changed, so the block counts and checks of the paragraphs above are unchanged. An independent Fable 5.1 audit of the drafts, the generator and the 41 files then present (28 September 2026) re-derived every drawn number from this guide and the handbook, reproduced every file byte for byte from the source, and found one wiring error (the post-LN skip in the Transformer-block figure, now Figure 22) and twenty smaller items: caption wording, notation, two label collisions and a clipped canvas, result numbers typed as literals in the generator, and this note's own description of the rendering checks; all were corrected and the changed files re-audited. Two figures were added later the same day by the same program, the output_padding ambiguity of section 7.5 (Figure 9) and the SE placement of section 13.3 (Figure 18), bringing the folder to 45 files; each was re-derived from this text, rendered at full width, at 380 pixels wide and on a dark page, and audited separately by a fresh Fable 5.1 pass, which found no error; its wording and generator suggestions were applied and the changed files re-audited.

19. Where each old note went

Old file	Section(s)
00_rust-result-error-guide.md	1
00_turbo_fish.md	2, 3.1
01_structureOfMatrices.md	3.2
2_attention_misc.md	3.4 (tensor transform), 4.8 (loaders)
00_VarBuilder.md, 00_VarBuilder1.md	4
04_mlp_multi_layer_perceptron.md, ..3.md (parameter essay)	4.1, 4.9, 4.10
04_mlp_multi_layer_perceptron2.md (MLP guide)	5, 6.5, 11.5
01_dotProductAndGradientDescent.md	5.1, 5.5
01_safe_softmax.md	6
00_ConvTranspose.md, 00_ConvTranspose2.md, 00_ConvTranspose3.md	7

Old file	Section(s)
00_1×1_Convolutions.md	8
00_Convolutions_grouped.md, 00_Convolutions_grouped1.md	9
00_depthWiseConv.md, 00_depthWiseConv1.md	9.1, 9.2
00_dilation.md, 00_dilation_enhanced.md	10
denseAndSptial.md	10.1, 10.5
00_Batch Normalization2.md	11
00_why_Batch Normalization_works.md	11.3
00_conv2d-batchnorm-fusion-guide.md	12
00_Squeeze-and-Excitation(SE).md, 00_Squeeze-and-Excitation(SE)2.md	13
2_attention.md	14.1, 14.3
2_self_attention_detailed.md, 21_self_attention_detailed.md	14.5, 3.6
2_self_attention_detailed2.md, 2_self_attention_detailed3.md	14.1 to 14.4, 14.6
3_attention_misc_2.md	14.5, 3.6
2_self_attention_detailed4.md	15
1_yolov10.md	16

The old files are untouched by this consolidation; deleting them is a separate decision. Three of them (00_Squeeze-and-Excitation(SE)2.md, 00_VarBuilder1.md, 2_self_attention_detailed4.md) already carried an uncommitted one-line change (a removed logo line) when this work started, and a separate consolidation of the same notes, RUST_AND_MACHINE_LEARNING_HANDBOOK.md, was being written in this folder by another session on the same day; the two documents were produced independently.

20. Term index

Term	Sections
1×1 convolution	8
? operator	1.2
absorb_bn	12.2
activation (ReLU, ReLU6)	5.2, 8.3
AdamW, ParamsAdamW	5.4
AlexNet (two GPUs)	9.3
anyhow	1.1, 1.3, 1.5
ASPP (atrous spatial pyramid pooling)	10.4
attention, scaled dot-product	14.2
backward, GradStore	4.6, 5.4, 5.5, 7.4
bail!	1.3
batch normalisation	11

Term	Sections
BatchNorm momentum conventions	11.6
bias before BatchNorm	11.4
bottleneck (Inception, ResNet)	8.3
Box<dyn Error>	1
broadcasting, broadcast_mul	3.3, 12.2
causal mask	14.5
CBAM	13.3
channel shuffle (ShuffleNet)	9.3
checkerboard artifacts	7.6
contiguous()	3.4, 14.5
conv2d argument order	7.1, 10.3
Conv2dConfig, cudnn_fwd_algo	8.1
conv_transpose2d, ConvTranspose2dConfig	7.3, 7.5
cross-attention	14.3
cross-correlation versus convolution	7.1
cross-entropy, binary cross-entropy	6.5
dense prediction	10.1
depthwise convolution, depthwise separable	9
detach	5.5, 7.4
devices (CPU, CUDA, Metal)	3.5
Dilated Residual Networks	10.6
dilation, effective kernel size	10.3
dot product	5.1
downcast_ref	1.2
DType, WithDType	2.2, 2.3
ECA-Net	13.3
error enum, thiserror	1.6
flatten_from	3.4, 14.7
fractionally strided convolution	7.5
freezing parameters	4.9
fusion of convolution and BatchNorm	12
gather	6.5
global average pooling	8.3, 8.4
gridding	10.4
grouped convolution	9
head specialisation and pruning	14.4
heads, split_heads	14.4, 14.5

Term	Sections
hybrid dilated convolution (HDC)	10.4
im2col, implicit GEMM	7.7
Init values, vb.get default (zeros)	4.3
inverted residual (MobileNetV2)	8.3
is_variable	3.1, 4.4
layer normalisation	11.5, 14.6
Linear, weight layout (out, in)	3.2
load, save (VarMap)	4.7
log-sum-exp, log_softmax	6.5
masks (padding, causal)	14.5
matmul	3.2
MLP (multi-layer perceptron)	5
ModuleT, forward_t	11.2
multi-head attention	14.4
Network in Network	8.3
output-size formulas	7.5, 10.3, 10.7
padding ("same")	10.3
parameter counting	5.3, 8.2, 9.2, 13.2, 14.5
permutation invariance	15.1
positional encoding (sinusoidal, learned, rotary)	15
pp, prefixes	4.2
PSA (YOLOv10)	16
query, key, value	14.1
receptive field	10.2
Result types	1.1
rows-are-items convention	3.2
Send + Sync errors	1.4
SGD	4.6, 5.4
softmax, safe softmax	6
Squeeze-and-Excitation	13
to_vec1, to_vec2, to_scalar	2.2
Toeplitz matrix	7.2, 7.3
tracing spans	3.6
Transformer block, pre-LN and post-LN	14.6
transposed convolution	7
turbofish	2
U-Net, Swin	10.5

Term	Sections
Var, Var:: <from_tensor, set<="" td=""><td>4.1, 4.4</td></from_tensor,>	4.1, 4.4
VarBuilder constructors	4.8
VarMap	4.1, 4.2
WaveNet	10.5
weight standardisation	11.5
where_cond	14.5
YOLOv10	16